

Relaxations and Exact Solutions to Quantum Max Cut via the Algebraic Structure of Swap Operators

Adam Bene Watts¹, Anirban Chowdhury¹, Aidan Epperly², J. William Helton², and Igor Klep^{3,4}

¹University of Waterloo

²University of California San Diego

³Faculty of Mathematics and Physics, University of Ljubljana

⁴Institute of Mathematics, Physics and Mechanics, Ljubljana, Slovenia

The Quantum Max Cut (QMC) problem has emerged as a test-problem for designing approximation algorithms for local Hamiltonian problems. In this paper we attack this problem using the algebraic structure of QMC, in particular the relationship between the quantum max cut Hamiltonian and the representation theory of the symmetric group.

The first major contribution of this paper is an extension of non-commutative Sum of Squares (ncSoS) optimization techniques to give a new hierarchy of relaxations to Quantum Max Cut. The hierarchy we present is based on optimizations over polynomials in the qubit swap operators. This is in contrast to the “standard” quantum Lasserre Hierarchy, which is based on polynomials expressed in terms of the Pauli matrices. To prove correctness of this hierarchy, we exploit a finite presentation of the algebra generated by the qubit swap operators. This presentation allows for the use of computer algebraic techniques to manipulate and simplify polynomials written in terms of the swap operators, and may be of independent interest. Surprisingly, we find that level-2 of this new hierarchy is numerically exact (up to tolerance 10^{-7}) on all QMC instances with uniform edge weights on graphs with at most 8 vertices.

The second major contribution of this paper is a polynomial-time algorithm that computes (in exact arithmetic) the maximum eigenvalue of the QMC Hamiltonian for certain graphs, including graphs that can be “decomposed” as a signed combination of cliques. A special case of the latter are complete bipartite graphs with uniform edge-weights, for which exact solutions are known from the work of Lieb and Mattis [LM62]. Our methods, which use representation theory of the symmetric group, can be seen as a generalization of the Lieb-Mattis result.

Adam Bene Watts: adam.benewatts1@uwaterloo.ca

Anirban Chowdhury: anirban.chnarayanchowdhury@uwaterloo.ca

Aidan Epperly: aepperly@ucsd.edu

J. William Helton: helton@math.ucsd.edu

Igor Klep: igor.klep@fmf.uni-lj.si

Contents

1	Intro	3
1.1	Results	6
1.2	Overview of Techniques	7
1.3	Reader's Guide	8
2	Quantum Max Cut and The Swap Matrix Algebra	9
2.1	Pauli Matrices and the Quantum Max Cut Hamiltonian	9
2.2	Representations of the Symmetric Group	10
2.2.1	Preliminary definitions	10
2.2.2	Irreducible representations of S_n	11
2.3	Swap Matrices and Permutations	11
2.4	Quantum Max Cut and Irreps	13
2.4.1	Maximizing irreps for 7 and 8 vertex graph Hamiltonians	15
2.5	Exact Arithmetic Solution for Clique Hamiltonians	16
2.6	Identities Satisfied by Swap Matrices	19
3	The Symbolic Swap Algebra	20
3.1	A Presentation of the Symbolic Swap Algebra	21
3.2	Relationship Between Symbolic Swap Algebra and Swap Matrix Algebra . .	22
3.3	The Irrep Symbolic Swap Algebra	26
3.4	Structure of Swap Variable Polynomials	26
3.4.1	Degree reducing polynomials in $\mathcal{I}^{\text{swap}}$	26
3.4.2	Useful graph properties	28
3.4.3	Monomials associated to their support graphs	28
3.4.4	Polynomials in swap variables	29
4	The Swap Algebra and the Non-Commutative Sum of Squares Hierarchy	30
4.1	A Monomial Order – grlex	30
4.2	Non-Commutative Sum Of Squares Hierarchy	30
4.2.1	Finding Linear Constrains in the Moment Matrix	35
4.3	Non-Commutative Sum Of Squares Hierarchy for the Swap algebra	36
4.3.1	First relaxation for the Swap algebra	36
4.3.2	Linear space spanned by the products of at most two swap matrices . .	37
4.3.3	Second relaxation for the Swap algebra	39
4.3.4	Swap relaxation behaves well in experiments	39
4.3.5	Segue	40
5	Gröbner Bases	40
5.1	Our Gröbner Basis Set Up	40
5.1.1	Motivation for why we need Gröbner generators vs any old set of generators for an ideal	40
5.2	Gröbner Bases for the Paulis	42
5.2.1	The d th relaxation of the quantum max-cut can be computed in polynomial time	43
5.3	Gröbner Bases for the Swap Algebra	44
5.4	d th Swap Relaxation	45
5.5	Irrep Swap Relaxations	46
5.5.1	Using the Gröbner basis for swaps	47

5.5.2	Using the Gröbner basis for irreps	47
6	Finding Eigenvalues via Clique Decompositions of Graphs	48
6.1	The Star Graph	49
6.2	Graph Clique Decomposition and Simpler Hamiltonians	50
6.2.1	The Tree Clique Decomposition	50
6.2.2	Clique decompositions and Hamiltonians	53
6.2.3	Example of clique decomposition, Algorithm 6	54
6.3	Computing Eigenvalues from Clique Decomposition	55
6.4	Example of Clique Decomposition, its Hamiltonians and their Eigenvalues .	59
6.4.1	The tree clique decomposition of G and the Hamiltonian decomposition	60
6.4.2	Eigenvalues	61
	Bibliography	62
A	Identities for the Proof of Lemma B.1	67
B	Linear space spanned by the products of at most three or four swap matrices	78
B.1	Linear space spanned by the products of at most three swap matrices . . .	79
B.2	Linear space spanned by the products of at most four swap matrices	80
C	Gröbner basis for the Swap algebra $\mathcal{A}_4^{\text{swap}}$	82
C.1	Gröbner bases for irreps $[4 - k, k]$	83
D	Example 4.8²	83
E	Comparison with [TRZ⁺]	85

1 Intro

The local Hamiltonian problem plays a central role in quantum complexity theory. Given as input a $2^n \times 2^n$ Hermitian matrix H , the goal of the problem is to compute its maximum (or minimum) eigenvalue. The matrix acts on n qubits and can be concisely specified as a sum of *local* terms,

$$H = \sum_{\substack{S \subseteq \{1, \dots, n\} \\ |S|=k}} H_S,$$

where each H_S acts non-trivially on at most a constant k number of qubits. We call such an H a k -local Hamiltonian.

The k -local Hamiltonian problem for any $k \geq 2$ is known to be hard for the complexity class QMA [[KSV02](#), [KKR06](#)], the quantum analog of NP. Thus, it is unlikely to be efficiently solvable in general. However, specific instances of the local Hamiltonian problem may be easy, in the sense that they admit exact arithmetic solutions [[LM62](#)], or polynomial-time algorithms to approximate the maximum eigenvalue to high precision [[LVV15](#)]. In other cases, it is interesting to study the best approximation to the maximum eigenvalue attainable in polynomial time. A number of works have, for example, provided algorithms to obtain constant-factor approximations to the maximum eigenvalue of local Hamiltonians [[GK12](#), [BH13](#), [BGKT19](#), [HM17](#)]. These can be viewed as quantum analogues of results about approximation algorithms for NP-hard problems and the hardness of approximation.

Particularly relevant to us are approaches that address the local Hamiltonian problem using the tools of semidefinite programming (SDP) relaxations and noncommutative polynomial optimization. Indeed the local Hamiltonian problem can be formulated as a maximization problem, where the goal is to compute $\text{eig}_{\max}(H) = \max_{\rho} \text{tr}(\rho H)$ subject to $\rho \succeq 0$, $\text{tr}(\rho) = 1$. This defines an SDP involving exponentially large matrices and thus becomes intractable as system size grows. However, one can define a hierarchy of efficiently computable SDP *relaxations* which provides upper bounds to the maximum eigenvalue [BH13, BGKT19, GP19, PT21, HO22]. This is in contrast to variational methods using Ansätze such as tensor networks which give lower bounds by approximating the maximum-eigenvalue eigenstate from within a subset of the set of quantum states [CPGSV21, BC17]. SDP relaxations, on the other hand, give *outer* approximations to quantum optimization problems – their solutions may not correspond to actual quantum states. These relaxations are efficiently computable in the sense that at any fixed level of the hierarchy one can compute the SDP value in polynomial time.

Quantum Max Cut We will focus on the local Hamiltonian problem defined on a specific family of 2-local Hamiltonians called Quantum Max Cut (QMC) Hamiltonians. (We sometimes simply call this problem Quantum Max Cut.) Each Quantum Max Cut Hamiltonian is defined with respect to a graph G on n vertices with edge set $E(G)$ and for weights w_{ij} as

$$H_G = \sum_{(i,j) \in E(G)} w_{ij} \left(I - \sigma_X^i \sigma_X^j - \sigma_Y^i \sigma_Y^j - \sigma_Z^i \sigma_Z^j \right), \quad (1.1)$$

where σ_X , σ_Y and σ_Z are the Pauli matrices, and $\sigma_W^i = \bigotimes_{j=1}^{i-1} I \otimes \sigma_W \otimes \bigotimes_{j=i+1}^n I$ for $W \in \{X, Y, Z\}$. Equivalently, H_G can be written in terms of qubit SWAP operators, $\text{Swap}_{ij} = \frac{1}{2} \left(I + \sigma_X^i \sigma_X^j + \sigma_Y^i \sigma_Y^j + \sigma_Z^i \sigma_Z^j \right)$, as

$$H_G = \sum_{(i,j) \in E(G)} 2w_{ij} (I - \text{Swap}_{ij}). \quad (1.2)$$

Quantum Max Cut was introduced in the context of approximation algorithms by Gharibian and Parekh [GP19] as a quantum generalization of the well-known Max Cut problem. The Hamiltonian H_G has also been studied extensively in physics as the antiferromagnetic quantum Heisenberg model. Determining the maximum eigenvalue of H_G for an arbitrary graph G is known to be the QMA-hard [PM17], and the Quantum Max Cut problem has received significant attention as a test-bed for designing approximation algorithms to solve QMA-hard problems [AGM20, PT21, PT22, Kin23, Lee22]. Additionally, exact arithmetic solutions are known in a few cases, such as for complete bipartite graphs [LM62] and one-dimensional chains [LM16].

Max Cut and Sum-of-Squares relaxations As suggested by the name, Quantum Max Cut is closely related to the classical problem of Max Cut where, given a graph G with edge-set $E(G)$ and weights $w_{ij} > 0$, the goal is to maximize the objective $\sum_{(i,j) \in E(G)} w_{ij} \frac{1 - z_i z_j}{2}$ over assignments of the variables $z_i \in \{+1, -1\}$ for $i = 1, \dots, n$. Determining the optimum value of the Max Cut objective is NP-hard, but nontrivial upper-bounds on it can be obtained from the Lasserre or Sum-of-Squares (SOS) hierarchy of SDP relaxations. Level d of this hierarchy involves optimizing over “pseudo-expectations” – linear functionals μ which are defined on polynomials of degree at most $2d$ in the z_i variables and which satisfy

$\mu(1) = 1$ and $\mu(f^2) \geq 0$ for any polynomial f with degree at most d .¹ The set of pseudo-expectations contains the set of “actual” expectation values which can be obtained from probability distributions assigning values to the variables z_i , and converges to this set as $d \rightarrow \infty$. For this reason, the SoS hierarchy gives a converging series of upper bounds on the true value of the Max Cut problem. Additionally, for any constant d , the level d SoS upper bound can be found in polynomial time, though in practice this optimization quickly becomes infeasible as d grows large.

The best-known approximation algorithm (i.e., constructive lower bound algorithm) for Max Cut is due to Goemans and Williamson and can be viewed as an algorithm which “rounds” the pseudo-expectation produced by a level 1 Lasserre relaxation to an actual expectation [GW95]. In general, Lasserre relaxations in conjunction with rounding have been powerful techniques for upper bounding and approximating hard constraint satisfaction problems (CSPs) [BRS11]. In fact, assuming the Unique Games Conjecture (UGC) of Khot [Kho02], Raghavendra [Rag08, Rag09] showed that it is NP-hard to outperform a canonical approximation algorithm based on rounding from the first level of the SoS hierarchy for any classical constraint satisfaction problem.

Quantum Lasserre hierarchy The success of SDP relaxations for classical CSPs has motivated their use in approximations to local Hamiltonian problems [BH13, BGKT19, HO22] and in particular Quantum Max Cut [GP19, PT21, PT22, Kin23, KPS18]. Formulating these quantum relaxations is non-trivial and builds on results in non-commutative polynomial optimization [HM04] and its application to problems in quantum information [DLTW08, NPA08].

The quantum Lasserre relaxation for Quantum Max Cut [GP19, PT22] was defined with respect to the n -qubit Pauli algebra. A solution to the level- d relaxation assigns pseudo-expectation values to operators corresponding to Pauli polynomials, i.e., polynomials of degree at most $2d$ in the non-commuting variables $\{X_i, Y_i, Z_i\}$ for $i = 1, \dots, n$. In analogy with the classical situation, the pseudo-expectation of the identity is fixed to be 1 and, for any Pauli polynomial P with degree at most d , the pseudo-expectation of its square must be non-negative. The relaxation can be viewed as relaxing from operator algebraic states (positive linear functionals) on the n -qubit Pauli algebra to pseudo-states that are positive only with respect to squares of degree- d polynomials.

In contrast to the classical case, the relationship between the quantum Lasserre Hierarchy and approximation algorithms for local Hamiltonian problems is much less well understood. In [GP19], Gharibian and Parekh give an approximation algorithm for Quantum Max Cut based on a rounding from the first level of the quantum Lasserre hierarchy. Later works then gave algorithms which rounded from the second level of the hierarchy and lead to better approximation ratios [PT21, PT22]. In [Lee22], a modified Lasserre relaxation was introduced, which considered only a subset of degree-2 polynomials sufficient to recover the Quantum Max Cut objective. Assuming the Unique Games Conjecture along with a technical conjecture known as the vector valued Borel’s inequality, the authors of [HNP⁺22] showed the second level of the quantum Lasserre hierarchy (and therefore also approximation algorithms building off the second level) could strictly outperform the first level. Indeed, the second level of the Lasserre hierarchy has been shown to be exact for certain graphs such as the unweighted star, and capture physically meaningful prop-

¹There is some inconsistency in the literature between d and $2d$ when discussing levels of the SoS hierarchy. Here we use the convention that level d of this hierarchy involves optimizing over linear functionals which are positive on squares of degree d polynomials; note these squares may include terms of degree up to $2d$.

erties of quantum states such as monogamy of entanglement inequalities [PT21]. These properties have been used to improve the analysis of rounding algorithms [PT21, PT22].

1.1 Results

The first major contribution of this paper is to extend non-commutative Sum of Squares (ncSoS) optimization techniques to give a new hierarchy of relaxations to the problem of computing the maximum eigenvalue of the QMC Hamiltonian. This hierarchy is a non-commutative sum-of-squares hierarchy (ncSoS) which captures the algebraic structure arising from the swap operators Swap_{ij} which define the QMC Hamiltonian objective.

A critical ingredient in our construction of the SDP hierarchy is the ability to manipulate polynomials in the swap operators and enforce polynomial identities. Towards this end, we give a fully algebraic characterization (i.e., finite presentation) of the swap operators. We note that this presentation follows quickly as a special case of results in representation theory, cf. [Pro07, §6.1 Theorem]² but provide a standalone proof for the sake of completeness. This characterization is necessary for the use of “dimension free” computer algebraic techniques (e.g. Gröbner bases) for manipulating polynomials in swap operators. That is to say, we do not need to use the matrix representations of the swap operators which could be exponentially large. We develop swap algebra theory including replacement rules useful for identifying polynomial identities in the swaps. These results may be of independent interest, beyond their use in constructing higher levels of the SDP hierarchy in swaps.

This new hierarchy of relaxations to Quantum Max Cut is distinct from, and may sometimes outperform, the usual quantum Lasserre relaxations over Paulis. We show that this hierarchy is exact on an n -vertex graph at the level $\lceil \frac{n}{2} \rceil$, no higher levels are needed – a consequence of the previously discussed identities for simplifying swap polynomials. For instance, the first level of this hierarchy enforces relations among the QMC terms which are not enforced by the first level of the quantum Lasserre relaxation [PT21]. Moreover, tests on small instances indicate that our hierarchy can provide tighter bounds to Quantum Max Cut compared to the level-1 and level-2 quantum Lasserre relaxations over Paulis. Specifically, we find that level-2 of this new hierarchy gives a numerically exact upper bound on the QMC problem for all ≤ 8 vertex graphs with uniform edge weights. Of course, these small instances can be solved by diagonalization in practice. But our numerical results suggest that the swap hierarchy can give fairly tight upper bounds on the value of the Quantum Max Cut problem. Thus, it is possible that a better theoretical understanding of the performance of the swap hierarchy can lead to improved approximation guarantees for Quantum Max Cut. We leave such an analysis for future work.

The second major contribution of this paper is a polynomial-time algorithm that computes in exact arithmetic the maximum eigenvalue of the QMC Hamiltonian for certain graphs, including graphs that can be “decomposed” as a signed combination of cliques. A special case of the latter are complete bipartite graphs with uniform edge-weights, for which exact arithmetic solutions are known from the work of Lieb and Mattis [LM62]. The Lieb-Mattis solutions use angular momentum algebra to identify invariant subspaces of the QMC Hamiltonian. Our methods, which use representation theory of the symmetric group, can be seen as a generalization of that of Lieb and Mattis.

In closing, we alert the reader that some care has been taken in this paper to distinguish between various notions of exactness. When discussing numerical results we use the phrase *numerically exact* to mean 10^{-7} accuracy. When discussing theoretical results we

²[Pro07, §6.1] applies to swaps on qudits, though here we only work with qubits.

distinguish algorithms which are *exact in theory* (meaning they give a provably correct output up to floating point precision) from the stricter notion of algorithms which provide as output an *exact arithmetic* representation of the solution.

Note While preparing this manuscript, we became aware of contemporaneous work [TRZ⁺] which establishes related results. The main commonality is that both papers construct hierarchies of semidefinite programs based on the swap matrices which give convergent series of upper bounds to the maximum eigenvalue of the QMC Hamiltonian. However, there are significant technical differences between the hierarchies constructed here and in [TRZ⁺], cf. Appendix E. More importantly, we focus on developing the algebraic machinery to construct a hierarchy in the swaps, whereas [TRZ⁺] focuses on understanding the performance of low levels of SDP hierarchies for Quantum Max Cut and a number of other many-body physics problems. While both papers in addition touch upon exact solvability of Quantum Max Cut, the results are quite distinct. We provide exact arithmetic solutions using representation theory of the symmetric group whereas [TRZ⁺] theoretically prove exactness of SDP relaxations for certain graphs. We refer the reader to Appendix E for a detailed comparison between the two papers.

1.2 Overview of Techniques

We prove both of these results by exploiting a connection between the QMC Hamiltonian and the representation theory of the symmetric group. More formally, we use the fact that the QMC Hamiltonian can be written as a sum of swap matrices, which can, in turn, be viewed as representations of the group algebra of the symmetric group. This representation can be understood (in particular, the irreducible representations, or *irreps*, appearing in this representation can be characterized) via Schur-Weyl duality, and plays a key role in deriving several important theoretical results in quantum information [Har13].

To construct a hierarchy of semidefinite programming relaxations specific to these swap matrices, we introduce an abstract $\ast$ -algebra, which we call the *Symbolic Swap Algebra*. We give a finite presentation for this algebra, and then, using the characterization of the swap matrices given by Schur-Weyl, we explain that this algebra is isomorphic to the one generated by the swap matrices, thereby establishing a special case of [Pro07, Theorem, §6.1]. Informally, this means that the Symbolic Swap Algebra captures the full behavior of the swap matrix algebra. Finally, we show how standard non-commutative optimization techniques (in particular, the non commutative sum of squares, or ncSoS hierarchy) can be specified to this $\ast$ -algebra to bound the eigenvalues of Quantum Max Cut Hamiltonians.

An important complexity in this process comes from the necessity of finding all linear relationships between polynomials in the symbolic swap algebra (or equivalently, finding a system for simplifying expressions written in terms of swap matrices). We solve this problem in a two ways, depending on the degree of the polynomials considered. First, for any integer n , we show how to construct a set of linearly independent polynomials which span all polynomials of degree ≤ 4 in the n -qubit symbolic swap algebra. This lets us simplify all terms appearing when running the ncSoS hierarchy up to level two in the symbolic swap algebra. To simplify terms appearing in higher levels of the hierarchy, we connect to the theory of non-commutative Gröbner bases and show that this lets us find, in principle, systems of rewrite rules for any swap matrix polynomials of fixed finite degree in polynomial time. A byproduct of our simplification theory is that to any polynomial p we can associate a “simpler” polynomial q with the same values on swap matrices.

The polynomial time algorithm for solving in exact arithmetic some QMC Hamiltonians (and simplifying others) is obtained by building on standard techniques for studying

representations of the symmetric group. One important concept we introduce to connect to these techniques is that of the *Quantum Max Cut Irrep Hamiltonian* – defined to be the matrix obtained by replacing the swap matrices appearing in the standard Quantum Max Cut Hamiltonian with the representation of equivalent permutations inside some irrep of the symmetric group. For any irrep, we then show how an exact arithmetic representation of the eigenvalues of the QMC Irrep Hamiltonian associated with a clique can be found in polynomial time using Schur’s Lemma and the Murnaghan-Nakayama rule. (This, combined with the characterization of the swap matrix algebra irreps given by Schur-Weyl, lets us reproduce a well known characterization of eigenvalues of the QMC Hamiltonian associated with a clique.)

Then we consider graphs whose associated QMC Hamiltonians decompose as a signed sum of cliques, and show how the previously discussed exact arithmetic solutions to the clique irrep Hamiltonians coupled with Young’s branching rule can be used to find the k max and min eigenvalues of these Hamiltonians. When graphs do not decompose entirely into a signed sum of cliques, we show similar techniques can be used to bound the max and min eigenvalues of these graphs in terms of the max and min eigenvalues of smaller “residual” graphs.

1.3 Reader’s Guide

In Section 2 we introduce some basic definitions related to the Quantum Max Cut problem, swap matrices, and the representation theory of the symmetric group algebra. We then introduce the concept of Quantum Max Cut Irrep Hamiltonians and give some basic results concerning them, including an exact arithmetic formula for the single (integer) eigenvalue of any Quantum Max Cut Irrep Hamiltonian associated with a clique.

In Section 3 we introduce the symbolic swap algebra, give a presentation for it, and show it is isomorphic to the swap matrix algebra. We then prove some basic properties about polynomials in this algebra, to include how one produces a “simple” polynomial q equivalent to a given polynomial p evaluated on swaps.

In Section 4 we discuss how the non-commutative Sum of Squares (ncSoS) algorithm can be applied to the symbolic swap algebra to produce a semidefinite programming hierarchy analogous to the Quantum Lasserre Hierarchy. We also construct an explicit linear algebra basis for polynomials of degree ≤ 4 in the symbolic swap algebra. This shows the first two levels of the swap-ncSoS hierarchy can be run in polynomial time.

Then, in Section 5, we discuss the theory of Gröbner bases and show it is possible to find rewrite rules for polynomials of any fixed degree in the swap variables. This lets us conclude that any finite level of the swap-ncSoS hierarchy can run in, in principle, polynomial time.

In Section 6 we discuss graphs for which we can compute, in exact arithmetic, any constant number of max and min eigenvalues associated to the QMC Hamiltonian using representation theoretic techniques. We give an algorithm which identifies these graphs, then we show how to compute their eigenvalues (or bound the eigenvalues of other graphs). Finally, we give some simple examples of these algorithms in practice.

There are 3 online appendices. Appendix A contains relationships on Swaps, all used for proofs in the paper. Appendix C gives some examples and properties of Gröbner bases for ideals involving Swaps. Finally, in Appendix E we compare and contrast this paper with the independent and simultaneously released paper [TRZ⁺].

Acknowledgements

The authors thank Claudio Procesi for bringing the notion of a “good permutation” to their attention, and for sharing his expertise in invariant and representation theory. IK thanks Jurij Volčič for discussions. ABW and AC thank Ojas Parekh for discussions and in particular pointing out the connection between quantum max cut and swap matrices. AC acknowledges discussions with Kevin Thompson. ABW and AC also thank William Slofstra and David Gosset for helpful discussions. Additionally, the authors wish to thank Brendon Rhoades for discussions and creating an alternative proof of Theorem 3.6. The authors also thank anonymous referees for helpful and insightful comments. ABW and AC acknowledge the support of the Natural Sciences and Engineering Research Council of Canada through grant number RGPIN-2019-04198. IK was supported by the Slovenian Research Agency program P1-0222 and grants J1-50002, J1-2453, N1-0217, J1-3004. IK’s work was performed within the project COMPUTE, funded within the QuantERA II Programme that has received funding from the EU’s H2020 research and innovation programme under the GA No 101017733 .

Code Availability

The code used to obtain the numerical results presented in this paper is available on request.

2 Quantum Max Cut and The Swap Matrix Algebra

In this section we explore a connection between the the Quantum Max Cut Hamiltonian discussed in the introduction and the representation theory of the symmetric group. We begin with a brief review of representation theory, particularly concerning irreducible representations (irreps) of the symmetric group. Then, we consider the algebra generated by the swap matrices, which we call the Swap Matrix Algebra, and which turns out to be a representation of the symmetric group algebra. We note that (as has been pointed out previously [Os06, PT21]) the QMC Hamiltonian lies inside this algebra. Connecting these ideas leads to the notation of the QMC Hamiltonian “inside” of an irrep. Finally, we show how combining all these ideas with some standard results in representation theory can be used to give a closed form expression for *all* eigenvalues of the QMC Hamiltonian on a clique with uniform edge weights.

2.1 Pauli Matrices and the Quantum Max Cut Hamiltonian

Recall the three Pauli matrices,

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.1)$$

Together with $\sigma_I := I$ these form a basis for $M_2(\mathbb{C})$. They satisfy the following relations:

$$\sigma_X^2 = \sigma_Y^2 = \sigma_Z^2 = I, \quad \sigma_X \sigma_Y = i \sigma_Z, \quad \sigma_Y \sigma_X = -i \sigma_Z. \quad (2.2)$$

Fix $n \in \mathbb{N}$. For $W \in \{I, X, Y, Z\}$ we shall also consider the matrices

$$\sigma_W^j := \underbrace{I \otimes \cdots \otimes I}_{j-1} \otimes \sigma_W \otimes I \otimes \cdots \otimes I \in M_{2^n}(\mathbb{C}). \quad (2.3)$$

Observe that

$$\{\sigma_{W_1}^1 \sigma_{W_2}^2 \cdots \sigma_{W_n}^n \mid W_j \in \{I, X, Y, Z\}, j = 1, \dots, n\} \quad (2.4)$$

is a basis of $M_{2^n}(\mathbb{C})$. Further, given $i \neq j$,

$$[\sigma_{W_i}^i, \sigma_{W_j}^j] = 0,$$

where $[\cdot, \cdot]$ denotes the additive commutator

$$[a, b] := ab - ba.$$

Let $\mathbb{G}^n$ denote the set of all graphs on n vertices. Let $E(G)$ denote its set of edges, and let $V(G)$ denote its set vertices.

Definition 2.1. *Given a graph G with vertex set $V(G)$, edge set $E(G)$ and edge weights $\{w_{ij} > 0 \mid (i, j) \in E(G)\}$, the Quantum Max Cut (QMC) Hamiltonian is defined to be*

$$H_G = \sum_{(i,j) \in E(G)} w_{ij} \left(I - \sigma_X^i \sigma_X^j - \sigma_Y^i \sigma_Y^j - \sigma_Z^i \sigma_Z^j \right). \quad (2.5)$$

While the right-hand side of Eq. (2.5) clearly depends on the weights w_{ij} , we suppress this dependence in the notation H_G .

2.2 Representations of the Symmetric Group

In this section we review standard facts about representations of the symmetric group which will be necessary to prove our results.

2.2.1 Preliminary definitions

For any finite-dimensional vector space V , we use $\text{GL}(V)$ to denote the group of invertible linear transformations from V to itself. We use S_n to denote the symmetric group, the group of permutations of n objects. We will specify elements group using cycle notation, with e denoting the identity element. Recall that a representation of S_n is a group homomorphism $\rho : S_n \rightarrow \text{GL}(V)$. The vector space V is also referred to as an S_n -module or simply a module.

The *group algebra* $\mathbb{C}[S_n]$ can be defined as an S_n -module as follows. Promote the elements $\{\pi_1, \pi_2, \dots, \pi_{n!}\} \in S_n$ to basis vectors $\pi_1, \pi_2, \dots, \pi_{n!}$ with the multiplication rule $\pi_i \pi_j = \pi_k$ if $\pi_i \pi_j = \pi_k$. Then $\mathbb{C}[S_n]$ is given by

$$\mathbb{C}[S_n] = \{c_1 \pi_1 + c_2 \pi_2 + \cdots + c_{n!} \pi_{n!} \mid c_j \in \mathbb{C}\} \quad (2.6)$$

where $c_i \in \mathbb{C}$ for all i , and S_n acts on $\mathbb{C}[S_n]$ by left-multiplication. A representation (ρ, V) of S_n also gives rise to a *representation of the algebra* $\mathbb{C}[S_n]$ via the homomorphism $\tilde{\rho} : \mathbb{C}[S_n] \rightarrow \mathbb{C}[\text{GL}(V)]$ defined by its action

$$\tilde{\rho} \left(\sum_{i=1}^{n!} c_i \pi_i \right) = \sum_{i=1}^{n!} c_i \rho(\pi_i). \quad (2.7)$$

It is common to use ρ to refer to the representation of both the group and its group algebra.

A S_n -module in general decomposes into a number of S_n -submodules. A module V is *irreducible* if the only submodules of V are itself and the trivial module $\{0\}$. For a

representation (ρ, V) of any finite group, it follows from Maschke's Theorem that there exists a decomposition of V as

$$V = \bigoplus_{\lambda} V_{\lambda} \quad (2.8)$$

where each V_{λ} is an irreducible module.

We will be interested in matrix representations ρ of S_n where V is $(\mathbb{C}^2)^{\otimes n}$, which is to say that $\rho(\pi)$ is a $2^n \times 2^n$ matrix for each $\pi \in S_n$. Maschke's Theorem then implies that there is an invertible matrix U such that

$$\rho(\pi) = U \left[\bigoplus_{\lambda} \rho_{\lambda}(\pi) \right] U^{-1}, \quad \pi \in S_n, \quad (2.9)$$

where ρ_{λ} are irreducible representations or irreps of S_n . The decomposition of Eq. (2.8) into irreducible modules can thus be seen as a *block-diagonalization* of matrices in $\rho(\mathbb{C}[S_n]) \subseteq M_{2^n}(\mathbb{C})$.

2.2.2 Irreducible representations of S_n

Irreducible representations of S_n are in one-to-one correspondence with integer partitions of n ,

$$\lambda = [\lambda_1, \lambda_2, \dots, \lambda_k], \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k > 0, \quad \sum_{i=1}^k \lambda_i = n \quad (2.10)$$

which we will denote as $\lambda \vdash n$. It is convenient to associate the partition λ with the *Young diagram* of shape λ , which consists of k rows indexed top to bottom such that the i -th row contains λ_i boxes. As an example, the partition $[3, 2]$ corresponds to the shape

$$\begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \quad (2.11)$$

Of crucial importance to us later in the paper will be those irreps $\rho_{[n-k, k]}$ that correspond to two row Young diagrams and the respective modules $V_{[n-k, k]}$, defined for $k = 0, \dots, \lfloor \frac{n}{2} \rfloor$.

2.3 Swap Matrices and Permutations

In order to analyze the QMC Hamiltonian, we introduce the *swap matrices* Swap_{ij} as permutations on n -qubit states.

Definition 2.2. *The swap matrices Swap_{ij} are defined by their action on tensor products of $|\psi_1\rangle, |\psi_2\rangle, \dots, |\psi_n\rangle \in \mathbb{C}^2$ as follows:*

$$\begin{aligned} \text{Swap}_{ij} \left(|\psi_1\rangle \otimes \dots \otimes |\psi_i\rangle \otimes \dots \otimes |\psi_j\rangle \otimes \dots \otimes |\psi_n\rangle \right) \\ = |\psi_1\rangle \otimes \dots \otimes |\psi_j\rangle \otimes \dots \otimes |\psi_i\rangle \otimes \dots \otimes |\psi_n\rangle. \end{aligned} \quad (2.12)$$

The connection to Quantum Max Cut is made by noticing that the Swap_{ij} is the following linear combination of the Pauli matrices

$$\text{Swap}_{ij} = \frac{1}{2} \left(I + \sigma_X^i \sigma_X^j + \sigma_Y^i \sigma_Y^j + \sigma_Z^i \sigma_Z^j \right). \quad (2.13)$$

Proposition 2.3. *The QMC Hamiltonian H_G of Eq. (2.5) in terms of the Swap_{ij} 's defined in Eq. (2.12) is given by*

$$H_G = \sum_{(i,j) \in E(G)} 2w_{ij} (I - \text{Swap}_{ij}) \quad (2.14)$$

Now, observe that we can write the right-hand side of Eq. (2.12) as

$$|\psi_{(i\ j)^{-1}(1)}\rangle \otimes \cdots \otimes |\psi_{(i\ j)^{-1}(i)}\rangle \otimes |\psi_{(i\ j)^{-1}(j)}\rangle \otimes \cdots \otimes |\psi_{(i\ j)^{-1}(n)}\rangle, \quad (2.15)$$

where $(i\ j) \in S_n$ is the transposition of objects i and j . This is in fact a representation of S_n on $(\mathbb{C}^2)^{\otimes n}$ defined by

$$\rho_n(\pi) (|\psi_1\rangle \otimes \cdots \otimes |\psi_n\rangle) = |\psi_{\pi^{-1}(1)}\rangle \otimes \cdots \otimes |\psi_{\pi^{-1}(n)}\rangle \quad (2.16)$$

Definition 2.4. *The algebra $\rho_n(\mathbb{C}[S_n])$, for the representation $(\rho_n, (\mathbb{C}^2)^{\otimes n})$ in Eq. (2.16), is called the Swap Matrix Algebra M_n^{swap} .*

Example 2.5. For small n , the swap algebra M_n^{swap} can be explicitly determined. Firstly, since the swap matrices Swap_{ij} are defined in terms of a representation of S_n , there is a $*$ -homomorphism

$$\mathbb{C}[S_n] \rightarrow M_n^{\text{swap}}, \quad (i\ j) \mapsto \text{Swap}_{ij}, \quad (2.17)$$

whence M_n^{swap} is isomorphic to a semisimple quotient of the group algebra $\mathbb{C}[S_n]$ of the symmetric group S_n . Thus representation theory of S_n can be used to study M_n^{swap} . For instance, S_3 has two one-dimensional representations (the trivial one and the signature), and one two-dimensional one, hence

$$\mathbb{C}[S_3] \cong \mathbb{C} \oplus \mathbb{C} \oplus M_2(\mathbb{C}).$$

Since $\dim M_3^{\text{swap}} = 5$, this immediately yields

$$M_3^{\text{swap}} \cong \mathbb{C} \oplus M_2(\mathbb{C}).$$

Similarly, classifying irreducible representations for S_4 gives

$$\mathbb{C}[S_4] \cong \mathbb{C} \oplus \mathbb{C} \oplus M_2(\mathbb{C}) \oplus M_3(\mathbb{C}) \oplus M_3(\mathbb{C}),$$

which together with $\dim M_4^{\text{swap}} = 14$ implies that

$$M_4^{\text{swap}} \cong \mathbb{C} \oplus M_2(\mathbb{C}) \oplus M_3(\mathbb{C}).$$

The general characterization of the Swap Matrix Algebra follows from Schur-Weyl duality [EGH⁺11] of S_n and $\text{GL}_2(\mathbb{C})$, the group of invertible 2×2 complex matrices. The natural representation of $\text{GL}_2(\mathbb{C})$ on $(\mathbb{C}^2)^{\otimes n}$ is defined by the diagonal action of the group elements on tensor products of $|\psi_1\rangle, |\psi_2\rangle, \dots, |\psi_n\rangle \in \mathbb{C}^2$,

$$\zeta_n(g) (|\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle) = g |\psi_1\rangle \otimes g |\psi_2\rangle \otimes \cdots \otimes g |\psi_n\rangle, \quad g \in \text{GL}_2(\mathbb{C}). \quad (2.18)$$

The irreducible modules of $\text{GL}_2(\mathbb{C})$ are indexed by two row Young diagrams with an unbounded number of boxes. We denote the former by $L_{[n-k, k]}$ for $n \in \mathbb{N}$ and $k = 0, \dots, \lfloor \frac{n}{2} \rfloor$. In fact, $L_{[n-k, k]}$ is the space of all linear maps from $V_{[n-k, k]}$ to $(\mathbb{C}^2)^{\otimes n}$ that commute with the action of S_n , that is

$$L_{[n-k, k]} = \text{Hom}_{S_n}(V_{[n-k, k]}, (\mathbb{C}^2)^{\otimes n}). \quad (2.19)$$

The following lemma is essentially a restatement of Schur-Weyl duality for S_n and $\text{GL}_2(\mathbb{C})$.

Lemma 2.6. *The algebras M_n^{swap} and $\zeta_n(\mathbb{C}[\text{GL}_2(\mathbb{C})])$ are centralizers of each other inside $\text{End}((\mathbb{C}^2)^{\otimes n}) = M_{2^n}(\mathbb{C})$. Moreover, the space $(\mathbb{C}^2)^{\otimes n}$ decomposes under the action of the direct product $\text{GL}_2(\mathbb{C}) \times S_n$ as*

$$(\mathbb{C}^2)^{\otimes n} \cong \bigoplus_{k=0}^{\lfloor \frac{n}{2} \rfloor} L_{[n-k,k]} \otimes V_{[n-k,k]}. \quad (2.20)$$

Proof. See, e.g., [EGH⁺11, Sec. 5.19] or [Pro07, §6.1]. $\square$

Since the action of S_n on $L_{[n-k,k]}$ is trivial, as an S_n -module, the space $(\mathbb{C}^2)^{\otimes n}$ decomposes by Lemma 2.6 into irreducible modules $V_{[n-k,k]}$ with multiplicities as follows:

$$(\mathbb{C}^2)^{\otimes n} = \bigoplus_{k=0}^{\lfloor \frac{n}{2} \rfloor} (V_{[n-k,k]})^{\dim(L_{[n-k,k]})}. \quad (2.21)$$

The Weyl character formula [EGH⁺11, Theorem 5.22.1] gives an explicit formula for $\dim(L_{[n-k,k]})$.

Further, Lemma 2.6 immediately leads to the following characterization of M_n^{swap} in terms of the irreducible representations of $\mathbb{C}[S_n]$; cf. [Pro21, (1.12)].

Theorem 2.7. *The Swap Matrix Algebra decomposes into the direct sum of simple algebras generated by the two row irreps of the symmetric group. That is, we have*

$$M_n^{\text{swap}} \cong \bigoplus_{k=0}^{\lfloor \frac{n}{2} \rfloor} \rho_{[n-k,k]}(\mathbb{C}[S_n]). \quad (2.22)$$

Proof. This is immediate from Lemma 2.6 and Eq. (2.21). $\square$

2.4 Quantum Max Cut and Irreps

Now we discuss how the decomposition of the Swap Matrix Algebra into irreps described in the previous section can be applied to calculations related to the eigenvalues of Quantum Max Cut Hamiltonians. Among other benefits the calculations in this section are essential to Section 6.

Definition 2.8. *Let $\lambda \vdash n$ be any partition labeling an irrep of S_n , and let G be an n vertex graph with edge set $E(G)$ and edge weights w_{ij} . Then define the QMC irrep Hamiltonian H_G^λ by*

$$H_G^\lambda = \rho_\lambda \left(\sum_{(i,j) \in E(G)} 2w_{ij} (I - (i \ j)) \right) \quad (2.23)$$

We will frequently use the phrase “all two row irrep Hamiltonians of G ” to refer to the set of irrep Hamiltonians

$$\{H_G^{[n-k,k]} : 1 \leq k \leq \lfloor n/2 \rfloor\}. \quad (2.24)$$

Now we state a straightforward corollary of Theorem 2.7 which makes the significance of the two row irrep Hamiltonians clear.

Corollary 2.9. *The spectrum of the QMC Hamiltonian of G is given by the union of the spectra of all two row irrep Hamiltonians of G . That is*

$$\text{eigs}(H_G) = \bigcup_{k=0}^{\lfloor \frac{n}{2} \rfloor} \text{eigs}(H_G^{[n-k,k]}) \quad (2.25)$$

In particular, we have

$$\text{eig}_{\max}(H_G) = \max_{k=0}^{\lfloor \frac{n}{2} \rfloor} \left(\text{eig}_{\max}(H_G^{[n-k,k]}) \right) \quad (2.26)$$

Proof. Immediate from Theorem 2.7. $\square$

Corollary 2.9 suggests an immediate technique for computing the max eigenvalue of a QMC Hamiltonian – rather than computing the maximum eigenvalue of the matrix H_G directly we can instead compute the maximum eigenvalue of each of the $H_G^{[n-k,k]}$ matrices. When combined with brute force computation this yields a modest computational advantage, since the dimension of each $H_G^{[n-k,k]}$ matrix is smaller than that of H_G .³ In the next lemma we calculate these dimensions. It should be pointed out that the dimension of these irreps still scales exponentially with n , meaning this approach does not give a polynomial time algorithm for computing the max eigenvalue of H_G . In Section 2.4.1 we give the results of some simple experiments regarding which irreps provide the maximizing eigenvalue for QMC Hamiltonians.

We calculate the dimension of each irrep by computing the character (or trace) of the identity element in the irrep. When working with the symmetric group, we will use $e \in S_n$ denote the identity and $\chi_{[n-k,k]} : S_n \rightarrow \mathbb{C}$ denote the character of $\rho_{[n-k,k]}$, so $\chi_{[n-k,k]}(\pi) := \text{Tr}(\rho_{[n-k,k]}(\pi))$ for all $\pi \in S_n$. Then, in particular,

$$\chi_{[n-k,k]}(e) = \text{Tr}(\rho_{[n-k,k]}(e)) \quad (2.27)$$

is the dimension of the irrep $\rho_{[n-k,k]}$ of S_n .

Lemma 2.10.

$$\chi_{[n-k,k]}(e) = \frac{n-2k+1}{n-k+1} \frac{n_{[k]}}{k!} = \frac{n-2k+1}{n-k+1} \binom{n}{k}. \quad (2.28)$$

Here we used the falling factorial notation $n_{[k]}$, defined by

$$n_{[k]} := \frac{n!}{(n-k)!} = n(n-1) \cdots (n-k+1), \quad (2.29)$$

and $n_{[0]} = 0_{[k]} = 1$ by definition.

³This approach also requires computing the form of the $H_G^{[n-k,k]}$ matrix in some basis. This can be done, for example, by explicitly computing matrix representations of transpositions in the $[n-k,k]$ irrep via their action on the Specht basis (see [Jam06] for details), then summing these matrices to obtain $H_G^{[n-k,k]}$. While involved, all these operations have runtime polynomial in the dimension of the $[n-k,k]$ irrep. In practice, these computations can be also be performed using a convenient software package, for example [Fon20], which was the approach used in this paper.

Proof. The dimension of the irrep of the partition $[n-k, k]$ is equal to the trace, or character, of the identity element in this irrep, that is, $\chi_{[n-k, k]}(e)$. We compute this quantity using the well-known hook length formula. Straightforward inspection gives that the product of the hook lengths of this irrep is given by:

$$\prod_{ij} \text{hook}_\lambda(i, j) = (n-k+1)^{[k]} (n-2k)! k! = \frac{(n-k+1)! k!}{(n-2k+1)!}. \quad (2.30)$$

Then the dimension of the $[n-k, k]$ irrep is given by

$$\chi_{[n-k, k]}(e) = \frac{n!}{\prod_{ij} \text{hook}_\lambda(i, j)} = \frac{n-2k+1}{n-k+1} \frac{n_{[k]}}{k!} = \frac{n-2k+1}{n-k+1} \binom{n}{k}. \quad \square$$

From Lemma 2.10 we see that, for any constant k , the irreps $\rho_{[n-k, k]}$ of the symmetric group S_n have dimension polynomial in k . Then the maximum eigenvalue (and indeed, entire spectrum) of the $H_G^{[n-k, k]}$ matrices can be computed in time polynomial in n for constant k .

2.4.1 Maximizing irreps for 7 and 8 vertex graph Hamiltonians

The following tables give the results of some numerical experiments comparing the maximum eigenvalue of irrep Hamiltonians and the maximum eigenvalues of the QMC Hamiltonian for small graphs. In Table 1 we focus on 7 vertex graphs, $n = 7$, and the associated irreps of S_7 . In Table 2 we focus on 8 vertex graphs, $n = 8$, and the associated irreps of S_8 .

The third column of both of these tables gives the number of graphs for which the maximum eigenvalue of the QMC Hamiltonian H_G is equal to the maximum eigenvalue of the irrep Hamiltonian H_G^λ for the appropriate irrep λ . The fourth column gives the minimum over all graphs of the ratio between the max eigenvalue of the irrep Hamiltonian and the max eigenvalue of the QMC Hamiltonian, that is it is equal to

$$\min_{\substack{n\text{-vertex} \\ \text{connected} \\ \text{graphs } G}} \frac{\text{eig}_{\max} H_G^\lambda}{\text{eig}_{\max} H_G}.$$

Irrep	Irrep Dim	# Irrep Max Eig=Swap Max Eig	Irrep Max Eig / Swap Max Eig
[6, 1]	6	1	0.413
[5, 2]	14	32	0.766
[4, 3]	14	824	0.714

Table 1: Summary data for all 853 connected graphs on 7 vertices.

Irrep	Irrep Dim	# Irrep Max Eig=Swap Max Eig	Irrep Max Eig / Swap Max Eig
[7, 1]	7	1	0.354
[6, 2]	20	45	0.665
[5, 3]	28	1445	0.75
[4, 4]	14	9114	0.625

Table 2: Summary data for all 11,117 connected graphs on 8 vertices.

One might wonder why the sum in column 3 does not equal the total number of graphs. This for two reasons. One: two different irreps can have the same maximum eigenvalue. Two: equality of eigenvalues was checked inexactly (to a tolerance of 10^{-13}).

2.5 Exact Arithmetic Solution for Clique Hamiltonians

Now we consider the QMC Hamiltonian H_{K_n} corresponding to the clique K_n on n vertices. We will show that it is possible to compute the maximum eigenvalue (and indeed – entire spectrum) of this Hamiltonian efficiently by using the irreducible representations of Swap Matrix Algebra. Note that this fact is known in the physics literature, see, e.g., [CMP18, Lemma 36] and also [Os06].

Our proof proceeds in two parts: in Lemma 2.11 we show the corresponding QMC Irrep Hamiltonians $H_{K_n}^\lambda$ are constant multiples of the identity for each irrep λ , that is

$$H_{K_n}^\lambda = \eta_\lambda I. \quad (2.31)$$

Then, in Lemma 2.12 we compute the value of these constants η_λ .

Lemma 2.11. *For any integer n and irrep λ of S_n , we have*

$$H_{K_n}^\lambda = \eta_\lambda I \quad (2.32)$$

where η_λ is some scalar depending only on the irrep λ and I is an identity matrix of the appropriate dimension.

Proof. We begin by showing something stronger: let

$$q_{K_n} = \sum_{(i,j) \in E(K_n)} (i \ j) \in \mathbb{C}[S_n] \quad (2.33)$$

denote the sum over all transpositions in S_n . Then, for any permutation $\pi \in S_n$, we have

$$\pi q_{K_n} \pi^{-1} = \pi \sum_{i,j=1,\dots,n} (i \ j) \pi^{-1} = \sum_{i,j=1,\dots,n} (\pi(i) \ \pi(j)) = q_{K_n} \quad (2.34)$$

from which it follows that the element q_{K_n} is central in the group algebra $\mathbb{C}[S_n]$. In particular, for any irrep $\lambda = [n - k, k]$, we see that $\rho_\lambda(q_{K_n})$ is central in the irreducible algebra of matrices $\rho_\lambda(\mathbb{C}[S_n])$ acting on the vector space V_λ as defined in Eq. (2.8).

Similarly for any such ρ_λ we see $\rho_\lambda(2\binom{n}{2}e - 2q_{K_n}) = H_{K_n}^\lambda$ is an S_n -linear map from the irreducible module V_λ to itself and hence, by Schur's lemma, is a scalar multiple of the identity matrix. This completes the proof. $\square$

With a little bit of extra work, we can also compute the scalar multiple η_λ from Eq. (2.32) associated with Hamiltonian of the clique in each irrep. We do this next.

Lemma 2.12. *Let η_λ be as in Lemma 2.11. Then, for any integer n and irrep $\rho_{[n-k,k]}$ we have*

$$\eta_{[n-k,k]} = 2k(n+1) - 2k^2 \quad (2.35)$$

Proof. We can find the value of η_λ by computing two quantities: the dimension of the λ irrep (i.e., the trace of $\rho_\lambda(e)$) and the trace of $H_{K_n}^\lambda$. It is possible to compute both of these quantities with the Frobenius trace formula, but we shall save some effort by instead

using the hook length formula and Murnaghan-Nakayama rule. To deal with the many partial factorials appearing in this calculation, recall the falling factorial notation $n_{[k]}$ from Eq. (2.29). We shall also use the dimension of the $[n-k, k]$ irrep given in Lemma 2.10. For ease of future use we also write this formula as

$$\chi_{[s,t]}(e) = \frac{s-t+1}{s+1} \binom{s+t}{t} = \frac{(s-t+1)(s+t)_{[t-1]}}{t!}. \quad (2.36)$$

Next we compute the trace of $H_{K_n}^\lambda$. We can do this by computing the character $\chi_\lambda((i j))$ of a single transposition $(i j)$ in the λ irrep. Since all transpositions belong to the same conjugacy class, they will also have the same character. We now compute this character. The Murnaghan-Nakayama rule relates the character of a permutation in some irrep to the character of smaller permutations in irreps of the symmetric group on fewer elements. In the special case considered here, it gives

$$\chi_\lambda((i j)) = \sum_{\xi} (-1)^{h(\xi)} \chi_{\lambda/\xi}(e) \quad (2.37)$$

where the sum runs over all ways of removing two adjacent boxes from the irrep λ while leaving a valid Young diagram; $h(\xi)$ is one if the boxes removed are stacked vertically, zero otherwise; and λ/ξ is the resulting partition when the boxes are removed from λ . More explicitly, we write $\lambda = [n-k, k]$ and then, for any $n > 3$:

$$\chi_{[n-k,k]}((i j)) = \begin{cases} \chi_{[n-k-2,k]}(e) & \text{if } k = 1 \\ \chi_{[n-k-2,k]}(e) + \chi_{[n-k,k-2]}(e) & \text{if } 2 \leq k \leq n/2 - 1 \\ \chi_{[n-k,k-2]}(e) & \text{if } k = (n-1)/2 \\ \chi_{[n-k,k-2]}(e) - \chi_{[n-k-1,k-1]}(e) & \text{if } k = n/2. \end{cases} \quad (2.38)$$

By Eq. (2.14) the Hamiltonian for a clique K_n is

$$H_{K_n} = 2 \sum_{(i,j) \in E(K_n)} I - 2 \sum_{(i,j) \in E(K_n)} \text{Swap}_{ij} \quad (2.39)$$

$$= 2 \binom{n}{2} I - 2 \sum_{(i,j) \in E(K_n)} \rho_{[n-k,k]}((i j)), \quad (2.40)$$

and taking the trace of this expression gives

$$\text{Tr}[H_{K_n}^{[n-k,k]}] = 2 \binom{n}{2} \chi_{[n-k,k]}(e) - 2 \sum_{(i,j) \in E(G)} \chi_{[n-k,k]}((i j)) \quad (2.41)$$

$$= 2 \binom{n}{2} (\chi_{[n-k,k]}(e) - \chi_{[n-k,k]}((1 2))). \quad (2.42)$$

But we also have

$$\text{Tr}[H_{K_n}^{[n-k,k]}] = \text{Tr}[\eta_{[n-k,k]} I] = \eta_{[n-k,k]} \chi_{[n-k,k]}(e) \quad (2.43)$$

by definition of the trace. Putting these together gives

$$\eta_{[n-k,k]} = \frac{2 \binom{n}{2} (\chi_{[n-k,k]}(e) - \chi_{[n-k,k]}((1 2)))}{\chi_{[n-k,k]}(e)} = 2 \binom{n}{2} - 2 \hat{\eta}_{[n-k,k]} \quad (2.44)$$

where

$$\widehat{\eta}_{[n-k,k]} := \frac{\sum_{(i,j) \in E(K_n)} \chi_{[n-k,k]}((i \ j))}{\chi_{[n-k,k]}(e)} = \binom{n}{2} \frac{\chi_{[n-k,k]}((1 \ 2))}{\chi_{[n-k,k]}(e)}. \quad (2.45)$$

Finally, combining Eqs. (2.36), (2.38) and (2.44) gives a closed form expression for $\eta_{[n-k,k]}$. We do that below and simplify, assuming that $n > 5$ and treating several cases separately.

Case $k = 1$:

$$\begin{aligned} \widehat{\eta}_{[n-k,k]} &= \binom{n}{2} \chi_{[n-k-2,k]}(e) / \chi_{[n-k,k]}(e) \\ &= \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k-1)(n-2)_{[k-1]}}{k!} \right) \left(\frac{k!}{(n-2k+1)n_{[k-1]}} \right) \\ &= \frac{n(n-3)}{2} \end{aligned}$$

Case $k = 2$:

$$\begin{aligned} \widehat{\eta}_{[n-k,k]} &= \binom{n}{2} (\chi_{[n-k-2,k]}(e) + \chi_{[n-k,k-2]}(e)) / \chi_{[n-k,k]}(e) \\ &= \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k-1)(n-2)_{[k-1]}}{k!} \right) \left(\frac{k!}{(n-2k+1)n_{[k-1]}} \right) \\ &\quad + \left(\frac{n(n-1)}{2} \right) \left(\frac{k!}{(n-2k+1)(n)^{[k-1]}} \right) \\ &= \frac{(n-1)(n-5)(n-2)}{2(n-3)} + \frac{(n-1)}{(n-3)} \\ &= \frac{n(n-5)}{2} + 2 \end{aligned}$$

Case $3 \leq k \leq (n-1)/2$:

$$\begin{aligned} \widehat{\eta}_{[n-k,k]} &= \binom{n}{2} (\chi_{[n-k-2,k]}(e) + \chi_{[n-k,k-2]}(e)) / \chi_{[n-k,k]}(e) \\ &= \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k-1)(n-2)_{[k-1]}}{k!} \right) \left(\frac{k!}{(n-2k+1)(n)^{[k-1]}} \right) \\ &\quad + \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k+3)(n-2)^{[k-3]}}{(k-2)!} \right) \left(\frac{k!}{(n-2k+1)(n)^{[k-1]}} \right) \\ &= \frac{(n-2k-1)(n-k+1)(n-k)}{2(n-2k+1)} + \frac{(n-2k+3)k(k-1)}{2(n-2k+1)} \\ &= \frac{n(n-1)}{2} + k^2 - k(n+1) \end{aligned}$$

Case $k = (n-1)/2$:

$$\widehat{\eta}_{[n-k,k]} = \binom{n}{2} \chi_{[n-k,k-2]}(e) / \chi_{[n-k,k]}(e)$$

$$\begin{aligned}
&= \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k+3)(n-2)^{[k-3]}}{(k-2)!} \right) \left(\frac{k!}{(n-2k+1)(n)^{[k-1]}} \right) \\
&= \frac{(n-2k+3)k(k-1)}{2(n-2k+1)} \\
&= \frac{(n-1)(n-3)}{4}
\end{aligned}$$

Case $k = n/2$:

$$\begin{aligned}
\widehat{\eta}_{[n-k,k]} &= \binom{n}{2} (\chi_{[n-k,k-2]}(e) - \chi_{[n-k-1,k-1]}(e)) / \chi_{[n-k,k]}(e) \\
&= \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k+3)(n-2)^{[k-3]}}{(k-2)!} \right) \left(\frac{k!}{(n-2k+1)n_{[k-1]}} \right) \\
&\quad - \left(\frac{n(n-1)}{2} \right) \left(\frac{(n-2k+1)(n-2)^{[k-2]}}{(k-1)!} \right) \left(\frac{k!}{(n-2k+1)n_{[k-1]}} \right) \\
&= \frac{(n-2k+3)k(k-1)}{2(n-2k+1)} - \frac{(n-k+1)k}{2} \\
&= \frac{3n(n-2)}{8} - \frac{n(n+2)}{8} \\
&= \frac{n(n-4)}{4}
\end{aligned}$$

Straightforward calculation then shows that all of these cases are consistent with the formula

$$\widehat{\eta}_{[n-k,k]} = \binom{n}{2} + k^2 - k(n+1) \quad (2.46)$$

from the $3 \leq k \leq (n-1)/2$ case. Hence $\eta_{[n-k,k]} = 2k(n+1) - 2k^2$ and we are done. $\square$

Lemma 2.12 gives us a straightforward way to compute the max eigenvalue (and in fact, *all* eigenvalues) of the Hamiltonian H_{K_n} . In Section 6.1 we show how, with a bit more work, we can also use this lemma to compute the eigenvalues of graphs other than the complete graph.

2.6 Identities Satisfied by Swap Matrices

A key tool that will be developed in the subsequent section and then used in the remainder of this paper is a fully algebraic characterization of the swap matrices. In preparation for this, we observe some simple algebraic facts about the swap matrices.

Since the transpositions $(i j)$ are generators of S_n , it follows that M_n^{swap} is generated by the swap matrices Swap_{ij} . The latter must satisfy the relations

$$\text{Swap}_{ij}^2 = I, \quad (2.47a)$$

$$\text{Swap}_{ij} \text{Swap}_{jk} = \text{Swap}_{ik} \text{Swap}_{ij}, \quad (2.47b)$$

$$\text{Swap}_{ij} \text{Swap}_{kl} = \text{Swap}_{kl} \text{Swap}_{ij}, \quad (2.47c)$$

for all i, j, k, l distinct, arising from the relations satisfied by the transpositions $(i j)$. One can verify using Eq. (2.13) that, in addition to the above relations, the swap matrices satisfy

$$\text{Swap}_{ij} \text{Swap}_{jk} + \text{Swap}_{jk} \text{Swap}_{ij} = \text{Swap}_{ij} + \text{Swap}_{jk} + \text{Swap}_{ik} - I, \quad (\triangle)$$

which is in general not true for transpositions $(i\ j), (j\ k) \in \mathbb{C}[S_n]$. For instance, under the signature representation $\pi \mapsto \text{sgn}(\pi)$ we have $\text{sgn}((i\ j))\text{sgn}((j\ k)) + \text{sgn}((j\ k))\text{sgn}((i\ j)) = 2$ and $\text{sgn}((i\ j)) + \text{sgn}((j\ k)) + \text{sgn}((i\ k)) - 1 = -4$.

Remark 2.13. The commutation relations Eq. (2.47c) are (somewhat surprisingly) implied by Eqs. (2.47a), (2.47b) and $(\triangle)$. To prove this, note Eq. (2.50) below contains a list of 19 triples (f_r, g_r, ℓ_r) of noncommutative polynomials in the variables s_{ij} , $1 \leq i, j \leq 4$ such that

$$[s_{12}, s_{34}] = \sum_{r=1}^{19} f_r g_r \ell_r. \quad (2.48)$$

Further, when the variables s_{ij} are specialized to the matrices Swap_{ij} , each $g_k(\text{Swap})$ becomes an expression from either Eq. (2.47) or Eq. $(\triangle)$. Thus

$$[\text{Swap}_{12}, \text{Swap}_{34}] = \sum_k f_k(\text{Swap}) g_k(\text{Swap}) \ell_k(\text{Swap}) = 0. \quad (2.49)$$

Of course, by reindexing indices in Eq. (2.49) we then obtain Eq. (2.47c).

$$\begin{aligned} & (1, -s_{13}s_{23} + s_{23}s_{12}, 1), (-s_{24}, -s_{13}s_{23} + s_{23}s_{12}, 1), \\ & (1, -s_{12}s_{13} + s_{13}s_{23}, 1), (-s_{24}, -s_{12}s_{13} + s_{13}s_{23}, 1), \\ & (-1, s_{23}s_{34} - s_{24}s_{23}, s_{12}), (-1, -s_{24}s_{34} + s_{34}s_{23}, s_{12}), \\ & (-1, -s_{23}s_{24} + s_{24}s_{34}, s_{12}), (1, 1 - s_{23} - s_{24} - s_{34} + s_{23}s_{34} + s_{34}s_{23}, s_{12}), \\ & (1, -s_{14}s_{24} + s_{24}s_{12}, 1), (-s_{23}, -s_{14}s_{24} + s_{24}s_{12}, 1), \\ & (1, -s_{12}s_{14} + s_{14}s_{24}, 1), (-s_{23}, -s_{12}s_{14} + s_{14}s_{24}, 1), \\ & (-1, -s_{14}s_{24} + s_{24}s_{12}, s_{13}), (-1, -s_{12}s_{14} + s_{14}s_{24}, s_{13}), \\ & (-s_{12}, s_{14}s_{13} - s_{34}s_{14}, 1), (-s_{12}, 1 - s_{13} - s_{14} - s_{34} + s_{14}s_{34} + s_{34}s_{14}, 1), \\ & (s_{12}, -s_{13}s_{14} + s_{14}s_{34}, 1), (-1, -s_{13}s_{23} + s_{23}s_{12}, s_{14}), (-1, -s_{12}s_{13} + s_{13}s_{23}, s_{14}) \end{aligned} \quad (2.50)$$

□

Remark 2.14. There are several sets of relations that could be used to define the symmetric group. One option is to define this group as being generated by the transpositions $(i\ j)$ satisfying relations Equations (2.47a) to (2.47c). Alternately, this group can be defined with generators $(i\ i+1)$ (that is, only the adjacent elements) and relations Eqs. (2.47a) and (2.47c) along with the relations

$$(\text{Swap}_{i,i+1} \text{Swap}_{i+1,i+2})^3 = I. \quad (2.51)$$

□

3 The Symbolic Swap Algebra

In this section we introduce an abstract $*$ -algebra generated by formal variables which satisfy the same relations as the swap matrices introduced in the previous section. We call this algebra the Symbolic Swap Algebra, and then show that it is isomorphic to the Swap Matrix Algebra. The main reason for introducing the Symbolic Swap Algebra comes from its use in constructing the *Non-Commutative Sum of Squares (ncSoS) Hierarchy* for Quantum Max Cut, discussed in Section 4. Specifically, this allows us to formulate rewrite rules for swap matrix polynomials (see Section 3.4 and Section 5) and express polynomial identities without explicitly using the matrix representations, which may be of independent interest.

An alternate approach to giving a fully algebraic characterization (i.e., a presentation) of the swap matrix algebra would be to build directly on the results of Section 2.6. There we identified generators for M_n^{swap} along with a set of relations (i.e., identities) that those generators had to satisfy. To prove that these generators and relations give a presentation of the swap matrix algebra all that remains is to show that *all* identities in the swap matrix algebra follow from these relations. Indeed, we will see in just a few pages that this is the case. We prefer the longer and slightly more mathematically involved approach described in the previous paragraph because it allows us to clearly distinguish elements of the swap matrix algebra M_n^{swap} , which we view as matrices, from elements of the isomorphic Symbolic Swap Algebra, which we view as formal non-commuting variables.

3.1 A Presentation of the Symbolic Swap Algebra

Definition 3.1. Define the n th Symbolic Swap Algebra, denoted $\mathcal{A}_n^{\text{swap}}$, to be the $*$ -algebra over $\mathbb{C}$ generated by the set of symmetric elements $\{s_{ij} \mid 1 \leq i < j \leq n\}$ satisfying relations:

- (1) $s_{ij}^2 = 1$;
- (2) $s_{ij}s_{jk} = s_{ik}s_{ij}$,
- (3) $s_{ij}s_{jk} + s_{jk}s_{ij} = s_{ij} + s_{jk} + s_{ik} - 1$.

for all $1 \leq i, j, k \leq n$ all distinct. In the relations above, for parsimony of notation, we have sometimes used s_{ij} with $i > j$ to mean s_{ji} . We will continue to use this s_{ij} shorthand later in the paper. The reason for this abuse of notation is discussed in Remark 3.2.

Equivalently, we could have defined the Symbolic Swap Algebra as a quotient of the free algebra. Consider the free algebra $\mathbb{C}\langle s_{ij} \mid 1 \leq i < j \leq n \rangle$ endowed with the involution $*$ that fixes each s_{ij} , and is complex conjugation on $\mathbb{C}$. Let $\mathcal{I}_n^{\text{swap}}$ be the ideal in this free algebra generated by Item (1), Item (2) and Item (3). Then

$$\mathcal{A}_n^{\text{swap}} = \mathbb{C}\langle s_{ij} \mid 1 \leq i < j \leq n \rangle / \mathcal{I}_n^{\text{swap}} \quad (3.1)$$

is the n th Symbolic Swap Algebra. We will refer to the elements s_{ij} generating the Symbolic Swap Algebra as the *swap variables*.

Remark 3.2. The abuse of notation used in Definition 3.1 above avoids a complication we now describe. Suppose that we rigidly insist that s_{ij} must have $i < j$. Then to define the Symbolic Swap Algebra we must use more relations, namely

- (1) $s_{ij}^2 - 1$
- (2) $s_{ij}s_{jk} = s_{ik}s_{ij}, \quad s_{ij}s_{ik} = s_{jk}s_{ij}, \quad s_{ik}s_{jk} = s_{jk}s_{ik},$
- (3) $(s_{ij}s_{jk} + s_{jk}s_{ij}) = (s_{ij} + s_{jk} + s_{ik} - 1), \quad (s_{ij}s_{ik} + s_{ik}s_{ij}) = (s_{ij} + s_{jk} + s_{ik} - 1),$
 $(s_{ik}s_{jk} + s_{jk}s_{ik}) = (s_{ij} + s_{jk} + s_{ik} - 1).$

To illustrate the issue, we claim that $s_{12}s_{13} = s_{23}s_{12}$ is true in the algebra $\mathcal{A}_n^{\text{swap}}$ but does not follow from the equations given in Definition 3.1 had we enforced index ordering $i < j < k$. However $s_{12}s_{13} = s_{23}s_{12}$ is of the form of the second equation of Item (2).

In Sections 4 and 5 when we turn to more computational aspects of $\mathcal{A}_n^{\text{swap}}$ we will enforce index ordering $i < j$ and encode relations as in Remark 3.2. $\square$

Remark 3.3. As in Remark 2.13, we deduce that swaps on disjoint indices commute, i.e.,

$$[s_{ij}, s_{k\ell}] = 0 \quad (3.2)$$

whenever i, j, k, ℓ are distinct. Similarly we can deduce the relations

$$(s_{ij}s_{jk})^3 = 1 \quad (3.3)$$

given in the standard presentation of the symmetric group. $\square$

We can also understand $\mathcal{A}_n^{\text{swap}}$ as a quotient of the symmetric group algebra $\mathbb{C}[S_n]$.

Lemma 3.4. *Let $\tilde{\mathcal{I}}_n^{\text{swap}}$ be the two-sided ideal of $\mathbb{C}[S_n]$ generated by the elements*

$$((i\ j)(j\ k) + (j\ k)(i\ j)) - ((i\ j) + (j\ k) + (i\ k) - 1) \quad (3.4)$$

for distinct $1 \leq i, j, k \leq n$. Then there is a natural $$ -homomorphism*

$$\mathbb{C}[S_n] \rightarrow \mathcal{A}_n^{\text{swap}}, \quad (i\ j) \mapsto s_{ij}, \quad \text{for } 1 \leq i < j \leq n,$$

and the kernel of this homomorphism is given by $\tilde{\mathcal{I}}_n^{\text{swap}}$.

Proof. We begin by noting the group algebra of the symmetric group $\mathbb{C}[S_n]$ can be presented as the algebra with generators $\{(i\ j) \mid 1 \leq i \neq j \leq n\}$ satisfying relations

- (1) $(i\ j)^2 = 1$;
- (2) $(i\ j)(j\ k) = (i\ k)(i\ j)$;
- (3) $(i\ j)(k\ \ell) = (k\ \ell)(i\ j)$

for all distinct integers i, j, k, ℓ .

The Symbolic Swap Algebra algebra has an equivalent set of generators with the only additional relations being relations being those given in Eq. (3.4) above, so the proof follows. $\square$

3.2 Relationship Between Symbolic Swap Algebra and Swap Matrix Algebra

Now we relate the Symbolic Swap Algebra to both the Swap Matrices and to the irreps of the symmetric group, cf. [Pro07, Theorem, §6.1].

Proposition 3.5. *The following are characterizations of $\mathcal{A}_n^{\text{swap}}$, the Symbolic Swap Algebra:*

- (a) *The elements in $\tilde{\mathcal{I}}_n^{\text{swap}}$ are precisely those that vanish under all irreps of S_n corresponding to at most two rows of the Young tableaux.*
- (b) *The polynomials in $\mathcal{I}_n^{\text{swap}}$ are exactly the polynomials which annihilate the matrices Swap_{ij} .*

Proof. For the sake of completeness, we recall additional terminology necessary for analysing the irreducible modules V_λ of S_n . A *Young tableau* of shape $\lambda \vdash n$ is a filling of the Young diagram λ by integers $1, 2, \dots, n$ such that each box is assigned a unique integer. The action of the group S_n on a Young tableau t follows by letting $\pi \in S_n$ act on the entries of t . Permuting entries within each row of a tableau t gives us an equivalence class of tableaux – a *tabloid*

$$\{t_1, t_2, \dots, t_k\} := \{t\}. \quad (3.5)$$

The group action extends to tabloids as $\pi\{t\} = \{\pi(t_1), \pi(t_2), \dots, \pi(t_k)\}$. The so-called Specht irreducible module V_λ of $\mathbb{C}[S_n]$ is spanned by *polytabloids*

$$e_t = \sum_{\pi \in C_t} \text{sgn}(\pi) \pi\{t\} \quad (3.6)$$

where t ranges over Young tableau of shape λ , and C_t is the set of permutations that permute elements only within the columns of t .

For a 3-row irrep given by $\lambda = [\lambda_1, \lambda_2, \lambda_3]$, let T be the Young tableau

$$T = \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & \dots & \dots & \lambda_1 \\ \hline \lambda_1+1 & \dots & \dots & \lambda_2 & \\ \hline \lambda_2+1 & \dots & \lambda_3 & & \\ \hline \end{array} \quad (3.7)$$

and e_T the corresponding polytabloid. Consider the transpositions $(1 \ \lambda_1 + 1)$, $(1 \ \lambda_2 + 1)$, $(\lambda_1 + 1 \ \lambda_2 + 1)$ all of which permute elements within the first column of T and are thus contained in C_T . Note that

$$(1 \ \lambda_1)e_T = e_{(1 \ \lambda_1)T} \quad (3.8)$$

and on expanding the right-hand side of Eq. (3.8) we see that T appears with coefficient $\text{sgn}((1 \ \lambda_1 + 1)) = -1$. By the exact same reasoning, the coefficients of T in $(1 \ \lambda_2)e_T$ and $(\lambda_1 \ \lambda_2)e_T$ also equal -1 . Now, each of $(1 \ \lambda_1 + 1)(\lambda_1 + 1 \ \lambda_2 + 1) = (1 \ \lambda_2 + 1 \ \lambda_1 + 1)$ and $(\lambda_1 + 1 \ \lambda_2 + 1)(1 \ \lambda_1 + 1) = (1 \ \lambda_1 + 1 \ \lambda_2 + 1)$ is an even permutation contained in C_T . Repeating the above for $(1 \ \lambda_1 + 1)(\lambda_1 + 1 \ \lambda_2 + 1)e_T$ and $(\lambda_1 + 1 \ \lambda_2 + 1)(1 \ \lambda_1 + 1)e_T$, we see that the coefficient of T in these must be $+1$. It follows that

$$[(1 \ \lambda_1 + 1) + (1 \ \lambda_2 + 1) + (\lambda_1 + 1 \ \lambda_2 + 1) - 1]e_T \neq \{(1 \ \lambda_1 + 1), (\lambda_1 + 1 \ \lambda_2 + 1)\}e_T \quad (3.9)$$

in the irrep specified by λ , and more generally in any irrep with 3 or more rows. That is, $s_{ij}s_{jk} + s_{jk}s_{ij} - (s_{ij} + s_{jk} + s_{ik} - 1)$ does not vanish under the evaluation $s_{ij} = \rho_\lambda(i \ j)$ for the above irrep λ . Thus we have proved that the swap relations are incompatible with a ≥ 3 row Young Tableaux.

For the converse, consider distinct indices i, j, k and a tabloid T of shape $[n - k, k]$. Let $\pi \in C_T$. We distinguish two cases:

- (a) i, j, k lie in the same row of $\pi\{T\}$.

Then each of the terms s_{ij}, s_{jk}, s_{ik} acts as the identity on $\pi\{T\}$, whence

$$(s_{ij}s_{jk} + s_{jk}s_{ij} - (s_{ij} + s_{jk} + s_{ik} - 1))(\pi\{T\}) = 0. \quad (3.10)$$

- (b) i, j lie in the same row of $\pi\{T\}$, but k does not.

Then,

$$s_{ij}(\pi\{T\}) = \pi\{T\}, \quad s_{ij}s_{jk}(\pi\{T\}) = s_{ik}(\pi\{T\}), \quad s_{jk}s_{ij}(\pi\{T\}) = s_{jk}(\pi\{T\}).$$

As before, this implies Eq. (3.10).

This shows that for each $\pi \in C_T$, Eq. (3.10) holds, whence $s_{ij}s_{jk} + s_{jk}s_{ij} - (s_{ij} + s_{jk} + s_{ik} - 1)$ vanishes under the evaluation $s_{ij} = \rho_\lambda(i \ j)$ for any irrep $\lambda = [n - k, k]$. $\square$

Theorem 3.6 ([Pro07, Theorem, §6.1]). *The algebras $\mathcal{A}_n^{\text{swap}}$ and M_n^{swap} are isomorphic. More precisely, the natural map*

$$\mathcal{A}_n^{\text{swap}} \rightarrow M_n^{\text{swap}}, \quad s_{ij} \mapsto \text{Swap}_{ij}$$

is an isomorphism.

Proof. An immediate corollary of Proposition 3.5. $\square$

The isomorphism between the Symbolic Swap Algebra and Swap Matrix Algebra given above lets us identify the QMC Hamiltonian associated with a graph G with an element of the Symbolic Swap Algebra. We make this clear in the following definition.

Definition 3.7. *For any graph g define the symbolic QMC Hamiltonian h_G by*

$$h_G := \sum_{(i,j) \in E(G)} 2w_{ij} (I - s_{ij}) \in \mathcal{A}_n^{\text{swap}} \quad (3.11)$$

We end this section by proving some results about the dimension of the Symbolic Swap Algebra. While the formula for the dimension is not used elsewhere in the paper it does bring to light a potentially interesting connection between the swap matrix algebra and the Catalan numbers.

Corollary 3.8. *Recall from Lemma 2.10 that $\chi_{[n-k,k]}(e)$ denotes the dimension of the module $V_{[n-k,k]}$ of S_n . Then*

$$(a) \quad \mathcal{A}_n^{\text{swap}} \cong \bigoplus_{k=0}^{\lfloor \frac{n}{2} \rfloor} M_{\chi_{[n-k,k]}(e)}(\mathbb{C}).$$

$$(b) \quad \dim \mathcal{A}_n^{\text{swap}} = \sum_{k=0}^{\lfloor \frac{n}{2} \rfloor} \left(\frac{n-2k+1}{n-k+1} \binom{n}{k} \right)^2 = \frac{1}{n+1} \binom{2n}{n},$$

which is the n -th Catalan number C_n .

Proof. Item (a) follows from Proposition 3.5.

Item (b): The first equality simply uses the dimension of the full matrix algebra. We shall give two proofs of the second equality.

Algebraic proof. Observe that

$$\begin{aligned} \frac{1}{n-k+1} \binom{n}{k} &= \frac{1}{n+1} \binom{n+1}{k} \\ k \binom{n+1}{k} &= (n+1) \binom{n}{k-1} \end{aligned}$$

which simplifies the summand $\left(\frac{n-2k+1}{n-k+1} \binom{n}{k} \right)^2$ into

$$\left(\binom{n+1}{k} - 2 \binom{n}{k-1} \right)^2 = \left(\binom{n}{k} - \binom{n}{k-1} \right)^2 = \binom{n}{k}^2 - 2 \binom{n}{k} \binom{n}{k-1} + \binom{n}{k-1}^2,$$

where we applied the recurrence relation on the binomial coefficients for the middle equality. Thus

$$\dim \mathcal{A}_n^{\text{swap}} = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{k}^2 - 2 \binom{n}{k} \binom{n}{k-1} + \binom{n}{k-1}^2. \quad (3.12)$$

By the Chu-Vandermonde identity,

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}, \quad (3.13)$$

whence by symmetry,

$$\sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{k}^2 = \begin{cases} \frac{1}{2} \binom{2n}{n} & n \text{ odd} \\ \frac{1}{2} \left(\binom{2n}{n} + \binom{n}{n/2}^2 \right) & n \text{ even.} \end{cases} \quad (3.14)$$

Likewise, we obtain

$$\sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{k-1}^2 = \begin{cases} \frac{1}{2} \left(\binom{2n}{n} - 2 \binom{n}{(n-1)/2}^2 \right) & n \text{ odd} \\ \frac{1}{2} \left(\binom{2n}{n} - \binom{n}{n/2}^2 \right) & n \text{ even.} \end{cases} \quad (3.15)$$

Similarly, the Chu-Vandermonde identity implies

$$\sum_{k=0}^n \binom{n}{k} \binom{n}{k-1} = \binom{2n}{n+1},$$

whence again by symmetry,

$$\sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n}{k} \binom{n}{k-1} = \begin{cases} \frac{1}{2} \left(\binom{2n}{n+1} - \binom{n}{(n-1)/2}^2 \right) & n \text{ odd} \\ \frac{1}{2} \binom{2n}{n+1} & n \text{ even.} \end{cases} \quad (3.16)$$

Inserting Eq. (3.14), Eq. (3.15), Eq. (3.16) into Eq. (3.12) yields

$$\dim \mathcal{A}_n^{\text{swap}} = \binom{2n}{n} - \binom{2n}{n+1} = \binom{2n}{n} - \frac{n}{n+1} \binom{2n}{n} = \frac{1}{n+1} \binom{2n}{n} = C_n,$$

as desired.

Combinatorial proof. We also provide a slicker but less self-contained combinatorial proof. It is well-known that the n -th Catalan number C_n counts the number of standard Young tableaux of the shape $[n, n]$ [Sta99, Exercise 6.19.ww]. Picking out the squares containing $1, \dots, n$ and $n+1, \dots, 2n$, respectively, from such a tableaux, we obtain two Young tableaux of the same shape $[n-k, k]$ for some $k \in \mathbb{N}$ (see Eq. (3.17) below for a simple example).

$$\begin{array}{|c|c|c|c|} \hline 1 & 3 & 4 & 6 \\ \hline 2 & 5 & 7 & 8 \\ \hline \end{array} \rightsquigarrow \left(\begin{array}{|c|c|c|} \hline 1 & 3 & 4 \\ \hline 2 & & \\ \hline \end{array}, \begin{array}{|c|c|c|} \hline & & 6 \\ \hline 5 & 7 & 8 \\ \hline \end{array} \right) \\ \rightsquigarrow \left(\begin{array}{|c|c|c|} \hline 1 & 3 & 4 \\ \hline 2 & & \\ \hline \end{array}, \begin{array}{|c|c|c|} \hline 1 & 2 & 4 \\ \hline 3 & & \\ \hline \end{array} \right). \quad (3.17)$$

Since standard Young tableaux of the shape $[n-k, k]$ correspond to a basis of the corresponding irrep of S_n , we are done by (a). $\square$

3.3 The Irrep Symbolic Swap Algebra

Recall H_{K_n} denotes the Hamiltonian the swap matrices associate with the clique K_n on n vertices, h_{K_n} denotes the element of the Symbolic Swap Algebra associated to the clique and that $H_{K_n}^\lambda$ denotes the representation of h_{K_n} in the λ irrep of the symmetric group S_n .

Lemma 3.9. *The function of k in Eq. (2.35) is strictly increasing for $0 \leq k \leq \frac{n}{2}$.*

Proof. Simply observe that the function's derivative w.r.t. k is

$$2(n+1) - 4k \geq 2(n+1) - 2n = 2 > 0. \quad \square$$

Proposition 3.10. *Let $h_{K_n} \in \mathcal{A}_n^{\text{swap}}$ denote the Hamiltonian polynomial for the n -clique. Then the ideal $\mathcal{I}_{n-k,k}$ of polynomials which annihilate $\rho_{[n-k,k]}$, is generated by $\mathcal{I}^{\text{swap}}$ together with $h_{K_n} - \eta_{n-k,k}$.*

Proof. Let $\mathcal{A}_n^\lambda$ denote the simple algebra of the irrep ρ_λ of S_n for $\lambda \vdash n$. It is isomorphic to $M_{\chi_\lambda(e)}(\mathbb{C})$. By Corollary 3.8,

$$\mathcal{A}_n^{\text{swap}} \cong \bigoplus_{k=0}^{\lfloor \frac{n}{2} \rfloor} \mathcal{A}_n^{[n-k,k]}. \quad (3.18)$$

With this notation,

$$\mathcal{A}_n^{\text{swap}} / \mathcal{I}_{n-k,k} \cong \mathcal{A}_n^{[n-k,k]}.$$

By Lemma 2.11 and Lemma 2.12, $h_{K_n} - \eta_{n-k,k} \in \mathcal{I}_{n-k,k}$. Since by Lemma 3.9, the map $k \mapsto \eta_{n-k,k}$ is injective for $0 \leq k \leq n/2$, the image under the isomorphism of Eq. (3.18) of $h_{K_n} - \eta_{n-k,k} \in \mathcal{I}_{n-k,k}$ is zero in the k -th entry and a nonzero number in each of the others. In particular,

$$\mathcal{A}_n^{\text{swap}} / \mathcal{I}(h_{K_n} - \eta_{n-k,k}) \cong \mathcal{A}_n^{[n-k,k]} \cong \mathcal{A}_n^{\text{swap}} / \mathcal{I}_{n-k,k}.$$

Since $\mathcal{I}(h_{K_n} - \eta_{n-k,k}) \subseteq \mathcal{I}_{n-k,k}$, this concludes the proof. $\square$

3.4 Structure of Swap Variable Polynomials

Now we describe some further properties of the Swap Algebra. The main result of this subsection is a sharp upper bound of $\lceil n/2 \rceil$ on the maximum degree of elements in the swap algebra, along with simplified form for elements in this algebra. Later, we will observe that this upper bound gives a corresponding upper bound on the level at which the hierarchy of semidefinite programs constructed in Section 4 gives an exact solution to the Quantum Max Cut problem. We head in that direction via a lemma.

Define the *support graph* G_p of a polynomial p in the variables s_{ij} to have vertices corresponding to integers which appear as indices i or j for some s_{ij} in p , with (i, j) an edge iff s_{ij} appears in p . A polynomial in $\mathcal{I}^{\text{swap}}$ is called *degree reducing* provided it has exactly one highest degree term.

3.4.1 Degree reducing polynomials in $\mathcal{I}^{\text{swap}}$

Lemma 3.11. *For $n \geq 4$ and $1 \leq i, j, k, \ell \leq n$ with no two indices being equal, there is a degree reducing polynomial in $\mathcal{I}^{\text{swap}}$ whose highest degree term has the form*

- (1) $s_{jk}s_{ik}s_{\ell k}$. a term with support graph a three edge star;
- (2) $s_{ij}s_{jk}s_{k\ell}$, $s_{jk}s_{ij}s_{k\ell}$, or $s_{jk}s_{k\ell}s_{ij}$, a term with support graph a three edge line.

Observe that any of the three edge line monomials equals a star graph monomial modulo a single triangle pair substitution.

Here a *triangle pair relation* is a polynomial of the form $s_{ij}s_{jk} - s_{jk}s_{ki}$. It lies in $\mathcal{I}^{\text{swap}}$ and relates behavior of edge pairs in the triangle which is the support of the polynomial.

Proof. Straightforward matrix multiplication can be used to verify the following matrix identity:

$$\begin{aligned} 2 \text{Swap}_{jk} \text{Swap}_{ik} \text{Swap}_{\ell k} &= 1 - \text{Swap}_{ij} - \text{Swap}_{il} - \text{Swap}_{jk} - \text{Swap}_{kl} + \text{Swap}_{ij} \text{Swap}_{ik} \\ &\quad + \text{Swap}_{ij} \text{Swap}_{il} + \text{Swap}_{ij} \text{Swap}_{kl} + \text{Swap}_{ik} \text{Swap}_{il} \\ &\quad - \text{Swap}_{ik} \text{Swap}_{jl} + \text{Swap}_{il} \text{Swap}_{jk} + \text{Swap}_{jk} \text{Swap}_{jl}, \end{aligned}$$

which implies the same identity for the s_{ij} by Theorem 3.6.

The proof of the three-edge-line case and the final paragraph of the lemma amount to the identities

$$\begin{aligned} s_{ij}s_{jk}s_{kl} &= s_{ij}s_{jl}s_{jk}, \\ s_{jk}s_{ij}s_{kl} &= s_{ik}s_{jk}s_{kl}, \\ s_{jk}s_{kl}s_{ij} &= s_{jl}s_{jk}s_{ij} \end{aligned}$$

so we can always reduce to the first case. $\square$

Lemma 3.12. *For $n \geq 6$ and $1 \leq i, j, k, a, b, c \leq n$ with no two indices being equal, there is a degree reducing polynomial in $\mathcal{I}^{\text{swap}}$ whose highest degree term has the form*

$$s_{ij}s_{jk}s_{ab}s_{bc}, \quad \text{a term whose support graph is two disjoint two edge lines.}$$

Proof. It is straightforward to verify the following identity for $2^6 \times 2^6$ matrices from which the lemma follows using Theorem 3.6 by reindexing if needed:

$$\begin{aligned} 4 \text{Swap}_{12} \text{Swap}_{23} \text{Swap}_{45} \text{Swap}_{56} &= 3 - 3 \text{Swap}_{12} - 3 \text{Swap}_{13} - 3 \text{Swap}_{23} - 3 \text{Swap}_{45} \\ &\quad - 3 \text{Swap}_{46} - 3 \text{Swap}_{56} + 2 \text{Swap}_{12} \text{Swap}_{13} + 3 \text{Swap}_{12} \text{Swap}_{45} + 3 \text{Swap}_{12} \text{Swap}_{46} \\ &\quad + 3 \text{Swap}_{12} \text{Swap}_{56} + 3 \text{Swap}_{13} \text{Swap}_{45} + 3 \text{Swap}_{13} \text{Swap}_{46} + 3 \text{Swap}_{13} \text{Swap}_{56} \\ &\quad + \text{Swap}_{14} \text{Swap}_{25} - \text{Swap}_{14} \text{Swap}_{26} - \text{Swap}_{14} \text{Swap}_{35} + \text{Swap}_{14} \text{Swap}_{36} \\ &\quad - \text{Swap}_{15} \text{Swap}_{24} + \text{Swap}_{15} \text{Swap}_{26} + \text{Swap}_{15} \text{Swap}_{34} - \text{Swap}_{15} \text{Swap}_{36} \\ &\quad + \text{Swap}_{16} \text{Swap}_{24} - \text{Swap}_{16} \text{Swap}_{25} - \text{Swap}_{16} \text{Swap}_{34} + \text{Swap}_{16} \text{Swap}_{35} \\ &\quad + 3 \text{Swap}_{23} \text{Swap}_{45} + 3 \text{Swap}_{23} \text{Swap}_{46} + 3 \text{Swap}_{23} \text{Swap}_{56} + \text{Swap}_{24} \text{Swap}_{35} \\ &\quad - \text{Swap}_{24} \text{Swap}_{36} - \text{Swap}_{25} \text{Swap}_{34} + \text{Swap}_{25} \text{Swap}_{36} + \text{Swap}_{26} \text{Swap}_{34} \\ &\quad - \text{Swap}_{26} \text{Swap}_{35} + 2 \text{Swap}_{45} \text{Swap}_{46} - 2 \text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{45} \\ &\quad - 2 \text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{46} - 2 \text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{56} \\ &\quad - 2 \text{Swap}_{12} \text{Swap}_{45} \text{Swap}_{46} - 2 \text{Swap}_{13} \text{Swap}_{45} \text{Swap}_{46} \\ &\quad - 2 \text{Swap}_{14} \text{Swap}_{25} \text{Swap}_{36} + 2 \text{Swap}_{14} \text{Swap}_{26} \text{Swap}_{35} \\ &\quad + 2 \text{Swap}_{15} \text{Swap}_{24} \text{Swap}_{36} - 2 \text{Swap}_{15} \text{Swap}_{26} \text{Swap}_{34} \\ &\quad - 2 \text{Swap}_{16} \text{Swap}_{24} \text{Swap}_{35} + 2 \text{Swap}_{16} \text{Swap}_{25} \text{Swap}_{34} \\ &\quad - 2 \text{Swap}_{23} \text{Swap}_{45} \text{Swap}_{46}. \end{aligned}$$

$\square$

3.4.2 Useful graph properties

Lemma 3.13. *Let G be a connected n vertex graph.*

- (1) *If $n \geq 4$, then G contains a three edge line or a three edge star graph as a subgraph.*
- (2) *If G has more than three edges, then G contains a three edge line or a three edge star or a triangle as a subgraph.*

Proof. Let G be as above. There exists some spanning tree T of G as G is connected. Given two points u and v in T , there is a unique path connecting them. The length of this path is the graph distance from u to v . Now, pick some point in T and call it r . We define V_i to be the set of vertices in T that are distance i from r . We know that $V_0 = \{r\}$. If V_1 contains three or more elements, then G must contain a three edge star graph as a subgraph. If V_1 contains fewer than three elements, then as $n \geq 4$, we must have an element in V_2 . Thus, G contains a three edge line.

The proof of the second assertion is obvious. $\square$

Lemma 3.14. *Suppose G is an n vertex graph with $|E(G)| \geq \lceil \frac{n}{2} \rceil + 1$. Then G must have*

- (1) *a single connected component with at least three edges, hence G contains a triangle, a three edge line, or a three edge star as a subgraph; or*
- (2) *two connected components each containing a 2 edge line.*

Proof. To see this, consider $G_1, \dots, G_k$, the connected components of G . The point is that when $|E(G)|$ is small the number of possible configurations is small:

- (a) $|E(G)| = \frac{n}{2} = \lceil \frac{n}{2} \rceil$ with n even implies each G_i has exactly one edge.
- (b) $|E(G)| = \lceil \frac{n}{2} \rceil$ with n odd implies each G_i except one, say G_1 , has exactly one edge and G_1 is a 2 edge line.
- (c) $|E(G)| = \frac{n}{2} + 1 = \lceil \frac{n}{2} \rceil + 1$ with n even implies each G_i except two, say G_1 and G_2 , have 1 edge and G_1 and G_2 each contain (actually equal) 2 edge lines.
- (d) $|E(G)| = \frac{n}{2} + 1 = \lceil \frac{n}{2} \rceil + 1$ with n odd implies each G_i except one, say G_1 , has exactly one edge and G_1 has 3 edges, so is a three edge line, a triangle or a three edge star graph.

Bigger $|E(G)|$ just adds edges to some of these components, thereby justifying inequality in the theorem statement. $\square$

3.4.3 Monomials associated to their support graphs

Lemma 3.15. *Any monomial in the n -vertex swap variables of the form $p = qs_{ij}$ can be written in the form $p = s_{ij}q'$, for $\deg(q') \leq \deg(q)$.*

Proof. For any s_{kl} and s_{ij} , we have $s_{kl}s_{ij} = s_{ij}s_e$ for some edge e . Indeed, we consider 5 cases: (1) if $\{i, j\} \cap \{k, \ell\} = \emptyset$, then $s_e = s_{kl}$; (2) if $j = k$, then $s_e = s_{ik}$; (3) if $i = k$, then $s_e = s_{jl}$; (4) if $i = \ell$, then $s_e = s_{kj}$; (5) if $j = \ell$, then $s_e = s_{ik}$. Thus, applying the above inductively, we get $qs_{ij} = s_{ij}q'$ for $\deg(q) \geq \deg(q')$. $\square$

Lemma 3.16. *Any monomial in the n -vertex swap variables of the form $p = s_{ij}qs_{ij}$ can be written as a monomial of degree at most $\deg(q)$. In particular, for fixed i, j any monomial in the n -vertex swap variables can be assumed to contain at most a single s_{ij} .*

Proof. Applying Lemma 3.15, we get $s_{ij}qs_{ij} = s_{ij}^2q' = q'$ for $\deg(q) \geq \deg(q')$.

The second statement follows from the first by induction. $\square$

Lemma 3.17. *A monomial in the n -vertex swap variables of the form $p = q_0 \prod_{j=1}^k (s_{e_j}q_j)$ with the e_j being distinct edges equals a monomial $(\prod_{j=1}^k s_{e_j})q$ of the same degree or smaller.*

Proof. Suppose $p = q_0 \prod_{j=1}^k (s_{e_j}q_j)$ as above. Then one application of Lemma 3.15 gives us $p = s_{e_1}q'_0q_1 \prod_{j=2}^k (s_{e_j}q_j)$. Two more applications yield $p = s_{e_1}s_{e_2}q''_0q'_1q_2 \prod_{j=3}^k (s_{e_j}q_j)$. Repeating this process inductively, we obtain $p = (\prod_{j=1}^k s_{e_j})q_0^{(n)}q_1^{(n-1)} \cdots q'_{n-1}q_n$. Note that at each step, the degree did not increase, so taking $q = q_0^{(n)}q_1^{(n-1)} \cdots q'_{n-1}q_n$ we obtain the desired result. $\square$

3.4.4 Polynomials in swap variables

After considerable preparation we now give and prove any polynomial p in swaps can be reduced to a simple form q . The proof is constructive (and does not use Gröbner bases).

Theorem 3.18. *Any polynomial p in the n -vertex swap variables, is mod $\mathcal{I}^{\text{swap}}$ equal to some polynomial q with $\deg(q) \leq \lceil \frac{n}{2} \rceil$.*

Moreover, one can take q to have each of its terms a monomial of the form

$$m = m_1m_2 \dots m_r \quad \text{mod } \mathcal{I}^{\text{swap}} \quad (3.19)$$

where $m_1, m_2, \dots, m_r$ are commuting monomials of all of degree one except possibly one has degree two. Also each pair m_i, m_j with $i \neq j$ are supported on vertex disjoint graphs.

Proof. Let $\tilde{p}$ be a polynomial of minimum degree within $p + \mathcal{I}^{\text{swap}}$. By Lemma 3.16 we may assume that in any term τ of $\tilde{p}$ the degree of any particular s_{ij} is at most 1. Suppose that $\deg(\tau) \geq \lceil \frac{n}{2} \rceil + 1$. Then $|E(G_\tau)| \geq \lceil \frac{n}{2} \rceil + 1$, so by Lemma 3.14 the graph G_τ contains

- (1) three edges e_1, e_2, e_3 which form a 3 edge line, or 3 edge star, or triangle; or
- (2) two vertex disjoint 2 edge lines.

In the first case we can use Lemma 3.17 to obtain $\tau = s_{e_1}s_{e_2}s_{e_3}q'$ where e_1, e_2, e_3 are the three edges in the three edge subgraph described above. By Lemma 3.11 we have a degree reducing relation for monomials supported on three edge lines, three edge star graphs, and triangles which combined with τ produce a polynomial equivalent to it mod $\mathcal{I}^{\text{swap}}$ but of degree less than $\deg(\tau)$. This contradicts τ having minimal degree.

To prove the second case Lemma 3.12 works similarly to give a degree drop contradiction. This proves the first part of the theorem.

Proof of the second part of the theorem. Since $\deg(q) \leq \lceil \frac{n}{2} \rceil$ we see Lemma 3.14 Item (a), Item (b) and Lemma 3.17 imply that each term m of q has the asserted form. $\square$

We end this section with a proof that the upper bound given in the preceding theorem is tight when n is even.

Theorem 3.19. *For any even integer n , the monomial $s_{12}s_{34} \dots s_{n-1,n} \in \mathcal{A}_n^{\text{swap}}$ cannot be written as a sum of monomials of degree less than $n/2$.*

Thus, for any n , we require polynomials of degree at least $\lfloor n/2 \rfloor$ to express all elements in the n qubit swap algebra $\mathcal{A}_n^{\text{swap}}$.

Proof. Consider the case where n is even. We prove the result for swap matrices, from which the theorem is immediate by Theorem 3.6. Consider the matrix $\text{Swap}_{12} \text{Swap}_{34} \dots \text{Swap}_{n-1,n}$. Expanding in the Pauli basis we see that it is supported on Pauli matrices of weight n (i.e. its expansion contains degree n products of Pauli matrices). But any product of at most $n/2 - 1$ swaps acts on at most $n - 2$ qubits, and so is supported on Pauli matrices of weight at most $n - 2$. Since the Pauli matrices form a orthogonal basis for $M_2(\mathbb{C})^{\otimes n}$, this shows the matrix $\text{Swap}_{12} \text{Swap}_{34} \dots \text{Swap}_{n-1,n}$ cannot be equal to a sum of products of swap matrices with degree at most $n/2 - 1$.

The result for odd n is immediate by considering the monomial $\text{Swap}_{12} \text{Swap}_{34} \dots \text{Swap}_{n-2,n-1}$. $\square$

4 The Swap Algebra and the Non-Commutative Sum of Squares Hierarchy

In this section we construct an ncSoS hierarchy for the Symbolic Swap Algebra. We begin by reviewing the general ncSoS technique in Section 4.2 and then show how it can be applied specifically to the swap algebra in Section 4.3.

Previously an ncSoS hierarchy applied over the Pauli algebra, the *Quantum Lasserre Hierarchy*, has been used to upper (or lower) bound the maximum (or minimum) eigenvalue of local Hamiltonians [BH13, GP19]. The Quantum Lasserre Hierarchy is general and capable of addressing any qubit Hamiltonian problem. Our hierarchy is distinct in that it is specific to the swap algebra. But in principle, an ncSoS hierarchy can be constructed for any suitably presented algebra. Hamiltonian problems studied in physics have algebraic structures associated with them. Thus, a message here is that ncSoS techniques can be adapted to the algebra of Hamiltonians and potentially give improved bounds to eigenvalue problems.

4.1 A Monomial Order – grlex

In computations with an algebra and their presentation via ideals we often need to place an order on monomials. In our situation the variables are $\{s_{ij} \mid 1 \leq i < j \leq n\}$ and we define a *graded lexicographic order* (*grlex*) $<$ on them as follows.

- (1) Order the alphabet as

$$s_{11} < s_{12} < \dots < s_{1n} < s_{23} < s_{24} < \dots < s_{2n} < \dots < s_{nn};$$

- (2) for any two monomials a and b in the s_{ij} , take $a < b$ if $\deg(a) < \deg(b)$;
- (3) if $\deg(a) = \deg(b)$, then look at them as words in the s_{ij} and sort them as you would in a dictionary according to alphabetical order.

4.2 Non-Commutative Sum Of Squares Hierarchy

Let $\mathcal{I}$ be a $*$ -ideal in $\mathbb{C}\langle s \rangle$. Suppose h is a symmetric polynomial, by which we mean that under the involution $*$, we have $h^* = h$. Define $\nu_d(h)$ to be the lower limit of upper bounds ν making

$$\nu - h \in \text{SOS}_{2d} + \mathcal{I}, \tag{4.1}$$

where SOS_{2d} denotes the set of all sums of squares of polynomials in the variables s_{ij} each having degree $\leq d$.

Note that if $\nu - h \in \text{SOS}_{2d} + \mathcal{I}$ for any d we must also have that $\pi(\nu - h) \geq 0$ for all representations $\pi : \mathbb{C}\langle s \rangle / \mathcal{I} \rightarrow \mathcal{B}(\mathcal{H})$. Thus, we have that any such ν , and in particular any $\nu_d(h)$, gives an upper bound on the max eigenvalue of h under all representations of $\mathbb{C}\langle s \rangle / \mathcal{I}$. For a fixed $\mathcal{I}$ we call this process the *dth relaxation* and the least upper bound $\nu_d(h)$ we call the *dth relaxed value*. Key to analyzing and computing this bound is expressing SOS_{2d} and membership in $\mathcal{I}$ succinctly.

Applying this formalism with $\mathcal{I} = \mathcal{I}^{\text{swap}}$ gives an upper bound

$$\nu_d(h) \geq \text{eig}_{\max}(h(\text{Swap})), \quad (4.2)$$

where $h(\text{Swap})$ denotes the matrix obtained by substituting each element s_{ij} in the polynomial h with the corresponding swap matrix Swap_{ij} ⁴, and we will investigate this particular situation more in the following section. For now we return to the case of a general ideal $\mathcal{I}$.

The Veroneses are column vectors, denoted $V_d(n)$, which consist of degree d monomials in the $n(n-1)/2$ variables s_{ij} , $i < j$, ordered w.r.t. grlex. This makes it possible to test membership in SOS_{2d} with the help of semidefinite programming (SDP).

Lemma 4.1. *Let $h = h^* \in \mathbb{C}\langle s \rangle$ be of degree $\leq 2d$. Then $h \in \text{SOS}$ iff there is a positive semidefinite matrix Γ such that*

$$h = V_d(n)^* \Gamma V_d(n). \quad (4.3)$$

Finding such a Γ can be done with an SDP.

Proof. This is well-known and routine [BKP16]. Eq. (4.3) yields a system of linear equations on the entries of Γ , so finding a positive semidefinite Γ satisfying these linear constraints amounts to a feasibility SDP. $\square$

Corollary 4.2. *Let $h = h^* \in \mathbb{C}\langle s \rangle$. Then $h \in \text{SOS}_{2d} + \mathcal{I}$ iff there is a positive semidefinite matrix Γ such that*

$$h - V_d(n)^* \Gamma V_d(n) \in \mathcal{I}. \quad (4.4)$$

Assuming a linear algebra basis for $\mathbb{C}\langle s \rangle / \mathcal{I}$ or a “good” generating set⁵ for $\mathcal{I}$ is known, finding such a Γ can be done with an SDP.

Proof. The first part of the statement follows as in Lemma 4.1. Then, to translate Eq. (4.4) into a linear system on the entries of Γ , we need to solve the ideal membership problem for $\mathcal{I}$; this can be done using a linear algebraic basis for the space $\mathbb{C}\langle s \rangle / \mathcal{I}$ or via Gröbner bases. For a longer discussion of this issue see Section 4.2.1. $\square$

With this, Eq. (4.1) can be expressed as the SDP

$$\begin{aligned} \nu_d(h) &= \inf \nu \\ \text{s.t. } \Gamma &\succeq 0 \\ \nu - h - V_d(n)^* \Gamma V_d(n) &\in \mathcal{I}. \end{aligned} \quad (4.5)$$

Remark 4.3. One can approximate Eq. (4.5) by using truncated ideals. Given generators $g_1, \dots, g_e$ for the ideal $\mathcal{I}$, form the degree $2d$ truncation of $\mathcal{I}$ as follows:

$$\mathcal{I}_{2d} := \text{span}\{ug_kv \mid u, v \text{ words in } s_{ij}, k = 1, \dots, e, \deg(ug_kv) \leq 2d\}.$$

⁴So, in particular, observe that we have $h_G(\text{Swap}) = H_G$ for any graph G .

⁵i.e., a Gröbner basis, cf. Section 5 below

(Beware, frequently, $\mathcal{I}_{2d} \subsetneq \mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}$!) Membership in $\mathcal{I}_{2d}$ can be expressed as a linear system on the coefficients in the linear combination. This yields the following approximation to Eq. (4.5):

$$\begin{aligned} \nu_d(h) = \inf \nu \\ \text{s.t. } \Gamma \succeq 0 \\ \nu - h - V_d(n)^* \Gamma V_d(n) \in \mathcal{I}_{2d}. \end{aligned} \quad (4.6)$$

Since $\mathcal{I}_{2d} \subsetneq \mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}$ in general, $\nu_d(h)$ of Eq. (4.6) can be strictly greater than $\nu_d(h)$ of Eq. (4.5). $\square$

Typically to solve a noncommutative sos SDP problem one applies standard duality and solves the associated dual SDP. We give its interpretation as an SDP involving pseudomoments. Following a standard Lagrangian duality argument, the dual SDP to the one in Eq. (4.5) is

$$\begin{aligned} \alpha_d(h) = \sup L(h) \\ \text{s.t. } L \in (\text{SOS}_{2d} + \mathcal{I})^\vee \\ L(1) = 1. \end{aligned} \quad (4.7)$$

Here $(\text{SOS}_{2d} + \mathcal{I})^\vee$ denotes the dual cone to the cone $\text{SOS}_{2d} + \mathcal{I}$,

$$(\text{SOS}_{2d} + \mathcal{I})^\vee = \{L : \mathbb{C}\langle s \rangle_{2d} \rightarrow \mathbb{C} \mid L \text{ *-linear with } L(\text{SOS}_{2d}) \subseteq \mathbb{R}_{\geq 0}, L(\mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}) = \{0\}\}.$$

As above, by replacing $\mathcal{I}$ in Eq. (4.7) by its truncation $\mathcal{I}_{2d}$, we obtain the approximation

$$\begin{aligned} \bar{\alpha}_d(h) = \sup L(h) \\ \text{s.t. } L \in (\text{SOS}_{2d} + \mathcal{I}_{2d})^\vee \\ L(1) = 1. \end{aligned} \quad (4.8)$$

Remark 4.4. Weak duality always holds for a primal-dual SDP pair, i.e.,

$$\nu_d(h) \geq \alpha_d(h), \quad (4.9)$$

and under natural mild conditions, strong duality holds, that is, we have equality in Eq. (4.9). This holds, for example, in the presence of so-called Slater points, a condition that is satisfied for $\mathcal{I} = \mathcal{I}^{\text{swap}}$, cf. Proposition 4.9 below. $\square$

Consider the pseudomoment matrix pattern $\mathcal{M}$ with symbolic entries

$$\mathcal{M}_d^n := V_d(n) V_d(n)^*.$$

Lemma 4.5. For $L : \mathbb{C}\langle s \rangle_{2d} \rightarrow \mathbb{C}$ *-linear, $L(\text{SOS}_{2d}) \subseteq \mathbb{R}_{\geq 0}$ iff $(I \otimes L)(\mathcal{M}_d^n) \succeq 0$.

Proof. For $p \in \text{SOS}_{2d}$, assume $p = \sum_j p_j^* p_j$, and write each $p_j = V_d(n)^* \vec{p}_j$, where $\vec{p}_j$ is the (column) vector of coefficients of p_j . Then

$$L(p) = \sum_j L(p_j^* p_j) = \sum_j L(\vec{p}_j^* V_d(n) V_d(n)^* \vec{p}_j) = \sum_j \vec{p}_j^* (I \otimes L)(\mathcal{M}_d(n)) \vec{p}_j,$$

from which the conclusion follows. $\square$

We call the entries of $\mathcal{M}_d(L) := (I \otimes L)(\mathcal{M}_d^n)$ the pseudomoments (of degree $\leq 2d$) of L . We can now rewrite Eq. (4.7) as an SDP as follows:

$$\begin{aligned} \alpha_d(h) = \sup \quad & L(h) \\ \text{s.t.} \quad & \mathcal{M}_d(L) \succeq 0 \\ & \mathcal{M}_d(L)_{1,1} = 1 \\ & L(\mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}) = \{0\}. \end{aligned} \quad (4.10)$$

The objective function $L(h)$ can be equivalently presented as

$$L(h) = \langle \mathcal{M}_d(L), \Gamma_h \rangle, \quad (4.11)$$

where Γ_h is a (not necessarily positive semidefinite) Gram matrix for h , i.e., $h = V_d(n)^* \Gamma_h V_d(n)$. We note that the right-hand side of Eq. (4.11) is independent of the Gram representation Γ_h chosen.

Remark 4.6. In general the supremum above, e.g. in Eq. (4.7) is taken all $*$ -linear functionals $L : \mathbb{C}\langle s \rangle_{2d} \rightarrow \mathbb{C}$. However in the case where $h^* = h \in \mathbb{R}\langle s \rangle$ and $\mathcal{I}^* = \mathcal{I}$ is generated by polynomials in $\mathbb{R}\langle s \rangle$, we can simplify this supremum and only consider $*$ -linear functionals $L : \mathbb{R}\langle s \rangle_{2d} \rightarrow \mathbb{R}$. To see why, assume we are in this case and note that for any function L achieving this supremum, we also have that the function L' defined by $L'(p) = \frac{1}{2} (L(p) + \overline{L(p)})$ also achieves this supremum. Since $L' : \mathbb{R}\langle s \rangle_{2d} \rightarrow \mathbb{R}$ this simplification is justified. $\square$

Example 4.7. Consider the case where $\mathcal{I} = \{0\}$. In this case the constraint $L(\mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}) = \{0\}$ is automatically satisfied.

Take $n = 3$, $d = 1$. Then $V_1(3) = (1, s_{12}, s_{13}, s_{23})^*$ and an arbitrary degree one polynomial $p \in \mathbb{C}\langle s \rangle$ has the form

$$p = p_0 + p_1 s_{12} + p_2 s_{13} + p_3 s_{23} = V_1(3)^* \vec{p} \quad \text{with} \quad \vec{p} := (p_0, p_1, p_2, p_3)^T \quad \text{and} \quad (4.12)$$

$$p^* p = \vec{p}^* \mathcal{M}_1^3 \vec{p} \quad \text{with} \quad \mathcal{M}_1^3 = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{23} \\ s_{12} & s_{12}^2 & s_{12}s_{13} & s_{12}s_{23} \\ s_{13} & s_{13}s_{12} & s_{13}^2 & s_{13}s_{23} \\ s_{23} & s_{23}s_{12} & s_{23}s_{13} & s_{23}^2 \end{bmatrix} \quad (4.13)$$

If $L : \mathbb{R}\langle s \rangle_2 \rightarrow \mathbb{R}$ is $*$ -linear, then

$$\begin{aligned} \mathcal{M}_1(L) &= \begin{bmatrix} L(1) & L(s_{12}) & L(s_{13}) & L(s_{23}) \\ L(s_{12}) & L(s_{12}^2) & L(s_{12}s_{13}) & L(s_{12}s_{23}) \\ L(s_{13}) & L(s_{13}s_{12}) & L(s_{13}^2) & L(s_{13}s_{23}) \\ L(s_{23}) & L(s_{23}s_{12}) & L(s_{23}s_{13}) & L(s_{23}^2) \end{bmatrix} \\ &= \begin{bmatrix} L(1) & L(s_{12}) & L(s_{13}) & L(s_{23}) \\ L(s_{12}) & L(s_{12}^2) & L(s_{12}s_{13}) & L(s_{12}s_{23}) \\ L(s_{13}) & L(s_{12}s_{13}) & L(s_{13}^2) & L(s_{13}s_{23}) \\ L(s_{23}) & L(s_{12}s_{23}) & L(s_{13}s_{23}) & L(s_{23}^2) \end{bmatrix} \end{aligned}$$

is a self-adjoint matrix.

Consider the Hamiltonian given by the polynomial $h = s_{12} + s_{13} + s_{23}$. The classical first relaxation Eq. (4.10) when the ideal $\mathcal{I}$ is $\{0\}$ (noting that we can restrict to linear functionals $L : \mathbb{R}\langle s \rangle_{2d} \rightarrow \mathbb{R}$ by Remark 4.6) becomes:

$$\alpha_1(h) = \max \ell_{12} + \ell_{13} + \ell_{23}$$

$$\begin{aligned} \text{s.t. } & \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} \\ \ell_{12} & \ell_{12,12} & \ell_{12,13} & \ell_{12,23} \\ \ell_{13} & \ell_{12,13} & \ell_{13,13} & \ell_{13,23} \\ \ell_{23} & \ell_{12,23} & \ell_{13,23} & \ell_{23,23} \end{bmatrix} \succeq 0 \\ & = \infty \end{aligned}$$

where $\ell_{12}, \ell_{13}, \ell_{23}, \ell_{12,12}, \ell_{12,13}, \ell_{12,23}, \ell_{13,23} \in \mathbb{R}$ are arbitrary, and should be thought of as giving the value of the linear functional L applied to the variables s_{ij} and monomials in them in the matrix $\mathcal{M}_1(L)$ above.

This gives an upper bound on the eigenvalues of the sum of three (arbitrary) self-adjoint matrices which is vacuous. However, we soon see other ideals where the first relaxation bound has substance, for example in Example 4.8.

Example 4.8. We repeat Example 4.7 but with $\mathcal{I}$ changed. We now take $\mathcal{I} = \mathcal{I}^{S_n}$, which we define to be the ideal generated by the elements

$$s_{ij}^2 - 1, \quad s_{ij}s_{jk} - s_{ik}s_{ij}, \quad s_{ij}s_{kl} - s_{kl}s_{ij} \quad i, j, k, l \text{ all distinct} \quad (4.14)$$

which we can think of as enforcing the defining relations

$$s_{ij}^2 = 1, \quad s_{ij}s_{jk} = s_{ik}s_{ij}, \quad s_{ij}s_{kl} = s_{kl}s_{ij} \quad i, j, k, l \text{ all distinct} \quad (4.15)$$

of the symmetric group S_n . We will see in a moment how enforcing the condition $L(\mathcal{I} \cap \mathbb{C}\langle s \rangle_{2d}) = \{0\}$ leads to linear constraints on the pseudomoment matrix pattern.

We begin with the pseudomoment matrix pattern

$$\mathcal{M}_1^3 = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{23} \\ s_{12} & s_{12}^2 & s_{12}s_{13} & s_{12}s_{23} \\ s_{13} & s_{13}s_{12} & s_{13}^2 & s_{13}s_{23} \\ s_{23} & s_{23}s_{12} & s_{23}s_{13} & s_{23}^2 \end{bmatrix} \quad (4.16)$$

obtained in Example 4.7. Now viewing these elements as elements of the algebra $\mathbb{C}\langle s \rangle / \mathcal{I}^{S_n}$ (or, equivalently, applying the identities given in Equation (4.15) and recalling that $s_{ij} = s_{ji}$ for any i, j) we see this pseudomoment matrix pattern is equivalent to the pseudomoment matrix pattern

$$\mathcal{M}_1^3 = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{23} \\ s_{12} & 1 & s_{12}s_{13} & s_{13}s_{12} \\ s_{13} & s_{13}s_{12} & 1 & s_{12}s_{13} \\ s_{23} & s_{12}s_{13} & s_{13}s_{12} & 1 \end{bmatrix} = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{23} \\ s_{12} & 1 & s_{12}s_{13} & (s_{12}s_{13})^* \\ s_{13} & (s_{12}s_{13})^* & 1 & s_{12}s_{13} \\ s_{23} & s_{12}s_{13} & (s_{12}s_{13})^* & 1 \end{bmatrix} \quad (4.17)$$

Writing $\mathcal{M}_1^3$ in this way we see that the first relaxation is:

$$\begin{aligned} \alpha_1(h) &= \max \ell_{12} + \ell_{13} + \ell_{23} \\ \text{s.t. } & \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} \\ \ell_{12} & 1 & \ell_{12,13} & (\ell_{12,13})^* \\ \ell_{13} & (\ell_{12,13})^* & 1 & \ell_{12,13} \\ \ell_{23} & \ell_{12,13} & (\ell_{12,13})^* & 1 \end{bmatrix} \succeq 0. \end{aligned} \quad (4.18)$$

Where we can take $\ell_{12}, \ell_{13}, \ell_{23}, \ell_{12,13}$ to be real valued by Remark 4.6. But then we also have $\ell_{12,13} = (\ell_{12,13})^*$ and the relaxation simplifies slightly further to

$$\alpha_1(h) = \max \ell_{12} + \ell_{13} + \ell_{23}$$

$$\begin{aligned} \text{s.t. } & \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} \\ \ell_{12} & 1 & \ell_{12,13} & \ell_{12,13} \\ \ell_{13} & \ell_{12,13} & 1 & \ell_{12,13} \\ \ell_{23} & \ell_{12,13} & \ell_{12,13} & 1 \end{bmatrix} \succeq 0 \\ & = 3. \end{aligned}$$

We refer the reader to Appendix D for the second relaxation.

In the example above we simplified the pseudomoment matrix pattern in a somewhat ad-hoc manner. In that case it is not too difficult to verify that that simplification was complete, i.e. that there were no linear constraints amongst entries of the pseudomoment matrix pattern that were missed when translating to the SDP. But for larger moment matrices or more complicated ideals it is not always obvious when the pseudomoment matrix pattern is fully reduced. In the next section (Section 4.2.1) we will discuss some techniques that can be used to find all the linear constraints amongst entries of an SDP that are enforced by a given ideal.

4.2.1 Finding Linear Constrains in the Moment Matrix

We now illustrate some terminology and discuss possible approaches to finding all the linear constraints in the SDP computing the d th relaxed value $\alpha_d(h)$ of some element $h \in \mathbb{C}\langle s \rangle / \mathcal{I}$ when working with a general $*$ -ideal $\mathcal{I}$. For concreteness, we will frequently refer back to the example in the previous section where we computed these linear constraints at level $d = 1$ for the ideal $\mathcal{I}^{S_3}$.

There are two general approaches that will be discussed in this paper.

- (1) *Equations become rules.* Every element of $\mathcal{I}$ can be thought of as enforcing equality between sums of monomials. A particularly nice case is when the equality is just between pairs of monomials – as occurred in Equation (4.15). Given two equivalent monomials, we want to pick a suitable representative. We can use the grlex monomial order of Section 4.1 to do this: the larger monomial in grlex order are replaced by the smaller. This procedure applied to the equalities in Equation (4.15) yields the following “replacement rules”

$$s_{ij}^2 \rightarrow 1, \quad s_{23}s_{13} \rightarrow s_{12}s_{23}, \quad s_{13}s_{12} \rightarrow s_{12}s_{23}, \quad (4.19)$$

$$s_{23}s_{12} \rightarrow s_{12}s_{13}, \quad s_{13}s_{23} \rightarrow s_{12}s_{13}. \quad (4.20)$$

When we apply the rules to a polynomial p , for example, take $p = s_{23}s_{12}s_{13}^2 + s_{12}^2$, then we get another polynomial $\tilde{p} = s_{12}s_{13} + 1$, having the key property $p - \tilde{p} \in \mathcal{I}^{S_n}$. Applying these rules to the monomials in the initial moment matrix pattern of Example 4.8 clearly gives $\mathcal{M}_3^1$ of Eq. (4.16) above.

For general ideals and their defining equations (or generators) we can similarly associate a list grlex based of rules. Then for some p , after applying rules, $\tilde{p}$ may depend on the order in which the rules are applied and even worse $p \in \mathcal{I}$ might not reduce to $\tilde{p} = 0$. Later, in Section 5, we show how this problem can be fixed by selecting rules corresponding to a set of generators for $\mathcal{I}$ called a Gröbner Basis (GB). Indeed the rules Equation (4.19) do correspond to a GB for S_3 w.r.t. the grlex monomial order. A longer discussion of Gröbner Basis can be found in Section 5.

- (2) *Linear algebra basis.* An alternate approach is possible if a good understanding of the quotient algebra $\mathcal{A}_3^{\mathcal{I}^{S_3}} := \mathbb{C}\langle s \rangle / \mathcal{I}^{S_3}$ is available. Namely, if a linear basis for the quotient space can be obtained (at least for images of polynomials up to a certain degree), then simple linear algebra allows us to form (lower) levels of the relaxation hierarchy.

For instance, a basis for the image of $\mathbb{C}\langle s \rangle_2$ in $\mathcal{A}_3^{\mathcal{I}^{S_3}}$ is

$$1, s_{12}, s_{13}, s_{12}s_{13}, s_{13}s_{12},$$

since all monomials in $\mathbb{C}\langle s \rangle_2$ are a linear combination of these modulo the $\mathcal{I}^{S_3}$, as can be derived from Equation (4.14). Thus producing a moment matrix pattern for the ideal S_3 is done by using the trivial linear expansion of monomials in terms of this basis,

$$s_{23}s_{13} = s_{12}s_{23}, \quad s_{13}s_{12} = s_{12}s_{23}, \quad s_{23}s_{12} = s_{12}s_{13}, \quad s_{13}s_{23} = s_{12}s_{13}.$$

The symmetric group S_3 is so simple that the distinction between the rule approach and the linear basis approach is merely semantic. However, for swaps the difference has substance. Indeed, Section 4.3 takes the linear basis approach while Section 5 takes the Gröbner Basis approach.

4.3 Non-Commutative Sum Of Squares Hierarchy for the Swap algebra

In this section we restrict our attention to the swap algebra. We begin with a brief discussion of convergence of Sum of Squares relaxations in this algebra. Then we turn our attention to implementations of the 1st and 2nd relaxations in the swap algebra using linear algebraic bases. We begin in Section 4.3.1 with an example of the 1st relaxation using a linear algebraic basis, then in Section 4.3.2, construct a linear algebraic basis for monomials of degree at most two in the swap algebra. In Section 4.3.3 we discuss an implementation of the 2nd swap relaxation using a linear algebraic basis for swap algebra monomials of degree at most four which we construct in Appendix B. We end this section with a discussion of some numerical results obtained using the 2nd swap relaxation.

Proposition 4.9. *Strong duality holds between the primal-dual pair of SDPs in Eq. (4.5) and Eq. (4.10).*

Proof. It suffices to find a strictly feasible point L for the dual SDP in Eq. (4.10). Equivalently, a strictly positive linear functional ℓ on the Swap algebra $\mathcal{A}_n^{\text{swap}}$, or equivalently, on M_n^{swap} . But this is easy: simply take the trace. $\square$

Theorem 4.10. *For $h = h^* \in \mathcal{A}_n^{\text{swap}}$, the sequences $\nu_d(h)$ and $\alpha_d(h)$ defined in Eq. (4.5) and Eq. (4.10) converge monotonically to $\text{eig}_{\max}(h(\text{Swap}))$. More precisely, for $d \geq \lceil \frac{n}{2} \rceil$,*

$$\nu_d(h) = \text{eig}_{\max}(h(\text{Swap})) = \alpha_d(h).$$

Proof. The convergence statement follows from the Helton-McCullough Positivstellensatz [HM04] since the algebra $\mathcal{A}_n^{\text{swap}}$ is finite-dimensional and hence archimedean. The convergence is finite by Theorem 3.18. $\square$

4.3.1 First relaxation for the Swap algebra

We now present in some detail the SDP arising from the first noncommutative sum of squares relaxation for the Swap algebra. We proceed as in Example 4.7 and Example 4.8 and define $V_1(n) = (1, s_{12}, s_{13}, \dots, s_{1n}, s_{23}, \dots, s_{n-1n})^*$ and consider $\mathcal{M}_1(L)$ for some $*$ -linear functional $L : \mathbb{C}\langle s \rangle_2 \rightarrow \mathbb{C}$ which satisfies $L(\mathbb{R}\langle s \rangle_2) \subseteq \mathbb{R}$.

To encode the constraint $L(\mathcal{I}^{\text{swap}} \cap \mathbb{C}\langle s \rangle_2) = \{0\}$, we need to understand the linear space spanned by monomials of degree at most two in the swap algebra. We construct a basis $\mathcal{B}_2$ for this space together with expansions for all the other products in terms of this basis to encode Eq. (4.10). This follows the approach outlined in Item (2) of Section 4.2.1.

Example 4.11. Swap Algebra, $n = 3$. A basis $\mathcal{B}_2$ for the linear space spanned by monomials of degree at most two in the swap algebra is given by the entries of $V_1(3)$ together with one element, e.g., $s_{12}s_{13}$. All the other quadratic terms in the s_{ij} can be expressed with these as follows:

$$\begin{aligned} s_{ij}^2 &= 1 \\ s_{12}s_{23} &= -1 + s_{12} + s_{13} + s_{23} - s_{12}s_{13} \\ s_{13}s_{12} &= -1 + s_{12} + s_{13} + s_{23} - s_{12}s_{13} \\ s_{13}s_{23} &= s_{12}s_{13} \\ s_{23}s_{12} &= s_{12}s_{13} \\ s_{23}s_{13} &= -1 + s_{12} + s_{13} + s_{23} - s_{12}s_{13} \end{aligned}$$

With this a unital $*$ -linear L with $L(\mathcal{I}^{\text{swap}} \cap \mathbb{C}\langle s \rangle_2) = \{0\}$ is determined by 3 real numbers $\ell_{ij} = L(s_{ij})$ and a complex number $q = L(s_{12}s_{13})$. Thus $\mathcal{M}_1(L)$ simplifies into

$$\begin{aligned} \mathcal{M}_1(L) &= \begin{bmatrix} L(1) & L(s_{12}) & L(s_{13}) & L(s_{23}) \\ L(s_{12}) & L(s_{12}^2) & L(s_{12}s_{13}) & L(s_{12}s_{23}) \\ L(s_{13}) & L(s_{12}s_{13})^* & L(s_{13}^2) & L(s_{13}s_{23}) \\ L(s_{23}) & L(s_{12}s_{23})^* & L(s_{13}s_{23})^* & L(s_{23}^2) \end{bmatrix} \\ &= \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} \\ \ell_{12} & 1 & q & -1 + \ell_{12} + \ell_{13} + \ell_{23} - q \\ \ell_{13} & q^* & 1 & q \\ \ell_{23} & -1 + \ell_{12} + \ell_{13} + \ell_{23} - q^* & q^* & 1 \end{bmatrix}. \end{aligned}$$

Conversely, each $\mathcal{M}_1(L)$ of this form yields a unital $*$ -linear L with $L(\mathcal{I}^{\text{swap}} \cap \mathbb{C}\langle s \rangle_2) = \{0\}$. This makes it straightforward to write down the SDP of Eq. (4.10) for the objective function $L(h)$ for $h = \ell_{12} + \ell_{13} + \ell_{23}$ in this case.

$$\begin{aligned} &\max \ell_{12} + \ell_{13} + \ell_{23} \\ &\text{s.t.} \quad \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} \\ \ell_{12} & 1 & q & -1 + \ell_{12} + \ell_{13} + \ell_{23} - q \\ \ell_{13} & q & 1 & q \\ \ell_{23} & -1 + \ell_{12} + \ell_{13} + \ell_{23} - q & q & 1 \end{bmatrix} \succeq 0 \\ &= 3. \end{aligned}$$

Here $\ell_{ij} \in \mathbb{R}$, and as explained in Remark 4.6, we have without loss of generality assumed that $q = L(s_{12}s_{13}) \in \mathbb{R}$.

The examples illustrate SDP relaxations using linear algebra constraints to capture $\mathcal{I}^{\text{swap}}$ when $n = 3$. The rest of this section and related appendices give the machinery needed, namely $\mathcal{B}_3, \mathcal{B}_4$, for any n and for 2nd swap relaxations.

4.3.2 Linear space spanned by the products of at most two swap matrices

Proposition 4.12. A basis $\mathcal{B}_2$ for the linear space spanned by monomials of degree at most two in the swap algebra M_n^{swap} is given by

$$\begin{aligned} &I \\ &\text{Swap}_{ij} \quad i < j \\ &\text{Swap}_{ij}\text{Swap}_{ik} \quad i < j < k \\ &\text{Swap}_{ij}\text{Swap}_{k\ell} \quad i < j, i < k < \ell. \end{aligned}$$

Proof. Let us first prove the spanning property of $\mathcal{B}_2$. Since two swap matrices with disjoint indices commute, it suffices to consider products of two swap matrices, where one of the indices repeats. Letting $i < j < k$, there are six such products, namely

$$\text{Swap}_{ij} \text{Swap}_{ik}, \text{Swap}_{ik} \text{Swap}_{ij}, \text{Swap}_{ij} \text{Swap}_{jk}, \text{Swap}_{jk} \text{Swap}_{ij}, \text{Swap}_{ik} \text{Swap}_{jk}, \\ \text{Swap}_{jk} \text{Swap}_{ik}.$$

The first of these has been included in $\mathcal{B}_2$. The second product can be expressed with the first one and linear terms in Swap_{pq} using Eq. (Δ). Further, by a routine calculation we have⁶

$$\begin{aligned} \text{Swap}_{ij} \text{Swap}_{jk} &= -1 + \text{Swap}_{ij} + \text{Swap}_{ij} + \text{Swap}_{jk} - \text{Swap}_{ij} \text{Swap}_{ik} \in \text{span } \mathcal{B}_2 \\ \text{Swap}_{jk} \text{Swap}_{ij} &= \text{Swap}_{ij} \text{Swap}_{ik} \in \mathcal{B}_2 \\ \text{Swap}_{ik} \text{Swap}_{jk} &= \text{Swap}_{ij} \text{Swap}_{ik} \in \mathcal{B}_2 \\ \text{Swap}_{jk} \text{Swap}_{ik} &= -1 + \text{Swap}_{ij} + \text{Swap}_{ij} + \text{Swap}_{jk} - \text{Swap}_{ij} \text{Swap}_{ik} \in \text{span } \mathcal{B}_2. \end{aligned}$$

We now turn to the linear independence of $\mathcal{B}_2$. Assume there are scalars $\alpha, \beta_{ij}, \gamma_{ijk}, \delta_{ijkl}$ satisfying

$$\alpha I + \sum_{i < j} \beta_{ij} \text{Swap}_{ij} + \sum_{i < j < k} \gamma_{ijk} \text{Swap}_{ij} \text{Swap}_{ik} + \sum_{\substack{i < j \\ i < k < \ell}} \delta_{ijkl} \text{Swap}_{ij} \text{Swap}_{k\ell} = 0. \quad (4.21)$$

Before expanding the left-hand side of Eq. (4.21) in terms of the Pauli σ_W , note that for $i < j < k$,

$$\begin{aligned} 4 \text{Swap}_{ij} \text{Swap}_{ik} &= 1 + \sigma_X^i \sigma_X^j + \sigma_X^i \sigma_X^k + \sigma_X^j \sigma_X^k + \sigma_Y^i \sigma_Y^j + \sigma_Y^i \sigma_Y^k + \sigma_Y^j \sigma_Y^k + \sigma_Z^i \sigma_Z^j + \sigma_Z^i \sigma_Z^k + \sigma_Z^j \sigma_Z^k \\ &\quad + i(\sigma_X^i \sigma_Y^j \sigma_Z^k - \sigma_X^i \sigma_Z^j \sigma_Y^k - \sigma_Y^i \sigma_X^j \sigma_Z^k + \sigma_Y^i \sigma_Z^j \sigma_X^k + \sigma_Z^i \sigma_X^j \sigma_Y^k - \sigma_Z^i \sigma_Y^j \sigma_X^k). \end{aligned}$$

Likewise, letting $i < j < k < \ell$, we have

$$\begin{aligned} 4 \text{Swap}_{ij} \text{Swap}_{k\ell} &= 1 + \sigma_X^i \sigma_X^j + \sigma_X^k \sigma_X^\ell + \sigma_Y^i \sigma_Y^j + \sigma_Y^k \sigma_Y^\ell + \sigma_Z^i \sigma_Z^j + \sigma_Z^k \sigma_Z^\ell \\ &\quad + \sigma_X^i \sigma_X^j \sigma_X^k \sigma_X^\ell + \sigma_X^i \sigma_X^j \sigma_Y^k \sigma_Y^\ell + \sigma_X^i \sigma_X^j \sigma_Z^k \sigma_Z^\ell + \sigma_Y^i \sigma_Y^j \sigma_X^k \sigma_X^\ell \\ &\quad + \sigma_Y^i \sigma_Y^j \sigma_Y^k \sigma_Y^\ell + \sigma_Y^i \sigma_Y^j \sigma_Z^k \sigma_Z^\ell + \sigma_Z^i \sigma_Z^j \sigma_X^k \sigma_X^\ell + \sigma_Z^i \sigma_Z^j \sigma_Y^k \sigma_Y^\ell + \sigma_Z^i \sigma_Z^j \sigma_Z^k \sigma_Z^\ell. \end{aligned}$$

If $i < k < j < \ell$, then

$$\begin{aligned} 4 \text{Swap}_{ij} \text{Swap}_{k\ell} &= 1 + \sigma_X^i \sigma_X^j + \sigma_X^k \sigma_X^\ell + \sigma_Y^i \sigma_Y^j + \sigma_Y^k \sigma_Y^\ell + \sigma_Z^i \sigma_Z^j + \sigma_Z^k \sigma_Z^\ell \\ &\quad + \sigma_X^i \sigma_X^k \sigma_X^j \sigma_X^\ell + \sigma_X^i \sigma_Y^k \sigma_X^j \sigma_Y^\ell + \sigma_X^i \sigma_Z^k \sigma_X^j \sigma_Z^\ell + \sigma_Y^i \sigma_X^k \sigma_Y^j \sigma_X^\ell \\ &\quad + \sigma_Y^i \sigma_Y^k \sigma_Y^j \sigma_Y^\ell + \sigma_Y^i \sigma_Z^k \sigma_Y^j \sigma_Z^\ell + \sigma_Z^i \sigma_X^k \sigma_Z^j \sigma_X^\ell + \sigma_Z^i \sigma_Y^k \sigma_Z^j \sigma_Y^\ell + \sigma_Z^i \sigma_Z^k \sigma_Z^j \sigma_Z^\ell \end{aligned}$$

Finally, when $i < k < \ell < j$, then

$$\begin{aligned} 4 \text{Swap}_{ij} \text{Swap}_{k\ell} &= 1 + \sigma_X^i \sigma_X^j + \sigma_X^k \sigma_X^\ell + \sigma_Y^i \sigma_Y^j + \sigma_Y^k \sigma_Y^\ell + \sigma_Z^i \sigma_Z^j + \sigma_Z^k \sigma_Z^\ell \\ &\quad + \sigma_X^i \sigma_X^k \sigma_X^\ell \sigma_X^j + \sigma_X^i \sigma_Y^k \sigma_Y^\ell \sigma_Y^j + \sigma_X^i \sigma_Z^k \sigma_Z^\ell \sigma_Z^j + \sigma_Y^i \sigma_X^k \sigma_X^\ell \sigma_X^j \\ &\quad + \sigma_Y^i \sigma_Y^k \sigma_Y^\ell \sigma_Y^j + \sigma_Y^i \sigma_Z^k \sigma_Z^\ell \sigma_Z^j + \sigma_Z^i \sigma_X^k \sigma_X^\ell \sigma_X^j + \sigma_Z^i \sigma_Y^k \sigma_Y^\ell \sigma_Y^j + \sigma_Z^i \sigma_Z^k \sigma_Z^\ell \sigma_Z^j \end{aligned}$$

⁶While the identities given in this section can be easily verified since they involve matrices of small to moderate size, we have in fact discovered many of them with the help of noncommutative Gröbner bases; cf. Section 5 below.

We are now ready to expand Eq. (4.21). We shall make heavy use of the Pauli basis as given in Eq. (2.4). Consider the term $\text{Swap}_{ij}\text{Swap}_{k\ell}$ next to δ_{ijkl} . If $i < j < k < \ell$, then the expansion will contain $\sigma_X^i \sigma_X^j \sigma_Y^k \sigma_Y^\ell$, a term that cannot appear anywhere else in Eq. (4.21). If $i < k < j < \ell$, then the expansion contains $\sigma_X^i \sigma_Y^k \sigma_X^j \sigma_Y^\ell$, again a term that cannot appear anywhere else in Eq. (4.21). Finally, if $i < k < \ell < j$, then the same conclusion holds for $\sigma_X^i \sigma_Y^k \sigma_Y^\ell \sigma_X^j$. Thus $\delta_{ijkl} = 0$ for all i, j, k, ℓ .

We next consider $\gamma_{ijk}\text{Swap}_{ij}\text{Swap}_{ik}$. Here the expansion contains $\sigma_X^i \sigma_Y^j \sigma_Z^k$ that is again a term that cannot appear elsewhere in Eq. (4.21). Thus all $\gamma_{ijk} = 0$.

Similar but easier reasoning will also imply $\beta_{ij} = 0$ and finally $\alpha = 0$ proving linear independence of $\mathcal{B}_2$. $\square$

Remark 4.13. The set $\mathcal{B}_2$ has

$$1 + \binom{n}{2} + \binom{n}{3} + 3\binom{n}{4} = \frac{1}{24} (3n^4 - 14n^3 + 33n^2 - 22n + 24)$$

elements. $\square$

4.3.3 Second relaxation for the Swap algebra

We continue our discussion of relaxation by describing the second relaxation. This requires sets of monomials $\mathcal{B}_3, \mathcal{B}_4$ (to be described later in Appendices B.1 and B.2 which are degree 3 and 4 analogs of $\mathcal{B}_2$).

In order to build the second relaxation, we replace $V_2(n)$ by the vector $\vec{\mathcal{B}}_2$ consisting of the swap basis $\mathcal{B}_2$ determined in Proposition 4.12. The corresponding symbolic pseudomoment matrix $\vec{\mathcal{B}}_2 \vec{\mathcal{B}}_2^*$ will contain products of up to four swaps. To form the SDPs in Eq. (4.5) or Eq. (4.10) we need to understand the linear space spanned by all products of up to four Swap matrices, a topic we describe in some detail below in Appendix B.

Algorithm 1: 2nd relaxation of the quantum max-cut

Input : Graph $G = (V, E)$ on n vertices

Output : Solution to the 2nd relaxation of quantum max-cut for h_G . Alternately, “upper bound on the minimum eigenvalue of the QMC Hamiltonian”

Form $\vec{\mathcal{B}}_2$;

Form the symbolic pseudomoment matrix $\mathcal{M}_2 := \vec{\mathcal{B}}_2 \vec{\mathcal{B}}_2^*$;

Express each entry in $\mathcal{M}_2$ as a linear combination of elements of $\mathcal{B}_4$ (see Appendix B.2) to obtain $\mathcal{M}_2'$;

Replace each distinct term appearing in $\mathcal{M}_2'$ with a new (scalar) variable; call the resulting matrix $\mathcal{M}_2''$;

Solve the SDP

$$\lambda_2(h_G) = \sup\{\langle \mathcal{M}_2'', \Gamma_G \rangle \mid \mathcal{M}_2'' \succeq 0, (\mathcal{M}_2'')_{1,1} = 1\}, \quad (4.22)$$

where Γ_G is any matrix satisfying $h_G = \vec{\mathcal{B}}_2^* \Gamma_G \vec{\mathcal{B}}_2$;

return $\lambda_2(h_G)$

4.3.4 Swap relaxation behaves well in experiments

We implemented Algorithm 1 in Mathematica (using the out-of-the-box semidefinite optimization module in Mathematica) on all graphs with ≤ 8 vertices:

Proposition 4.14. *For $n \leq 8$ the 2nd relaxed value of an n vertex Quantum Max Cut Hamilton with uniform edge weights is up to the tolerance of 10^{-7} equal to the max eigenvalue of that Hamiltonian.*

It would be interesting to find the smallest graph on which the second relaxation is not exact.

Remark 4.15. Proposition 4.14 is surprising when compared to classical (commutative) max cut relaxation which performs much worse.

For example, the second Lasserre relaxation is not exact for the 5 cycle. Also, while we do not give comparative statistics, the first classical relaxation is worse than the one for swaps, e.g., even the triangle is classically not first Lasserre exact. $\square$

4.3.5 Segue

Since, clearly, it is not feasible to find linear algebra bases for linear spaces spanned by products of $d > 4$ swaps by hand, we turn to Gröbner bases in next.

5 Gröbner Bases

This section gives a few observations about Gröbner Bases (GB) with an eye toward computation.

5.1 Our Gröbner Basis Set Up

Our presentation is aimed at readers who have a basic familiarity with GBs; a standard reference in the commutative setting is [CLO15], while [Mor86, Gre00, Xiu12] describe the appropriate nc analogs. Noncommutative GBs have properties similar to those of commutative GB with the dramatic exception that a noncommutative GB might not be finite. However, we start with some conceptual motivation to provide a little context for readers who are unfamiliar with GBs.

5.1.1 Motivation for why we need Gröbner generators vs any old set of generators for an ideal

The reader is advised to review Example 4.8 which foreshadowed some of the ideas here.

Now we describe precisely the two big drawbacks of doing calculations on an ideal $\mathcal{I}$ using a set of generators for $\mathcal{I}$ which is too small. Let B be a set of generators for $\mathcal{I}$ a two sided ideal in the free algebra $\mathbb{C}\langle s \rangle$, and let $\mathcal{A} := \mathbb{C}\langle s \rangle / \mathcal{I}$. A basic question is: how do we tell if two elements $a, \tilde{a}$ of $\mathcal{A}$ are the same? Equivalently, do two given nc polynomials $p, \tilde{p}$ have difference $\delta := p - \tilde{p}$ in $\mathcal{I}$?

The simplest naïve approach is to use the graded lexicographic (grlex) monomial order introduced in Section 4.1. Then each polynomial $b \in B$ is a weighted sum of monomials; one of these monomials is bigger than the rest in the monomial order. Denote it by $\text{LT}(b)$ and construct a replacement rule, denoted r_b via

$$\text{LT}(b) \rightarrow b - \text{LT}(b)$$

Now apply the rules r_B gotten from the polynomials in B repeatedly to a polynomial p until they no longer cause any changes⁷; this leaves us with a polynomial decomposition

$$p = \bar{p}^B + \delta.$$

⁷The fact that we are using grlex – an “admissible monomial order” – implies that the successive application of rules will stop.

where $\delta \in \mathcal{I}$.

All of this does not tell us if p actually is in the ideal $\mathcal{I}$ and the remainder $\bar{p}^B$ might even depend on the order in which the rules are applied. This is extremely unsatisfying. Next, Gröbner Bases to the rescue: B is called a *Gröbner Basis* if the remainder $\bar{p}^B$ of a polynomial p is 0 iff $p \in \mathcal{I}$. Equivalently, $\bar{p}^B$ can be looked at as the remainder of the division of p by B . Hence, $\bar{p}^B$ is uniquely determined, independent of how the rules are applied, and is called *the canonical form of p modulo $\mathcal{I}$* (w.r.t. the grlex order). So far we only have established semantics to the effect that GBs solve our main difficulties, but the next theorem has impact.

Theorem 5.1 ([Mor86]). *A finitely generated ideal $\mathcal{I}$ in $\mathbb{C}\langle s \rangle$ has a Gröbner Basis $GB_{\mathcal{I}}$ w.r.t. grlex, which may be infinite.*

Comments on proof. Loosely speaking, the noncommutative Buchberger criterion [Mor86, Theorem 5.1] states that B is a GB iff each S -polynomial built off B can be expressed in terms of the elements of B with control on the grlex order of multipliers. Thus algorithms for building GBs work by producing S -polynomials for pairs a_i, a_j of (not necessarily distinct) polynomials which are generators of $\mathcal{I}$. Here, S -polynomials are ones of the form

$$S_{i,j}(w_i, w'_i, w_j, w'_j) = \frac{1}{\text{LC}(a_i)} w_i a_i w'_i - \frac{1}{\text{LC}(a_j)} w_j a_j w'_j \quad (5.1)$$

for words w_i, w'_i, w_j, w'_j in x satisfying

$$w_i \text{LT}(a_i) w'_i = w_j \text{LT}(a_j) w'_j. \quad (5.2)$$

Here $\text{LC}(a)$ denotes the coefficient of the leading term of a .

At each step in construction of a GB, one considers a collection of polynomials $\hat{B}$. If the “remainder” of an S -polynomial after division by $\hat{B}$ is nonzero, one adds it to $\hat{B}$ and repeats the process. Ultimately (maybe in an infinite number of steps) $\hat{B}$ grows to B , a GB. $\square$

Remark 5.2. (1) In this exposition we used the grlex monomial order. However, many (but not all) monomial orders would work in the same way with the same properties. A linear order on monomials is a *monomial order* if Item (a) holds, and is called *admissible* if Item (b) holds as well:

- (a) $m_1 < m_2$ implies $mm_1m' < mm_2m'$ for all monomials m, m', m_1, m_2 ;
- (b) the given ordering is a well-ordering on the set of monomials, i.e., every descending chain of monomials becomes eventually stationary.

We refer the reader to [Mor86, Gre00] for details.

- (2) An equivalent characterization of B being a GB is: the set of leading terms $\text{LT}(B)$ of elements of B generates the same monomial ideal $\text{LT}(\mathcal{I})$ as all leading terms of $\mathcal{I}$.
- (3) While we do not go into this, given an admissible monomial order, to every ideal and we can associate a “reduced” GB [Xiu12], and these are unique. Loosely speaking, a reduced GB is one in which no polynomial is redundant and none of the monomials appearing in a polynomial of the GB are redundant.

The standard GB packages output a reduced GBs as the default.

- (4) Any maximal set of linearly independent remainders among $\{\bar{p}^B \mid p \in \mathbb{C}\langle s \rangle\}$ for a GB B forms a basis for the quotient algebra $\mathbb{C}\langle s \rangle / \mathcal{I}$.

- (5) GB algorithms are very time and memory consuming both in the commutative and the non-commutative case. For the complexity analysis of the state-of-the-art F_5 algorithm for commutative GBs see [BFS15]. □

5.2 Gröbner Bases for the Paulis

The purpose of this subsection is to demonstrate Gröbner bases in action in a toy example, namely for the Paulis, where it leads to the expected result. However, the choice of the monomial order matters as we illustrate in Example 5.5.

Consider the free algebra $\mathbb{C}\langle x_1, \dots, x_n, y_1, \dots, y_n, z_1, \dots, z_n \rangle$ and its quotient by the ideal $\mathcal{I}^{\text{Pauli}}$ generated by

$$\begin{aligned} x_j^2 = y_j^2 = z_j^2 = 1, \quad x_j y_j = i z_j, \quad y_j x_j = -i z_j, \\ [u_j, w_k] = 0 \quad \text{for } j \neq k \text{ and } u, w \in \{x, y, z\} \end{aligned} \quad (5.3)$$

for $j, k = 1, \dots, n$. The quotient algebra $\mathbb{C}\langle x, y, z \rangle / \mathcal{I}^{\text{Pauli}}$ is isomorphic to the algebra generated by the Paulis $\{\sigma_W^j \mid 1 \leq j \leq n, W \in \{X, Y, Z, I\}\}$, that is, to $M_{2^n}(\mathbb{C})$.

Example 5.3. With the aid of a computer algebra system we computed the GB in the case $n = 4$. With respect to the graded lex order with $x_1 < y_1 < z_1 < x_2 < \dots < z_4$ it is given by the following 90 polynomials, all of which come from Eq. (5.3):

$$\begin{aligned} & -1 + x_1^2, -1 + x_2^2, -1 + x_3^2, -1 + x_4^2, -1 + y_1^2, -1 + y_2^2, -1 + y_3^2, -1 + y_4^2, \\ & -1 + z_1^2, -1 + z_2^2, -1 + z_3^2, -1 + z_4^2, -iz_1 + x_1 y_1, -iz_2 + x_2 y_2, \\ & -iz_3 + x_3 y_3, -iz_4 + x_4 y_4, iz_1 + y_1 x_1, iz_2 + y_2 x_2, iz_3 + y_3 x_3, iz_4 + y_4 x_4, \\ & x_1 x_2 - x_2 x_1, x_1 y_2 - y_2 x_1, x_1 z_2 - z_2 x_1, -x_2 y_1 + y_1 x_2, y_1 y_2 - y_2 y_1, y_1 z_2 - z_2 y_1, \\ & -x_2 z_1 + z_1 x_2, -y_2 z_1 + z_1 y_2, z_1 z_2 - z_2 z_1, x_1 x_3 - x_3 x_1, x_1 y_3 - y_3 x_1, x_1 z_3 - z_3 x_1, \\ & -x_3 y_1 + y_1 x_3, y_1 y_3 - y_3 y_1, y_1 z_3 - z_3 y_1, -x_3 z_1 + z_1 x_3, -y_3 z_1 + z_1 y_3, z_1 z_3 - z_3 z_1, \\ & x_1 x_4 - x_4 x_1, x_1 y_4 - y_4 x_1, x_1 z_4 - z_4 x_1, -x_4 y_1 + y_1 x_4, y_1 y_4 - y_4 y_1, y_1 z_4 - z_4 y_1, \\ & -x_4 z_1 + z_1 x_4, -y_4 z_1 + z_1 y_4, z_1 z_4 - z_4 z_1, -x_1 x_2 + x_2 x_1, x_2 y_1 - y_1 x_2, x_2 z_1 - z_1 x_2, \\ & -x_1 y_2 + y_2 x_1, -y_1 y_2 + y_2 y_1, y_2 z_1 - z_1 y_2, -x_1 z_2 + z_2 x_1, -y_1 z_2 + z_2 y_1, \\ & -z_1 z_2 + z_2 z_1, x_2 x_3 - x_3 x_2, x_2 y_3 - y_3 x_2, x_2 z_3 - z_3 x_2, -x_3 y_2 + y_2 x_3, y_2 y_3 - y_3 y_2, \\ & y_2 z_3 - z_3 y_2, -x_3 z_2 + z_2 x_3, -y_3 z_2 + z_2 y_3, z_2 z_3 - z_3 z_2, x_2 x_4 - x_4 x_2, x_2 y_4 - y_4 x_2, \\ & x_2 z_4 - z_4 x_2, -x_4 y_2 + y_2 x_4, y_2 y_4 - y_4 y_2, y_2 z_4 - z_4 y_2, -x_4 z_2 + z_2 x_4, -y_4 z_2 + z_2 y_4, \\ & z_2 z_4 - z_4 z_2, -x_1 x_3 + x_3 x_1, x_3 y_1 - y_1 x_3, x_3 z_1 - z_1 x_3, -x_1 y_3 + y_3 x_1, -y_1 y_3 + y_3 y_1, \\ & y_3 z_1 - z_1 y_3, -x_1 z_3 + z_3 x_1, -y_1 z_3 + z_3 y_1, -z_1 z_3 + z_3 z_1, -x_2 x_3 + x_3 x_2, x_3 y_2 - y_2 x_3, \\ & x_3 z_2 - z_2 x_3, -x_2 y_3 + y_3 x_2, -y_2 y_3 + y_3 y_2, y_3 z_2 - z_2 y_3, -x_2 z_3 + z_3 x_2, -y_2 z_3 + z_3 y_2, \\ & -z_2 z_3 + z_3 z_2, x_3 x_4 - x_4 x_3, x_3 y_4 - y_4 x_3, x_3 z_4 - z_4 x_3, -x_4 y_3 + y_3 x_4, y_3 y_4 - y_4 y_3, \\ & y_3 z_4 - z_4 y_3, -x_4 z_3 + z_3 x_4, -y_4 z_3 + z_3 y_4, z_3 z_4 - z_4 z_3, -x_1 x_4 + x_4 x_1, x_4 y_1 - y_1 x_4, \\ & x_4 z_1 - z_1 x_4, -x_1 y_4 + y_4 x_1, -y_1 y_4 + y_4 y_1, y_4 z_1 - z_1 y_4, -x_1 z_4 + z_4 x_1, -y_1 z_4 + z_4 y_1, \\ & -z_1 z_4 + z_4 z_1, -x_2 x_4 + x_4 x_2, x_4 y_2 - y_2 x_4, x_4 z_2 - z_2 x_4, -x_2 y_4 + y_4 x_2, -y_2 y_4 + y_4 y_2, \\ & y_4 z_2 - z_2 y_4, -x_2 z_4 + z_4 x_2, -y_2 z_4 + z_4 y_2, -z_2 z_4 + z_4 z_2, -x_3 x_4 + x_4 x_3, x_4 y_3 - y_3 x_4, \\ & x_4 z_3 - z_3 x_4, -x_3 y_4 + y_4 x_3, -y_3 y_4 + y_4 y_3, y_4 z_3 - z_3 y_4, -x_3 z_4 + z_4 x_3, -y_3 z_4 + z_4 y_3, \\ & -z_3 z_4 + z_4 z_3 \end{aligned}$$

Thus the “natural generators” for $\mathcal{I}^{\text{Pauli}}$ are themselves a GB.

Proposition 5.4. *With respect to the graded lex order under which*

$$x_1 < y_1 < z_1 < x_2 < \cdots < z_n,$$

polynomials arising from Eq. (5.3) together with

$$x_j z_j + i y_j, z_j x_j - i y_j, y_j z_j - i x_j, z_j y_j + i x_j, \quad 1 \leq j \leq n \quad (5.4)$$

form a Gröbner basis for $\mathcal{I}^{\text{Pauli}}$. It has $\frac{9}{2}n(n+1)$ elements.

Proof. Firstly, note that the polynomials in Eq. (5.4) belong to $\mathcal{I}^{\text{Pauli}}$. For instance,

$$\begin{aligned} x_j z_j + i y_j &= -i y_j (z_j^2 - 1) - y_j (x_j y_j - i z_j) z_j + (x_j y_j - i z_j) y_j z_j \\ &\quad + (y_j x_j + i z_j) y_j y_j - x_j (y_j^2 - 1) z_j. \end{aligned}$$

We now apply the noncommutative Buchberger algorithm (see, e.g., [Mor86]), to check that the constructed set $\mathcal{B}$ of polynomials is a GB. Firstly, self-obstructions. Given one of the polynomials $p \in \mathcal{B}$ it involves at most two indices i, j . Hence all its nontrivial self-obstructions can also involve only these two indices. But modulo the GB these all reduce to 0 (by Example 5.3). Likewise, given polynomials $p_1, p_2 \in \mathcal{B}$, they involve at most four distinct indices. These same indices are the only ones that can appear in a nontrivial S-polynomial constructed from p_1, p_2 . But, again by Example 5.3, these must reduce to 0 since p_1, p_2 appear in the GB associated to the chosen four indices.

To count the number of elements in this GB, note there are $3n$ polynomials of the form $w_j^2 = 1$ for $w \in \{x, y, z\}$ and $1 \leq j \leq n$. There are $6n$ polynomials of the form $w_j u_j \pm i v_j$ with $\{w, u, v\} = \{x, y, z\}$. Finally, there are $3^2 \cdot \binom{n}{2}$ commutators in $\mathcal{B}$. Adding these numbers we get the desired count. $\square$

Example 5.5. Changing the monomial order to grlex under which

$$x_1 < x_2 < \cdots < x_n < y_1 < \cdots < z_n,$$

the obtained GB is much more complicated. For instance, with $n = 2$ the GB contains the degree three polynomial $y_2 z_1 z_2 - i x_2 z_1$, with $n = 3$ the GB contains the degree four polynomial $y_3 z_1 z_2 z_3 - i x_3 z_1 z_2$, etc.

5.2.1 The d th relaxation of the quantum max-cut can be computed in polynomial time

Proposition 5.4 makes it possible to compute the d th relaxation of the quantum max-cut in polynomial time.

Algorithm 2: d th relaxation of the quantum max-cut supported by Paulis

Input : Graph $G = (V, E)$ on n vertices, level of relaxation d

Output : Solution to the d th relaxation of quantum max-cut for h_G

Find the Veronese vector $V_d(n)$ containing all monomials of degree $\leq d$ in the swap variables, and express them in terms of symbolic Paulis x_j, y_j, z_j ;
Replace each entry of $V_d(n)$ by its remainder upon division with the GB of $\mathcal{I}^{\text{Pauli}}$;
Compute a basis for the span of all the entries of $V_d(n)$;
Form the vector $\mathcal{V}_d(n)$ with the basis as its entries;
Find $\mathcal{M}_d(n) := \mathcal{V}_d(n)\mathcal{V}_d(n)^*$;
Replace each entry of $\mathcal{M}_d(n)$ by its remainder upon division with the GB of $\mathcal{I}^{\text{Pauli}}$;
Replace each distinct term appearing in $\mathcal{M}_d(n)$ with a new (scalar) variable; call the resulting matrix $\mathcal{M}_d(L)$;
Solve the SDP

$$\kappa_d(h_G) = \sup\{\langle \mathcal{M}_d(L), \Gamma_G \rangle \mid \mathcal{M}_d(L) \succeq 0, \mathcal{M}_d(L)_{1,1} = 1\}, \quad (5.5)$$

where Γ_G is any matrix satisfying $h_G = \mathcal{V}_d(n)^* \Gamma_G \mathcal{V}_d(n)$;

return $\kappa_d(h_G)$

Theorem 5.6. *Algorithm 2 computes the value of the d th relaxation of the quantum max-cut in polynomial time.*

Proof. Each step takes polynomial time. The only argument required is to establish that the constructed SDP of Eq. (5.5) can be solved in polynomial time using standard interior point solvers. But this follows from SDP complexity theory (see, e.g., [dK02, Section 1.9]); we only need the existence of Slater points (which we established in the proof of Proposition 4.9) together with bounds on the feasible region of the SDP. Since our variables s_{ij} are involutions, the diagonal of $\mathcal{M}_d(L)$ in Eq. (5.5) is constantly 1 yielding a bound of 2 on each positive semidefinite $\mathcal{M}_d(L)$. $\square$

Remark 5.7. As pointed to us by Claudio Procesi, an alternative method for providing a polynomial time SDP hierarchy for the quantum max-cut can be based on the notion of good permutations [Pro21]. For $d \in \mathbb{N}$, a permutation $\sigma \in S_n$ is d -bad if there are $1 \leq i_1 < i_2 < \dots < i_d \leq n$ with $\sigma(i_1) > \sigma(i_2) > \dots > \sigma(i_d)$. Otherwise it is d -good. Since each permutation is a product of transpositions, this notion can also be applied to certain elements of the swap algebra, e.g., to products of the s_{ij} . Procesi [Pro21, Theorem 8] shows that 3-good permutations form a basis of the swap algebra. Further, the proof of his theorem gives a recursive method for expressing an element of the swap algebra as a linear combination w.r.t. this basis.

Hence providing an ordering for the set of 3-good permutations one can build increasing size Veronese vectors to be used in the ncSoS hierarchy. (Note that the transposition $(i \ j)$ with $j - i > 1$ is not 3-good, but an induction on $j - i$ together with Item (3) of Definition 3.1 can be used to express s_{ij} in terms of 3-good permutations.) It would be interesting to investigate this further and compare the performance of this method with the one presented here. $\square$

5.3 Gröbner Bases for the Swap Algebra

Ideally one would want to simplify Algorithm 2 to avoid expanding everything in terms of Paulis. This is indeed possible if one computes the Gröbner basis for the Swap algebra.

Unfortunately, the GB for the Swap algebra appears more involved than the one for the Paulis, and we were unable to identify its form in general. However, for small n it can be explicitly computed.

Consider the lex order on pairs (i, j) , i.e., $(i, j) < (k, \ell)$ if $i < k$ or $i = k$ and $j < \ell$, and order swaps s_{ij} w.r.t. lex order on the indices. We used Magma [BCP97] to bootstrap the computation of the GBs. Namely, we computed GB for some small n , say $n = 4$. Then replace indices $(1, 2, 3, 4)$ with any increasing 4-subset of $(1, 2, 3, 4, 5)$ to obtain a generating set for $\mathcal{I}_5^{\text{swap}}$. On this generating set we next run the GB algorithm to obtain a GB. Then rinse and repeat. The GB for $n = 4$ is given in Appendix C.

Proposition 5.8. *The GB for $\mathcal{I}_3^{\text{swap}}$ is given by the following eight polynomials:*

$$\begin{aligned} & -1 + s_{12}^2, \quad 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{12}s_{23}, \\ & 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{13}s_{12}, \quad -1 + s_{13}^2, \\ & \quad -s_{12}s_{13} + s_{13}s_{23}, \quad -s_{12}s_{13} + s_{23}s_{12}, \\ & 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{23}s_{13}, \quad -1 + s_{23}^2 \end{aligned} \tag{5.6}$$

Proof. Verifying this is a straightforward, though tedious calculation. Alternately, this can be obtained with the help of a computer algebra system such as Magma or NCAAlgebra under Mathematica. $\square$

5.4 d th Swap Relaxation

Finally, we have all the ingredients needed to give the algorithm for computing the d th relaxation based purely in terms of the swaps. It generalizes Algorithm 1 from $d = 2$ to arbitrary d .

Algorithm 3: d th relaxation of the quantum max-cut

Input : Graph $G = (V, E)$ on n vertices, level of relaxation d ,

GB $_n$ =Gröbner basis for $\mathcal{I}_n^{\text{swap}}$

Output: Solution to the d th relaxation of quantum max-cut for h_G

Find $V_d(n)$;

Replace each entry of $V_d(n)$ by its remainder upon division with GB $_n$;

Compute a basis for the span of all the entries of $V_d(n)$;

Form the vector $\mathcal{V}_d(n)$ with the basis as its entries;

Find $\mathcal{M}_d(n) := \mathcal{V}_d(n)\mathcal{V}_d(n)^*$;

Replace each entry of $\mathcal{M}_d(n)$ by its remainder upon division with GB $_n$;

Replace each distinct term appearing in $\mathcal{M}_d(n)$ with a new (scalar) variable; call the resulting matrix $\mathcal{M}_d(L)$;

Solve the SDP

$$\alpha_d(h_G) = \sup\{\langle \mathcal{M}_d(L), \Gamma_G \rangle \mid \mathcal{M}_d(L) \succeq 0, \mathcal{M}_d(L)_{1,1} = 1\}, \tag{5.7}$$

where Γ_G is any matrix satisfying $h_G = \mathcal{V}_d(n)^* \Gamma_G \mathcal{V}_d(n)$;

return $\alpha_d(h_G)$

This algorithm, like Algorithm 2 runs in finite time with similar justifications. However, this algorithm requires as input a Gröbner basis for $\mathcal{I}_n^{\text{swap}}$. Finding such a Gröbner basis for finite d and arbitrary n may not be possible in polynomial time.

5.5 Irrep Swap Relaxations

As seen in Corollary 2.9, $\text{eig}_{\max}(H_G) = \max_{k=0}^{\lfloor \frac{n}{2} \rfloor} \left(\text{eig}_{\max}(H_G^{[n-k,k]}) \right)$. In this section we explain how to form SDP relaxations to find $\text{eig}_{\max}(H_G^{[n-k,k]})$ for a given graph G on n vertices and $k \leq n/2$.

From the proof of Lemma 2.12 (in particular Eq. (2.46)) we have that the following constraint is satisfied inside the $[n-k, k]$ irrep

$$\sum_{(i,j) \in E(K_n)} \rho_{[n-k,k]}((i \ j)) = \widehat{\eta}_{[n-k,k]} I = \left(\binom{n}{2} + k^2 - k(n+1) \right) I. \quad (5.8)$$

Enforcing this constraint is equivalent to enforcing that

$$h_{K_n} = \eta_{n-k,k} \iff h_{K_n} - \eta_{n-k,k} = 0. \quad (5.9)$$

and so, by Proposition 3.10 the algebra formed by the s_{ij} variables satisfying this extra relation is isomorphic to the relevant Irrep Symbolic Swap Algebra. Stated differently, Proposition 3.10 shows that all constraints in the $[n-k, k]$ irrep can be derived from Equation (5.8) above along with the relations given in Definition 3.1.

We can force the swap variables to satisfy the same constraint by requiring that one of them, namely $s_{n-1 \ n}$, (the last one in our chosen monomial order) can be written in terms of the others:

$$s_{n-1 \ n} = - \sum_{i < j \leq n, i \neq n-1} s_{ij} + \widehat{\eta}_{[n-k,k]}. \quad (5.10)$$

We can now adapt Algorithm 3 to find a relaxation for $\text{eig}_{\max}(H_G^{[n-k,k]})$.

5.5.1 Using the Gröbner basis for swaps

Algorithm 4: d th relaxation for $\text{eig}_{\max}(H_G^{[n-k,k]})$

Input : Graph $G = (V, E)$ on n vertices, $k \leq n/2$, level of relaxation d
 $\text{GB}_n = \text{Gröbner basis for } \mathcal{I}_n^{\text{swap}}$

Output : Solution to the d th relaxation for $\text{eig}_{\max}(H_G^{[n-k,k]})$

Find $V_d(n)$;
while $V_d(n)$ has not stabilized **do**
 Replace each entry of $V_d(n)$ by its remainder upon division with GB_n ;
 In each of the entries replace $s_{n-1}n$ with the expression derived from Eq. (5.10)
end
Compute a basis for the span of all the entries of $V_d(n)$;
Form the vector $\mathcal{V}_d(n)$ with the basis as its entries;
Find $\mathcal{M}_d(n) := \mathcal{V}_d(n)\mathcal{V}_d(n)^*$;
while $\mathcal{M}_d(n)$ has not stabilized **do**
 Replace each entry of $\mathcal{M}_d(n)$ by its remainder upon division with GB_n ;
 In each of the entries replace $s_{n-1}n$ with the expression derived from Eq. (5.10)
end
Replace each distinct term appearing in $\mathcal{M}_d(n)$ with a new (scalar) variable; call
the resulting matrix $\mathcal{M}_d(L)$;
Solve the SDP

$$\lambda_d(H_G^{[n-k,k]}) = \sup\{\langle \mathcal{M}_d(L), \Gamma_G \rangle \mid \mathcal{M}_d(L) \succeq 0, \mathcal{M}_d(L)_{1,1} = 1\}, \quad (5.11)$$

where Γ_G is any matrix satisfying $h'_G = \mathcal{V}_d(n)^* \Gamma_G \mathcal{V}_d(n)$; here h'_G is h_G , where $s_{n-1}n$ was replaced as before;

return $\lambda_d(H_G^{[n-k,k]})$

We now discuss the runtime of Algorithm 4. Replacing $s_{n-1}n$ with the expression derived from Eq. (5.8) produces monomials in the s_{ij} that are all smaller than the original one in the monomial order. We also note that there are $\binom{n}{2} = n(n-1)/2$ distinct monomials of degree one in the swap algebra, and hence at most $(n(n-1)/2)^d$ monomials of degree at most d . Thus, the two while loops in Algorithm 4 terminate after at most $(n(n-1)/2)^d$ many steps. For finite d Algorithm 4 then runs in polynomial time, provided a Gröbner basis is supplied as input to the algorithm.

5.5.2 Using the Gröbner basis for irreps

To avoid having to deal with the two loops in Algorithm 4, we can find a GB for the ideal $\mathcal{I}_{n-k,k}$. Recall from Proposition 3.10 that $\mathcal{I}_{n-k,k}$ is generated by $\mathcal{I}^{\text{swap}}$ together with $h_{K_n} - \eta_{n-k,k}$. So we can either find the Gröbner basis GB_n for $\mathcal{I}^{\text{swap}}$, then add the ‘clique polynomial’, $h_{K_n} - \eta_{n-k,k}$, and run the GB algorithm again. Or, one takes the generating set of polynomials in Remark 3.2 for $\mathcal{I}^{\text{swap}}$, adds the clique polynomial, and runs a GB. Once the Gröbner basis $\text{GB}_{n-k,k}$ for $\mathcal{I}_{n-k,k}$ is available, one simply adapts Algorithm 3 by replacing GB_n by $\text{GB}_{n-k,k}$. Thus we obtain Algorithm 5.

Algorithm 5: d th relaxation for $\text{eig}_{\max}(H_G^{[n-k,k]})$

Input : Graph $G = (V, E)$ on n vertices, $k \leq n/2$, level of relaxation d
 $\text{GB}_{n-k,k}$ = Gröbner basis for $\mathcal{I}_{n-k,k}$

Output : Solution to the d th relaxation for $\text{eig}_{\max}(H_G^{[n-k,k]})$

Find $V_d(n)$;

Replace each entry of $V_d(n)$ by its remainder upon division with $\text{GB}_{n-k,k}$;

Compute a basis for the span of all the entries of $V_d(n)$;

Form the vector $\mathcal{V}_d(n)$ with the basis as its entries;

Find $\mathcal{M}_d(n) := \mathcal{V}_d(n)\mathcal{V}_d(n)^*$;

Replace each entry of $\mathcal{M}_d(n)$ by its remainder upon division with $\text{GB}_{n-k,k}$;

Replace each distinct term appearing in $\mathcal{M}_d(n)$ with a new (scalar) variable; call the resulting matrix $\mathcal{M}_d(L)$;

Solve the SDP

$$\alpha_d(h_G) = \sup\{\langle \mathcal{M}_d(L), \Gamma_G \rangle \mid \mathcal{M}_d(L) \succeq 0, \mathcal{M}_d(L)_{1,1} = 1\}, \quad (5.12)$$

where Γ_G is any matrix satisfying $h_G \equiv \mathcal{V}_d(n)^* \Gamma_G \mathcal{V}_d(n) \pmod{\text{GB}_{n-k,k}}$

return $\alpha_d(H_G^{[n-k,k]})$

As in the case of Algorithms 3 and 4 this algorithm has polynomial runtime provided a Gröbner basis is provided as input for the Algorithm. The Gröbner bases $\text{GB}_{4-k,k}$ for $k \in \{1, 2\}$ are given in Appendix C.

6 Finding Eigenvalues via Clique Decompositions of Graphs

The QMC problem is dramatically easier to solve on smaller graphs. So, it would be ideal if we could cheaply compute the solution to QMC on a large graph G from the solutions to QMC on a number of smaller graphs. There is an obvious way this can be implemented if the graph G is not connected. In this case, we can solve QMC on each of the connected components of G and add up the solutions to obtain the solution for G .

Less obvious is that a similar decomposition is possible when the complement of a graph G is disconnected. Suppose, for instance, that an n vertex graph G has a complement G^c with two connected components G_1^c and G_2^c . In this case, the QMC Hamiltonian for G can be written as

$$H_G = H_{K_n} - H_{G_1^c} - H_{G_2^c},$$

where K_n denotes an n -vertex clique. Lemma 2.11 gives that $H_{K(G)}$ on the $[n-k, k]$ irrep is equal to $\eta_{n-k,k}I$, and since G_1^c and G_2^c are disjoint we obtain

$$\text{eigs}(H_G^{[n-k,k]}) = \{\eta_{[n-k,k]}\} - \text{eigs}(H_{G_1^c}^{[n-k,k]}) - \text{eigs}(H_{G_2^c}^{[n-k,k]}),$$

where the addition and subtraction is the Minkowski sum of sets. Combining this observation with the Young branching rule for irreps, we can solve the QMC problem on G by first solving the problem on the smaller graphs G_1^c and G_2^c .

In this section we fill in details above and develop this type of graph decomposition “to its fullest.” As a result, we obtain algorithms for approximating eigenvalues and even computing them in exact arithmetic in some special cases. We begin by doing a simple example in more detail.

6.1 The Star Graph

As a first example, we consider the n -vertex star graph, which we denote $\star_n$. Eigenvalues of the corresponding QMC Hamiltonian $H_{\star_n}$ have been computed previously [LM62]. Here we re-derive these results using the methods of this section.

Lemma 6.1. *For any n and $k \leq n/2$ the matrix $H_{\star_n}^{[n-k,k]}$ has two eigenvalues, whose values are given by*

$$e_1 = 2(n - k + 1), \quad e_2 = 2k. \quad (6.1)$$

If n is even and $k = n/2$, then $H_{\star_n}^{[n/2,n/2]}$ is $(n + 2)$ times the identity matrix.

Proof. Consider first the case $k < n/2$. Label the vertices of the star graph so that the n -th vertex corresponds to the center of the star. Then $\star_n^c = K_{n-1}$ which yields $\star_n = K_n - K_{n-1}$, so

$$h_{\star_n} = h_{K_n} - h_{K_{n-1}}. \quad (6.2)$$

Apply the $[n - k, k]$ irrep to get

$$H_{\star_n}^{[n-k,k]} = H_{K_n}^{[n-k,k]} - H_{K_{n-1}}^{[n-k,k]}. \quad (6.3)$$

Now we consider the matrices $H_{K_n}^{[n-k,k]}$ and $H_{K_{n-1}}^{[n-k,k]}$.

Lemmas 2.11 and 2.12 immediately give

$$H_{K_n}^{[n-k,k]} = \eta_{[n-k,k]} I = (2k(n + 1) - 2k^2) I. \quad (6.4)$$

However, these results do not immediately say anything about the matrix $H_{K_{n-1}}^{[n-k,k]}$. To deal with this element we invoke Young's branching rule, which states that the restriction of an irrep λ of S_n to S_{n-1} corresponds to a direct sum over all irreps which can be obtained by removing a single cell from the partition corresponding to λ . In the case considered here, that means that $H_{K_{n-1}}^{[n-k,k]}$ can be decomposed into a direct sum of the matrices $H_{K_{n-1}}^{[n-k-1,k]}$ and $H_{K_{n-1}}^{[n-k,k-1]}$. The matrices $H_{K_n}^{[n-k,k]}$ and $H_{K_{n-1}}^{[n-k,k]}$ also commute (since $H_{K_n}^{[n-k,k]}$ is just a multiple of the identity), hence are simultaneously diagonalizable. Then the possible eigenvalues of $H_{\star_n}^{[n-k,k]}$ are given by a difference of the single eigenvalue of $H_{K_n}^{[n-k,k]}$ and the two possible eigenvalues of $H_{K_{n-1}}^{[n-k,k]}$. This gives

$$\begin{aligned} e_1 &= \eta_{[n-k,k]} - \eta_{[n-k,k-1]} \\ &= (2k(n + 1) - 2k^2) - (2(k - 1)n - 2(k - 1)^2) \\ &= 2k + 2n - 2k^2 + 2(k - 1)^2 \\ &= 2(n - k + 1) \end{aligned}$$

and

$$\begin{aligned} e_2 &= \eta_{[n-k,k]} - \eta_{[n-k-1,k]} \\ &= (2k(n + 1) - 2k^2) - (2kn - 2k^2) \\ &= 2k. \end{aligned}$$

Now assume n is even, and let $k = n/2$. Applying Eq. (6.3) we have

$$H_{\star_n}^{[n/2, n/2]} = H_{K_n}^{[n/2, n/2]} - H_{K_{n-1}}^{[n/2, n/2]}, \quad (6.5)$$

and by Eq. (6.4),

$$H_{K_n}^{[n/2, n/2]} = \eta_{[n/2, n/2]} I = \frac{1}{2} n(n+2) I. \quad (6.6)$$

The difference to the first case occurs when applying the branching rule to compute $H_{K_{n-1}}^{[n/2, n/2]}$. Since there is only one valid way to remove a box from a rectangular $2 \times n/2$ Young tableaux, we obtain

$$H_{K_{n-1}}^{[n/2, n/2]} = H_{K_{n-1}}^{[n/2, n/2-1]} = \eta_{[n/2, n/2-1]} I = \frac{1}{2} (n^2 - 4) I. \quad (6.7)$$

Combining Eq. (6.7) with Eq. (6.6) in Eq. (6.5) yields

$$H_{\star_n}^{[n/2, n/2]} = (n+2) I. \quad \square$$

6.2 Graph Clique Decomposition and Simpler Hamiltonians

In the previous example we showed how to compute the eigenvalues of the star graph Hamiltonian by writing it as a difference of cliques and then using Young's Branching Rule. In that example, the cliques considered differed in size by one vertex. But the same techniques also apply generally to any graph whose corresponding Hamiltonian can be decomposed as an arbitrary signed sum of clique Hamiltonians. In this section, we provide an algorithm which computes such a sum for a given graph G whenever such a decomposition is possible. Then we show how to speed up the computation of some QMC Hamiltonian's eigenvalues using this decomposition. While this algorithm is straightforward, we do not know of an occurrence of it elsewhere in the literature, and so for completeness prove correctness of the algorithm and runtime guarantees in this paper. We also note that a similar, but distinct, decomposition of a graphs is considered in [BPR22, PKT22]. These papers investigate ways in which graphs can be decomposed into sums of cliques over $\mathbb{F}_2$ (i.e. decompositions where existence of an edge in the original graph corresponds to the parity of the number of cliques containing that edge). These decomposition can be applied more generally than the decomposition considered in this paper.

6.2.1 The Tree Clique Decomposition

To keep track of how the QMC Hamiltonian associated with a graph G decomposes into a sum of signed cliques we introduce an object which we call the *Tree Clique Decomposition* of G . This is introduced formally in the next definition. In that definition and throughout the remainder of the paper we assume that all trees discussed are rooted (or equivalently directed) trees.

Definition 6.2 (Tree Clique Decomposition). *For any connected graph G , the tree clique decomposition of G , denoted $\mathcal{T}(G)$ consists of an m -vertex tree T and set of n connected graphs $\{G(v_1), \dots, G(v_m)\}$, where each graph $G(v_i)$ is associated with a vertex v_i of T and the following properties hold:*

- (1) *For the root vertex v_1 of T , we have $G(v_1) = G$.*

- (2) For any vertex v_i of T which is not a leaf vertex, let $c_1, \dots, c_k$ be its children. Then we have

$$G(v_i)^c = \bigcup_{j \in \{1, 2, \dots, k\}} G(c_j), \quad (6.8)$$

where we understand the union of graphs to be their disjoint union.

- (3) For any leaf vertex v_j of T we have that $G(v_j)^c$ is connected or $G(v_j)^c$ is totally disconnected.

Given a graph G it is reasonably straightforward to compute the tree clique decomposition of G by iteratively taking complements of graphs and then breaking them into their connected components. We make this process formal in Algorithm 6.

Algorithm 6: Tree Clique Decomposition

Input : A graph G to be decomposed.

Output : A tree clique decomposition $\mathcal{T}(G) = \{T, \{G(v_1), \dots, G(v_m)\}\}$

Initialize T as the singleton graph with vertex v_1 ;

activeTLeaves := $\{v_1\}$;

$i := 0$;

$G(v_1) := G$;

while **activeTLeaves** is non-empty **do**

$i := i + 1$;

 remove a vertex from **activeTLeaves**, call it v_i ;

if $G(v_i)^c$ is disconnected and $G(v_i)^c$ is not totally disconnected **then**

foreach connected component H of $G(v_i)^c$ **do**

 Add a vertex c as a child to vertex v_i in the tree T ;

$G(c) := H$;

 Add c to **activeTLeaves**;

end

end

end

return $\{T, \{G(v_1), G(v_2), \dots, G(v_m)\}\}$

Now we prove some simple properties of the tree clique decomposition, then prove correctness and bound the runtime of Algorithm 6.

Theorem 6.3. *The tree clique decomposition $\mathcal{T}(G)$ of an n -vertex graph G is unique (up to permutations amongst siblings in T). Additionally:*

- (1) *The associated tree T has depth at most n .*
- (2) *For any $i \leq n$ the total number of vertices contained in graphs indexed by vertices at depth i in T is at most n , that is*

$$\sum_{w: \text{depth}(w)=i} |V(G(w))| \leq n. \quad (6.9)$$

Proof. We first show uniqueness of the tree clique decomposition. Assume not. Then there is a graph G with two tree clique decompositions $\mathcal{T}(G) = \{T, \{G(v_1), G(v_2), \dots, G(v_m)\}\}$ and $\mathcal{T}(G)' = \{T', \{G(w_1)', G(w_2)', \dots, G(w_{m'})'\}\}$. Because both are tree clique decompositions we must have

$$G(v_1) = G(w_1)' = G \quad (6.10)$$

where w_1 and v_1 are the root vertices of T and T' respectively. Now, let v^* and w^* be minimal vertices (in terms of depth) at which the child structure of the clique decompositions $\mathcal{T}(G)$ and $\mathcal{T}(G)'$ differ, meaning that either v^* and w^* have different numbers of children or that sets of graphs associated with these children are not identical. Either way, letting $C(v^*)$ be the children of v^* and $C'(w^*)$ be the children of w^* , we must have

$$\bigcup_{c \in C(v^*)} G(c) \neq \bigcup_{c' \in C'(w^*)} G(c'). \quad (6.11)$$

But because this was a minimal occurrence with respect to depth, we must also have

$$G(v^*) = G(w^*). \quad (6.12)$$

But then finally, because $\mathcal{T}(G)$ and $\mathcal{T}(G)'$ are both tree clique decompositions we have

$$G(v^*)^c = \bigcup_{c \in C(v^*)} G(c) \neq \bigcup_{c' \in C'(w^*)} G(c') = G(w^*)^c \quad (6.13)$$

and this contradiction proves the result.

Next, we prove Item (1), that is we show the tree T appearing in the tree clique decomposition $\mathcal{T}(G) = \{T, \{G(v_1), \dots, G(v_m)\}\}$ of an n -vertex graph G has depth at most n . By definition of the tree clique decomposition, for any non-leaf vertex $w \in T$ with child c we must have that $G(w)^c$ is disconnected with $G(c)$ corresponding to a connected component of $G(w)^c$. But then $G(c)$ is necessarily a graph on fewer vertices than $G(w)$. It follows that the number of vertices in a graph $G(v)$ is a strictly decreasing function of the depth of the vertex v in the tree T . Then, since the root vertex v_1 of T corresponds to the n vertex graph $(v_1) = G$, the bound on the depth of T follows.

To prove Item (2) we first observe that, for any vertex $w \in T$ with children $c_1, \dots, c_k$ we have

$$\sum_i |V(G(c_i))| = |V(G(w))|, \quad (6.14)$$

since $\bigcup_i G(c_i) = G(w)^c$. Then we also have that the total number of vertices involved in graphs indexed at depth i of the tree T is a non-increasing function of i and, in particular, letting v_1 be the root vertex of T we have

$$\sum_{w: \text{depth}(w)=i} |V(G(w))| \leq |V(G(v_1))| = n \quad \forall i, \quad (6.15)$$

as desired. $\square$

Corollary 6.4. *Algorithm 6 computes the tree clique decomposition of the input graph G in time $O(n^3)$.*

Proof. By inspection, the tree T and associated graphs $G(v_1), \dots, G(v_m)$ returned by the algorithm satisfy all the properties of the tree clique decomposition outlined in Definition 6.2 so correctness of Algorithm 6 is immediate. To bound the runtime of the algorithm first note that an n -vertex graph can be decomposed into its connected components in time $O(n^2)$ (using depth first or breadth first search). This is the dominant step in the inner loop of the algorithm, so the algorithm takes time at most $O(|V(G(w))|^2)$ to complete the inner loop that removes vertex w from **activeLeaves**. Next, note that every vertex in T

appears in `activeTleaves` at most once. Then, letting d denote the depth of T , the total runtime of the algorithm is bounded by:

$$O\left(\sum_{w \in V(T)} |V(G(w))|^2\right) = O\left(\sum_{i=1}^d \sum_{w: \text{depth}(w)=i} |V(G(w))|^2\right) \quad (6.16)$$

$$\leq O\left(\sum_{i=1}^d n^2\right) \leq O(n^3), \quad (6.17)$$

where the first inequality follows from Item (2) of Theorem 6.3 and the second inequality follows from Item (1). $\square$

6.2.2 Clique decompositions and Hamiltonians

Finally we show how the tree graph decomposition can be used to write the QMC Hamiltonian associated with a graph G as a signed sum of clique Hamiltonians and smaller “residual” graph Hamiltonians.

Theorem 6.5. *Let G be a graph and $\mathcal{T}(G) = \{T, \{G(v_1), \dots, G(v_n)\}\}$ be the tree-clique decomposition of G . Also, for any graph G , let $K(G)$ denote the complete graph on the vertices of G . Then the following claims hold:*

- (1) *For any vertex $v \in T$ with children $c_1, \dots, c_k$ we have*

$$H_{G(v)} = H_{K(G(v))} - \sum_{j \in \{1, \dots, k\}} H_{G(c_j)} \quad (6.18)$$

- (2) *Let L denote the set of leaf vertices in T , and R be all non-leaf vertices. Also, for any vertex $v \in T$, let $d(v)$ denote the depth of vertex v in the tree, with the root vertex having depth $d(v_1) = 0$. Then we have*

$$H_G = \sum_{r \in R} (-1)^{d(r)} H_{K(G(r))} + \sum_{l \in L} (-1)^{d(l)} H_{G(l)} \quad (6.19)$$

Proof. Part (1) of the theorem follows from Item (2) in Definition 6.2 which gives

$$G(v)^c = \bigcup_{j \in \{1, \dots, k\}} G(c_j) \quad (6.20)$$

and hence

$$H_{G(v)} = H_{K(G(v))} - H_{G(v)^c} = H_{K(G(v))} - \sum_{j \in \{1, \dots, k\}} H_{G(c_j)}. \quad (6.21)$$

Part (2) then follows from repeated application of (1), beginning with the root vertex of T . $\square$

6.2.3 Example of clique decomposition, Algorithm 6

Before giving an explicit example of Algorithm 6 we note any complete k -partite graph has a particularly simple tree clique decomposition which Algorithm 6 finds in one step. This is because a complete k -partite graph is by definition the complement of the disjoint union of k different complete graphs; these are the output of the algorithm.

Now we give an explicit illustration of Algorithm 6 on a graph which is not a complete k -partite graph.

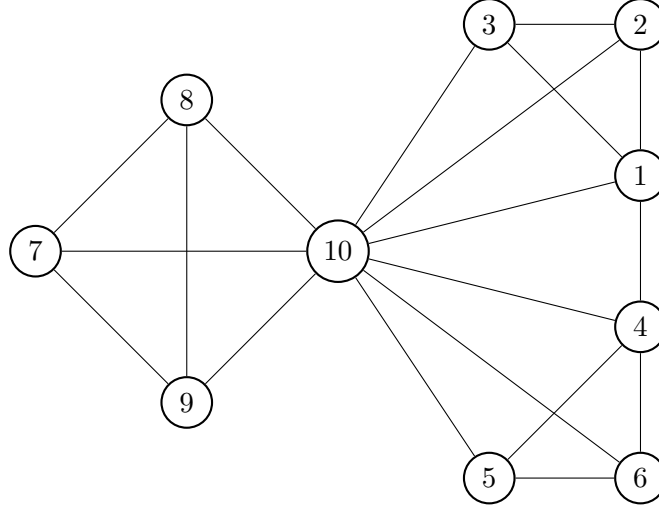


Figure 1: This is the input graph to the algorithm. The output tree T is in Section 6.2.3 and the graphs $G(v_j)$ are in Figure 3.

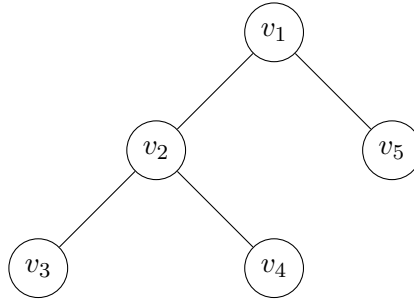


Figure 2: This is the output T from Algorithm 6 run on the graph in Figure 1.

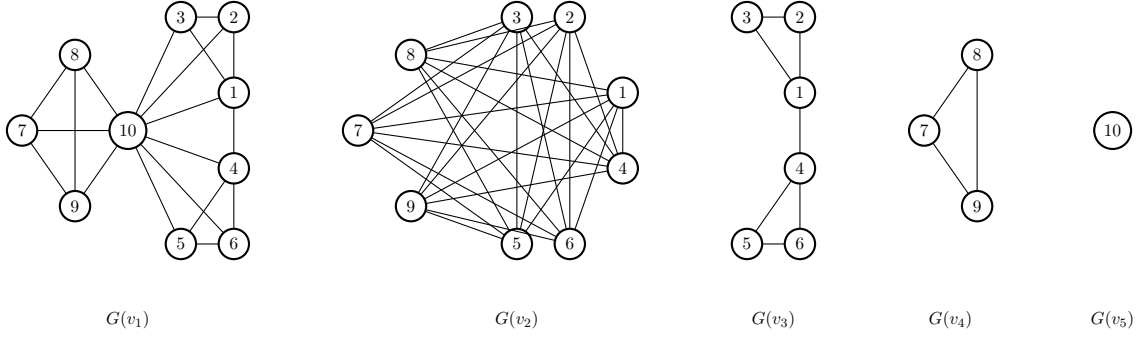


Figure 3: This is the list of graphs which are output. The leaf graph, the graphs on which the algorithm terminates, correspond to v_3 , v_4 , and v_5 . The first of these is a clique, the second does not decompose further and the last is an isolated point.

6.3 Computing Eigenvalues from Clique Decomposition

Now we show how we can use the tree clique decomposition to speed up the computation of the eigenvalues of some QMC graph Hamiltonians.

A key part of this argument will be a use of Young's branching rule to characterize the irreps that can arise when considering restriction of an S_n irrep to S_m for some $m < n$. We introduce notation for this situation and prove its correctness in the next lemma.

Lemma 6.6. *Given any two row partition $[n - k, k]$ with n boxes and positive integer $m < n$ let $\text{Rstrct}(m, [n - k, k])$ be the set of all two row Young diagrams with m boxes total, at most $n - k$ boxes in the first row, and at most k boxes in the second. Stated in notation:*

$$\text{Rstrct}(m, [n - k, k]) = \{\text{Irrep}[m - j, j]\}_{j \in \mathcal{J}} \quad (6.22)$$

with $\mathcal{J} := \{j : m + k - n \leq j \leq \min(k, m/2)\}$.

Then the restriction of an irrep $[n - k, k]$ of S_n to S_m (i.e., the action of that irrep on just the elements of S_m) is isomorphic to a direct sum over all irreps in $\text{Rstrct}(m, [n - k, k])$ where each irrep occurs with some non-zero multiplicity.

Proof. A single application of Young's Branching rule tells us that the restriction of the irrep λ to S_{n-1} is isomorphic to a direct sum over the irreps that can be obtained by removing one box from λ while leaving a valid Young diagram. Applying the branching rule again to each of these irreps gives a set of irreps whose direct sum is isomorphic to the restriction of λ to S_{n-2} . Continuing this process inductively, we see the restriction of λ to S_m is isomorphic to a direct sum over all the irreps which can be obtained by removing $n - m$ boxes from λ while leaving a valid Young diagram (with each irrep occurring with some non-zero multiplicity). Since we can only remove boxes in this process (and not add any) this is equivalent to a direct sum over all valid irreps with m boxes total, at most $n - k$ boxes on the first row, and at most k boxes on the second. But this is exactly the set of irreps in $\text{Rstrct}(m, [n - k, k])$, and we are done. $\square$

Lemma 6.7. *Let G and R be vertex-disjoint graphs on n vertices and assume we know the spectra $\text{eigs}(H_G^{n-k,k})$ and $\text{eigs}(H_R^{n-k,k})$, then the following hold:*

- (1) *The spectrum of $H_{K_n}^{n-k,k} - H_G^{n-k,k}$ is $\{\eta_{n-k,k} - \alpha : \alpha \in \text{eigs}(H_G^{n-k,k})\}$;*
- (2) *The spectrum of $H_{G \cup R}^{n-k,k} = H_G^{n-k,k} + H_R^{n-k,k}$ for G and R disjoint is the Minkowski sum*

$$\{\alpha + \beta : \alpha \in \text{eigs}(H_G^{n-k,k}), \beta \in \text{eigs}(H_R^{n-k,k})\}.$$

Proof. Item (1) is an immediate corollary of Lemma 2.11, but remains useful to state outright.

To show Item (2), we begin with a toy example. Suppose that A^0 is an n by n matrix with $\alpha \in \text{eigs}(A^0)$ and B^0 is an m by m matrix with $\beta \in \text{eigs}(B^0)$. Hence, $A^0 u = \alpha u$ and $B^0 v = \beta v$. Then, if $A = A^0 \otimes I_m$ and $B = I_n \otimes B^0$, we have that

$$A(u \otimes v) = (A^0 u) \otimes (I_m v) = \alpha(u \otimes v) \quad (6.23)$$

$$B(u \otimes v) = (I_n u) \otimes (B^0 v) = \beta(u \otimes v). \quad (6.24)$$

Thus $u \otimes v$ is an eigenvector of $A + B$ corresponding to the eigenvalue $\alpha + \beta$. Thus, $\text{eigs}(A) + \text{eigs}(B) \subseteq \text{eigs}(A + B)$. If the spectra are viewed as multi-sets with multiplicity, there are mn many element in each multi-set. Thus $\text{eigs}(A) + \text{eigs}(B) = \text{eigs}(A + B)$.

Going back to the definition of the swap matrices Definition 2.2, it is clear that if G and R are vertex-disjoint, then up to a canonical shuffle which (effectively) relabels the vertices we can reduce to the case of the toy example Eq. (6.23) to show that the spectrum of $H_G^{n-k,k} + H_R^{n-k,k}$ equals the Minkowski sum $\text{eigs}(H_G^{n-k,k}) + \text{eigs}(H_R^{n-k,k})$. $\square$

Next, we consider a connected graph G and show we can characterize the eigenvalues of the irrep Hamiltonian H_G^λ of this graph in terms of the eigenvalues of irrep Hamiltonians of the connected graphs appearing in the complement of G .

Lemma 6.8. *Let G be a connected graph on n vertices, and let G^c denote its complement. Further, let $G_1^c, \dots, G_L^c$ denote the connected components of the graph G^c and let $\nu_1, \dots, \nu_L$ be the number of vertices in each of these connected components.*

Then, for any two row irrep λ of S_n we have

$$\text{eigs}(H_G^\lambda) = \left\{ \{\eta_\lambda\} - \sum_{j=1}^L \left\{ \bigcup_{\xi \in \text{Rstrct}(\nu_j, \lambda)} \text{eigs}(H_{G_j^c}^\xi) \right\} \right\} \quad (6.25)$$

where we understand addition and subtraction between sets to be Minkowski addition, so

$$\{A\} \pm \{B\} = \{a \pm b\}_{a \in A, b \in B}. \quad (6.26)$$

Proof. First note that we can write

$$H_G^\lambda = H_{K(G)}^\lambda - \sum_{j=1}^L H_{G_j^c}^\lambda \quad (6.27)$$

by definition of the graph complement. Additionally Lemma 2.11 gives that

$$H_{K(G)}^\lambda = \eta_\lambda I. \quad (6.28)$$

Combining this with Eq. (6.27) above immediately gives

$$\text{eigs}(H_G^\lambda) = \left\{ \{\eta_\lambda\} - \text{eigs} \left(\sum_{j=1}^L H_{G_j^c}^\lambda \right) \right\} \quad (6.29)$$

where subtraction between sets again denotes Minkowski subtraction. Then, to prove the claim, all that remains is to show

$$\text{eigs} \left(\sum_{j=1}^L H_{G_j^c}^\lambda \right) = \sum_{j=1}^L \left\{ \bigcup_{\xi \in \text{Rstrct}(\nu_j, \lambda)} \text{eigs}(H_{G_j^c}^\xi) \right\}. \quad (6.30)$$

We do this in two steps. First, note that because the graphs $G_1^c, \dots, G_L^c$ are disjoint, we have

$$\text{eigs} \left(\sum_{j=1}^L H_{G_j^c}^\lambda \right) = \sum_{j=1}^L \left\{ \text{eigs} \left(H_{G_j^c}^\lambda \right) \right\}. \quad (6.31)$$

by Lemma 6.7.

Next, consider the irrep Hamiltonian $H_{G_j^c}^\lambda$ corresponding to a single connected component of G^c . Recall that G has n vertices and G_j^c has ν_j vertices, and note that λ specifies a two row irrep of S_n . For concreteness say this is the $[n-k, k]$ irrep. Then $H_{G_j^c}^\lambda$ corresponds to a representation of an element of S_{ν_j} inside the λ irrep of S_n , i.e., the *restriction* of a S_n irrep to an element of S_{ν_j} . But, by Lemma 6.6, this restriction is isomorphic to a direct sum over all irreps in $\text{Rstrct}(\nu_j, \lambda)$ (each occurring with some nonzero multiplicity). Then we conclude

$$\text{eigs} \left(H_{G_j^c}^\lambda \right) = \bigcup_{\xi \in \text{Rstrct}(\nu_j, \lambda)} \text{eigs}(H_{G_j^c}^\xi). \quad (6.32)$$

Combining Eqs. (6.31) and (6.32) proves the result, and we are done. $\square$

The dimension of an irrep Hamiltonian's H_G^λ can scale exponentially with the number of vertices in the graph G . Thus, in general we should not expect to be able to keep track of the full spectra of H_G^λ . However a straightforward corollary of Lemma 6.8 is that we can also keep track of the r maximum and minimum values of a irrep Hamiltonian H_G^λ provided we know the r maximum and minimum eigenvalues of the irrep Hamiltonians of the graphs appearing in the complement of G .

Corollary 6.9. *Define G , G^c , $G_1^c, \dots, G_L^c$ and $\text{Rstrct}(\nu, [n-k, k])$ as in Lemma 6.8. Also, for any integer r and finite set $S \subset \mathbb{R}$, let $r\text{-max}(S)$ denote the r largest elements of S and $r\text{-min}(S)$ denote the r smallest elements of S . Then we have*

$$r\text{-max} \left(\text{eigs}(H_G^\lambda) \right) = r\text{-max} \left(\left\{ \left\{ \eta_\lambda \right\} - \sum_{j=1}^L \left\{ \bigcup_{\xi \in \text{Rstrct}(\nu_j, \lambda)} r\text{-min} \left(\text{eigs}(H_{G_j^c}^\xi) \right) \right\} \right\} \right). \quad (6.33)$$

In particular:

$$\text{eig}_{\max}(H_G^\lambda) = \max \left(\left\{ \left\{ \eta_\lambda \right\} - \sum_{j=1}^L \left\{ \bigcup_{\xi \in \text{Rstrct}(\nu_j, \lambda)} \text{eig}_{\min}(H_{G_j^c}^\xi) \right\} \right\} \right) \quad (6.34)$$

$$= \eta_\lambda - \sum_{j=1}^L \left(\min_{\xi \in \text{Rstrct}(\nu_j, \lambda)} \left(\text{eig}_{\min}(H_{G_j^c}^\xi) \right) \right). \quad (6.35)$$

Corollary 6.9 shows that we can compute the maximum (resp. minimum) eigenvalues of the irrep Hamiltonian H_G^λ of a connected graph G provided we know the minimum (resp. maximum) eigenvalues of the irrep Hamiltonian's of each connected component in the complement of G . We make this observation formally in Algorithm 7.

Before stating this algorithm, we remind the reader of the use the phrase “all two row irrep Hamiltonians of G ” introduced in Section 2.4, which refers to the set of irrep Hamiltonians

$$\{H_G^{[n-k, k]} : 1 \leq k \leq \lfloor n/2 \rfloor\}. \quad (6.36)$$

This convention is particularly important in the following algorithms, where the graphs $\tilde{G}$ (and hence the set of corresponding irreps) considered vary throughout the algorithm.

Algorithm 7: Inductive Step in an Eigenvalue Computation

Input : An n -vertex connected graph G whose complement G^c has connected components $\mathcal{G}^c = \{G_1^c, \dots, G_k^c\}$.

Min and max eigenvalues of all two row irrep Hamiltonians of $\tilde{G}$ for each $\tilde{G} \in \mathcal{G}^c$.

Output: Min and max eigenvalues all two row irrep Hamiltonians of G .

foreach two row irrep λ of S_n **do**

$\text{eig}_{\max}(H_G^\lambda) := \eta_\lambda - \sum_{\tilde{G} \in \mathcal{G}^c} \left(\min_{\xi \in \text{Rstrct}(|V(\tilde{G})|, \lambda)} \left(\text{eig}_{\min}(H_{\tilde{G}}^\xi) \right) \right);$
 $\text{eig}_{\min}(H_G^\lambda) := \eta_\lambda - \sum_{\tilde{G} \in \mathcal{G}^c} \left(\max_{\xi \in \text{Rstrct}(|V(\tilde{G})|, \lambda)} \left(\text{eig}_{\max}(H_{\tilde{G}}^\xi) \right) \right);$

end

Now by applying Algorithm 7 repeatedly we can compute the maximum and minimum eigenvalues of an irrep Hamiltonian H_G^λ (and hence the maximum and minimum eigenvalues of the QMC Hamiltonian H_G) provided we know the maximum and minimum eigenvalues of all the irrep Hamiltonians of all graphs G corresponding to the leaves in the tree clique decomposition of G . We make this process formal in Algorithm 8.

Algorithm 8: Tree Clique Eigenvalue Computation

Input : An n -vertex graph G along with its tree clique decomposition $\mathcal{T}(G) = \{T, \{G(v_1), \dots, G(v_m)\}\}$.

Min and max eigenvalues of all two row irrep Hamiltonians of $G(l)$ for every leaf vertex l of T .

Output: Min and Max eigenvalues of all two row irrep Hamiltonians of G .

$d := \text{depth}(T);$

foreach i in $(d - 1, d - 2, \dots, 1)$ **do**

foreach non-leaf vertex $w \in T$ at depth i **do**

Assert that we know the max and min eigenvalues of all two row irrep Hamiltonians of $G(c)$ for each child c of w ;
 Use Algorithm 7 to compute the max and min eigenvalues of all two row irrep Hamiltonians of $G(w)$ from the irrep Hamiltonians of all children $G(c)$;

end

end

Next we prove correctness and bounding the runtime of Algorithms 7 and 8, then giving some remarks concerning generalizations and consequences of these algorithms.

Theorem 6.10. *Algorithms 7 and 8 both correctly compute the irrep eigenvalues of their input graph G . Algorithm 7 runs in time $O(n^2)$ and Algorithm 8 runs in time $O(n^3)$.*

Proof. Correctness of Algorithm 7 follows immediately from Corollary 6.9. To bound its runtime, first note that there are at most $m/2 \in O(m)$ different two row irreps corresponding to any m -vertex graph $\tilde{G}$. Additionally, we have $\bigcup_{\tilde{G} \in \mathcal{G}^c} = G^c$ by construction and $|V(G^c)| = n$ by assumption. So the inner loop of Algorithm 7 involves maximizing/minimizing over partitions of at most n elements and then summing over the results of that maximization/minimization, all of which can be done in time $O(n)$. The graph G has at most n vertices, hence at most $O(n)$ two row irreps, and so the outer loop of Algorithm 7 is repeated at most $O(n)$ times. The runtime of $O(n^2)$ follows.

Correctness of Algorithm 8 follows from the observation that the **assert** statement in the inner loop of the algorithm is always satisfied, since we know the max and min eigenvalues of the irrep Hamiltonians for graphs corresponding to leaf vertices by assumption, and all other graph irrep Hamiltonians are computed from the “bottom up” by construction. To bound the runtime of Algorithm 8 note that, by Item (2) of Theorem 6.3 and the bound on the runtime of Algorithm 7 given above, for any $i \in \{d-1, \dots, 1\}$, Algorithm 8 requires time at most $O(n^2)$ to compute the max and min irrep eigenvalues of all graphs $G(w)$ corresponding to vertices w at depth i in the tree clique decomposition of G . Then Item (1) of Theorem 6.3 gives that $d \leq n$ and the upper bound of $O(n^3)$ follows. $\square$

Remark 6.11. A straightforward generalization of Algorithms 7 and 8 using Corollary 6.9 gives us an algorithm computing the r highest and lowest eigenvalues of all irrep Hamiltonians of G given the r highest and lowest eigenvalues of all irrep Hamiltonians of all graphs indexed by leaves in the clique tree decomposition of G . The runtime of this algorithm remains polynomial in r and n . $\square$

Remark 6.12. If all the graphs indexed by leaves in the clique tree decomposition of G are cliques, then we can compute all eigenvalues of all irrep Hamiltonians of those graphs in exact arithmetic by Lemmas 2.11 and 2.12. In this case all the eigenvalues of G will be obtained by taking sums and differences of clique eigenvalues η_λ for different values of λ and, in particular, all eigenvalues will be integer.

We give an example of a graph G admitting such a decomposition in the next section. $\square$

Remark 6.13. If we apply the same procedure as described in Algorithm 8 but take as input approximate eigenvalues of the irrep Hamiltonians of the graphs indexed by leaves in the clique tree decomposition of G we obtain approximate eigenvalues of G . It is straightforward to show that the error in this approximation is at worst the sum of the approximation errors for the eigenvalues of each input graph. Additionally, if the algorithm is given upper/lower bounds on the minimum/maximum eigenvalues of all irrep Hamiltonians of graphs indexed by leaves in the tree clique decomposition of G , it produces upper/lower bounds on the maximum/minimum eigenvalues of G .

One way to obtain these bounds is by running the ncSoS algorithm inside all two row irreps of the graphs indexed by leaves in the tree clique decomposition of G . Details of this use of ncSoS is given in Section 5.5. $\square$

6.4 Example of Clique Decomposition, its Hamiltonians and their Eigenvalues

In this subsection we apply the algorithms discussed previously in this section to analyze the graph G shown in Figure 4.

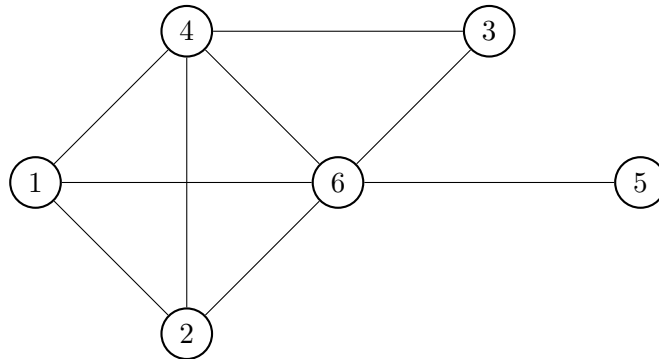


Figure 4: The graph G to be analyzed in this example.

6.4.1 The tree clique decomposition of G and the Hamiltonian decomposition

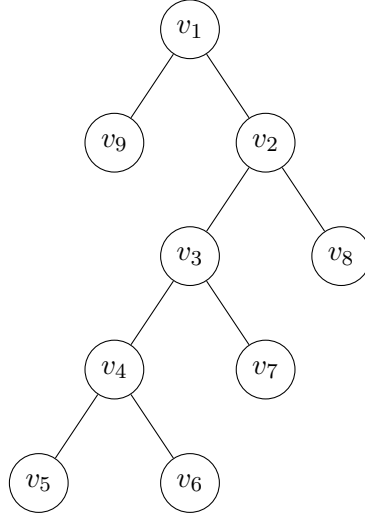


Figure 5: The output tree, T

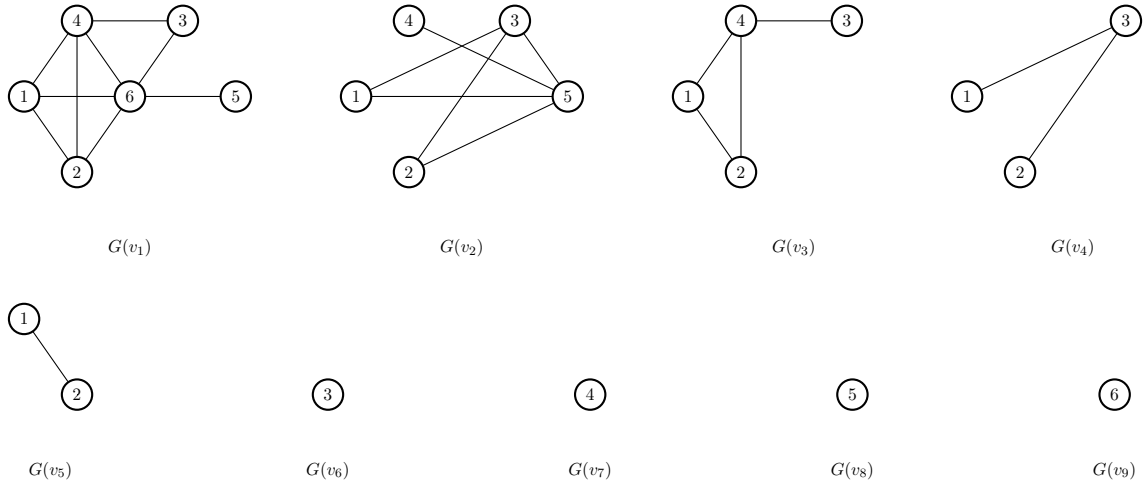


Figure 6: The output graphs. The second row of graphs corresponds to the leaves L of T , the first row to R . Note $G(v_1)^c$ is the union of its children, the two depth 1 graphs $G(v_9)$ and $G(v_2)$. Likewise $G(v_2)^c$ is the union of two depth 2 graphs and $G(v_3)^c$ is the union of two depth 3 graphs and $G(v_4)^c$ is the union of two depth 4 graphs. This can be read off from the tree.

To write down the Hamiltonian clique decomposition for G , first see that the leaves of tree T are $L := \{v_9, v_5, v_6, v_7, v_8\}$ and the rest are $R := \{v_1, v_2, v_3, v_4\}$. The corresponding depths $d(v_j)$ are read off from T and listed in Table 3

	$G(v_1)$	$G(v_2)$	$G(v_3)$	$G(v_4)$	$G(v_5)$
Depth in T	0	1	2	3	4
SignHam		+	-	+	-
Dim $K(G(v))$	6	5	4	3	2

Table 3: Clique decomposition of graph G , ignoring single vertex graphs

and force the signs of the decomposing Hamiltonians in the table. By Theorem 6.5 Item (2)

the Hamiltonian H_G has the decomposition:

$$H_G = +H_{K(G(v_1))} - H_{K(G(v_2))} + H_{K(G(v_3))} - H_{K(G(v_4))} \quad (6.37)$$

$$+ H_{G(v_5)} + H_{G(v_6)} - H_{G(v_7)} + H_{G(v_8)} - H_{G(v_9)} \quad (6.38)$$

Next use that $G(v_j)$ for $j = 6, 7, 8, 9$ are singletons, to see the corresponding Hamiltonians are 0; also $K(G(v_5)) = G(v_5)$. Thus

$$H_G = +H_{K(G(v_1))} - H_{K(G(v_2))} + H_{K(G(v_3))} - H_{K(G(v_4))} + H_{K(G(v_5))}, \quad (6.39)$$

a sum/difference of clique Hamiltonians.

We mention that for this special example, one sees from Figure 4 the containments $V(G(v_j)) \supseteq V(G(v_{j+1}))$ for $j = 1, 2, 3, 4$, so

$$K(G(v_1)) \supseteq K(G(v_2)) \supseteq K(G(v_3)) \supseteq K(G(v_4)) \supseteq K(G(v_5)) = K(G(v_5)). \quad (6.40)$$

6.4.2 Eigenvalues

Now we see how one can use Algorithm 8 to compute max/min eigenvalues of H_G ; we restrict to one case $\text{eig}_{\max}(H_G^{[3,3]})$, since the min eigenvalues and other irreps behave similarly. Firstly, we list the data needed for eigenvalues (and also above).

	K_6	K_5	K_4	K_3	K_2
Sign	+, max	-, min	+, max	-, min	+, max
	$\eta_{[3,3]} = 24$	$\eta_{[3,2]} = 16$	$\eta_{[2,2]} = 12$	$\eta_{[2,1]} = 6$	$\eta_{[2,0]} = 0$
	x	x	$\eta_{[3,1]} = 8$	$\eta_{[2,1]} = 6$	$\eta_{[1,1]} = 4$
	x	x	x	$\eta_{[3,0]} = 0$	$\eta_{[2,0]} = 0$

Table 4: G and Irrep[3, 3]. Data for computing $\text{eig}_{\max}(H_G^{[3,3]})$. The number next to $[m, k]$ is $\eta_{m,k}$. Entries to the right are needed in light of those to the left because of Young's Branching Rule.

We shall apply Algorithm 7 repeatedly to compute $\text{eig}_{\max}(H_G^{[3,3]})$.

$$\text{eig}_{\max}(H_G^{[3,3]}) = \eta_{[3,3]} - \left(\min_{\xi \in \text{Rstrct}(5, [3,3])} \left(\text{eig}_{\min}(H_{G(v_2)}^\xi) \right) \right). \quad (6.41)$$

$$= \eta_{[3,3]} - \text{eig}_{\min}(H_{G(v_2)}^{[3,2]}) \quad \text{and} \quad (6.42)$$

$$\text{eig}_{\min}(H_{G(v_2)}^{[3,2]}) = \eta_{[3,2]} - \max_{\xi \in \text{Rstrct}(4, [3,2])} \left(\text{eig}_{\max}(H_{G(v_3)}^\xi) \right) \quad (6.43)$$

$$= \eta_{[3,2]} - \max \left(\text{eig}_{\max}(H_{G(v_3)}^{[3,1]}), \text{eig}_{\max}(H_{G(v_3)}^{[2,2]}) \right) \quad (6.44)$$

Combine these to get

$$\text{eig}_{\max}(H_G^{[3,3]}) = \eta_{[3,3]} - \eta_{[3,2]} + \max \left(\text{eig}_{\max}(H_{G(v_3)}^{[3,1]}), \text{eig}_{\max}(H_{G(v_3)}^{[2,2]}) \right) \quad (6.45)$$

Next

$$\text{eig}_{\max}(H_{G(v_3)}^{[3,1]}) = \eta_{3,1} - \min \left(\text{eig}_{\min}(H_{G(v_4)}^{[3,0]}), \text{eig}_{\min}(H_{G(v_4)}^{[2,1]}) \right) \quad (6.46)$$

$$\text{eig}_{\max}(H_{G(v_3)}^{[2,2]}) = \eta_{2,2} - \text{eig}_{\min}(H_{G(v_4)}^{[2,1]}) \quad (6.47)$$

Before combining again we compute the $\text{eig}_{\max}$ ingredients of these formulas:

$$\text{eig}_{\min}(H_{G(v_4)}^{[3,0]}) = \eta_{3,0} - \text{eig}_{\max}(H_{G(v_5)}^{[2,0]}) = \eta_{3,0} - \eta_{[2,0]}$$

$$\text{eig}_{\min}(H_{G(v_4)}^{[2,1]}) = \eta_{2,1} - \max\left(\text{eig}_{\max}(H_{G(v_5)}^{[2,0]}, \text{eig}_{\max}(H_{G(v_5)}^{[1,1]})\right) = \eta_{2,1} - \max\left(\eta_{[2,0]}, \eta_{[1,1]}\right)$$

The last step in each line uses that $G(v_5)$ is a clique. Now we combine all these, working backward to get

$$\begin{aligned}\text{eig}_{\max}(H_{G(v_3)}^{[3,1]}) &= \eta_{3,1} - \min\left(\eta_{3,0} - \eta_{[2,0]}, (\eta_{2,1} - \max(\eta_{[2,0]}, \eta_{[1,1]}))\right) \\ \text{eig}_{\max}(H_{G(v_3)}^{[2,2]}) &= \eta_{2,2} - (\eta_{2,1} - \max(\eta_{[2,0]}, \eta_{[1,1]}))\end{aligned}$$

and finally substituting these into Eq. (6.45) gives

$$\begin{aligned}\text{eig}_{\max}(H_G^{[3,3]}) &= \eta_{[3,3]} - \eta_{[3,2]} \\ &+ \max\left(\eta_{3,1} - \min\left(\eta_{3,0} - \eta_{[2,0]}, \eta_{2,1} - \max(\eta_{[2,0]}, \eta_{[1,1]})\right), \eta_{2,2} - \eta_{2,1} + \max(\eta_{[2,0]}, \eta_{[1,1]})\right).\end{aligned}$$

Now we substitute numbers in for the $\eta_{m,k}$ and get

$$\text{eig}_{\max}(H_G^{[3,3]}) = 24 - 16 + \max(8 - \min(0 - 0, 6 - \max(0, 4)), 12 - 6 + \max(0, 4)) = 18,$$

which by our theory is $\text{eig}_{\max}(H_G^{[3,3]})$, as can be independently checked. We note that this “working backwards” step we just worked through is essentially the algorithm for computing max eigenvalues outlined in Algorithm 8.

References

- [AGM20] Anurag Anshu, David Gosset, and Karen Morenz. Beyond Product State Approximations for a Quantum Analogue of Max Cut. In *15th Conference on the Theory of Quantum Computation, Communication and Cryptography TQC 2020*, volume 158 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:15, Riga, Latvia, 2020. Schloss Dagstuhl - Leibniz-Zentrum für Informatik GmbH, Dagstuhl Publishing. OCLC: 1194977622. URL: <https://drops.dagstuhl.de/opus/volltexte/2020/12066>, doi:10.4230/LIPIcs.TQC.2020.7. [p. 4]
- [BC17] Jacob C Bridgeman and Christopher T Chubb. Hand-waving and interpretive dance: an introductory course on tensor networks. *Journal of Physics A: Mathematical and Theoretical*, 50(22):223001, may 2017. URL: <https://dx.doi.org/10.1088/1751-8121/aa6dc3>, doi:10.1088/1751-8121/aa6dc3. [p. 4]
- [BCP97] Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993). URL: <http://dx.doi.org/10.1006/jSCO.1996.0125>, doi:10.1006/jSCO.1996.0125. [p. 45]
- [BFS15] Magali Bardet, Jean-Charles Faugère, and Bruno Salvy. On the complexity of the F_5 Gröbner basis algorithm. *J. Symb. Comput.*, 70:49–70, 2015. doi:10.1016/j.jsc.2014.09.025. [p. 42]
- [BGKT19] Sergey Bravyi, David Gosset, Robert König, and Kristan Temme. Approximation algorithms for quantum many-body problems. *Journal of Mathematical Physics*, 60(3):032203, March 2019. URL: <http://aip.scitation.org/doi/10.1063/1.5085428>, doi:10.1063/1.5085428. [pp. 3, 4, 5]
- [BH13] Fernando G.S.L. Brandao and Aram W. Harrow. Product-state approximations to quantum ground states. In *Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing, STOC '13*, page 871–880, New York, NY, USA, 2013. Association for Computing Machinery. doi:10.1145/2488608.2488719. [pp. 3, 4, 5, 30]

- [BKP16] Sabine Burgdorf, Igor Klep, and Janez Povh. *Optimization of polynomials in non-commuting variables*. SpringerBriefs in Mathematics. Springer, [Cham], 2016. doi:10.1007/978-3-319-33338-0. [p. 31]
- [BPR22] Calum Buchanan, Christopher Purcell, and Puck Rombach. Subgraph completion and minimum rank. *Electron. J. Combin.*, 29(1):Paper No. 1.38, 20, 2022. doi:10.37236/10383. [p. 50]
- [BRS11] Boaz Barak, Prasad Raghavendra, and David Steurer. Rounding semidefinite programming hierarchies via global correlation. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science—FOCS 2011*, pages 472–481. IEEE Computer Soc., Los Alamitos, CA, 2011. doi:10.1109/FOCS.2011.95. [p. 5]
- [CLO15] David A. Cox, John Little, and Donal O’Shea. *Ideals, varieties, and algorithms. An introduction to computational algebraic geometry and commutative algebra*. Undergraduate Texts in Mathematics. Springer, Cham, fourth edition, 2015. doi:10.1007/978-3-319-16721-3. [p. 40]
- [CMP18] Toby S. Cubitt, Ashley Montanaro, and Stephen Piddock. Universal quantum Hamiltonians. *Proceedings of the National Academy of Sciences*, 115(38):9497–9502, September 2018. URL: <https://pnas.org/doi/full/10.1073/pnas.1804949115>, doi:10.1073/pnas.1804949115. [p. 16]
- [CPGSV21] J. Ignacio Cirac, David Pérez-García, Norbert Schuch, and Frank Verstraete. Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. *Rev. Mod. Phys.*, 93:045003, Dec 2021. URL: <https://link.aps.org/doi/10.1103/RevModPhys.93.045003>, doi:10.1103/RevModPhys.93.045003. [p. 4]
- [dK02] Etienne de Klerk. *Aspects of semidefinite programming*, volume 65 of *Applied Optimization*. Kluwer Academic Publishers, Dordrecht, 2002. Interior point algorithms and selected applications. doi:10.1007/b105286. [p. 44]
- [DLTW08] Andrew C. Doherty, Yeong-Cherng Liang, Ben Toner, and Stephanie Wehner. The quantum moment problem and bounds on entangled multi-prover games. In *2008 23rd Annual IEEE Conference on Computational Complexity*, pages 199–210. IEEE, 2008. doi:10.1109/CCC.2008.26. [p. 5]
- [EGH⁺11] Pavel Etingof, Oleg Golberg, Sebastian Hensel, Tiankai Liu, Alex Schwendner, Dmitry Vaintrob, and Elena Yudovina. *Introduction to representation theory*, volume 59 of *Student Mathematical Library*. American Mathematical Society, Providence, RI, 2011. With historical interludes by Slava Gerovitch. doi:10.1090/stml/059. [pp. 12, 13]
- [Fon20] Renato M. Fonseca. Groupmath: A mathematica package for group theory calculations. 2020. arXiv:2011.01764 [hep-th]. arXiv:arXiv:2011.01764, doi:10.1016/j.cpc.2021.108085. [p. 14]
- [GK12] Sevag Gharibian and Julia Kempe. Approximation algorithms for QMA-complete problems. *SIAM Journal on Computing*, 41(4):1028–1050, 2012. doi:10.1137/110842272. [p. 3]
- [GP19] Sevag Gharibian and Ojas Parekh. Almost Optimal Classical Approximation Algorithms for a Quantum Generalization of Max-Cut. volume 145 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:17. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019. Artwork Size: 17 pages Medium: application/pdf Publisher: Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik GmbH, Wadern/Saarbruecken, Germany Version Number: 1.0. URL: <http://drops.dagstuhl.de/opus/volltexte/2019/11246/>, doi:10.4230/LIPICS.APPROX-RANDOM.2019.31. [pp. 4, 5, 30]

- [Gre00] Edward L. Green. Multiplicative bases, Gröbner bases, and right Gröbner bases. *Journal of Symbolic Computation*, 29(4-5):601–623, 2000. doi:10.1006/jsco.1999.0324. [pp. 40, 41]
- [GW95] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, November 1995. URL: <https://dl.acm.org/doi/10.1145/227683.227684>, doi:10.1145/227683.227684. [p. 5]
- [Har13] Aram Wettroth Harrow. The church of the symmetric subspace, August 2013. arXiv:1308.6595 [quant-ph]. doi:10.48550/arXiv.1308.6595. [p. 7]
- [HM04] J. William Helton and Scott McCullough. A Positivstellensatz for noncommutative polynomials. *Transactions of the American Mathematical Society*, 356(9):3721–3737, 2004. doi:10.1016/j.aim.2012.04.028. [pp. 5, 36]
- [HM17] Aram W. Harrow and Ashley Montanaro. Extremal eigenvalues of local Hamiltonians. *Quantum*, 1:6, April 2017. doi:10.22331/q-2017-04-25-6. [p. 3]
- [HNP⁺22] Yeongwoo Hwang, Joe Neeman, Ojas Parekh, Kevin Thompson, and John Wright. Unique Games hardness of Quantum Max-Cut, and a conjectured vector-valued Borell’s inequality, September 2022. arXiv:2111.01254 [quant-ph]. URL: <http://arxiv.org/abs/2111.01254>, doi:10.48550/arXiv.2111.01254. [p. 5]
- [HO22] Matthew B. Hastings and Ryan O’Donnell. Optimizing strongly interacting fermionic Hamiltonians. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, page 776–789, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3519935.3519960. [pp. 4, 5]
- [Jam06] Gordon Douglas James. *The representation theory of the symmetric groups*, volume 682. Springer, 2006. doi:10.1007/BFb0067708. [p. 14]
- [Kho02] Subhash Khot. On the power of unique 2-prover 1-round games. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, pages 767–775, 2002. doi:10.1145/509907.510017. [p. 5]
- [Kin23] Robbie King. An Improved Approximation Algorithm for Quantum Max-Cut on Triangle-Free Graphs. *Quantum*, 7:1180, November 2023. doi:10.22331/q-2023-11-09-1180. [pp. 4, 5]
- [KKR06] Julia Kempe, Alexei Kitaev, and Oded Regev. The complexity of the local hamiltonian problem. *SIAM Journal on Computing*, 35(5):1070–1097, 2006. arXiv:<https://doi.org/10.1137/S0097539704445226>, doi:10.1137/S0097539704445226. [p. 3]
- [KPS18] Se-Jin Kim, Vern Paulsen, and Christopher Schafhauser. A synchronous game for binary constraint systems. *Journal of Mathematical Physics*, 59(3):032201, 2018. doi:10.1063/1.4996867. [p. 5]
- [KSV02] Alexei Ju Kitaev, Aleksandr Ch Shen, and Michail N. Vyalii. *Classical and quantum computation*. Number 47 in Graduate studies in mathematics. American Mathematical Society, Providence, RI, 2002. doi:10.1090/gsm/047. [p. 3]
- [Lee22] Eunou Lee. Optimizing quantum circuit parameters via SDP. In *33rd International Symposium on Algorithms and Computation*, volume 248 of *LIPICs. Leibniz Int. Proc. Inform.*, pages Paper No. 48, 16. Schloss Dagstuhl. Leibniz-Zent. Inform., Wadern, 2022. doi:10.4230/lipics.isaac.2022.48. [pp. 4, 5]
- [LM62] Elliott Lieb and Daniel Mattis. Ordering energy levels of interacting spin systems. *Journal of Mathematical Physics*, 3(4):749–751, jan 1962. doi:10.1063/1.1724276. [pp. 1, 3, 4, 6, 49, 87]

- [LM16] Fedor Levkovich-Maslyuk. The Bethe Ansatz. *Journal of Physics A: Mathematical and Theoretical*, 49(32):323004, 2016. doi:10.1088/1751-8113/49/32/323004. [p. 4]
- [LVV15] Zeph Landau, Umesh Vazirani, and Thomas Vidick. A polynomial time algorithm for the ground state of one-dimensional gapped local Hamiltonians. *Nature Physics*, 11(7):566–569, 2015. doi:10.1038/nphys3345. [p. 3]
- [Mor86] Ferdinando Mora. Groebner bases for noncommutative polynomial rings. In *Algebraic algorithms and error correcting codes (Grenoble, 1985)*, volume 229 of *Lecture Notes in Comput. Sci.*, pages 353–362. Springer, Berlin, 1986. URL: https://doi.org/10.1007/3-540-16776-5_740, doi:10.1007/3-540-16776-5_740. [pp. 40, 41, 43]
- [NPA08] Miguel Navascués, Stefano Pironio, and Antonio Acín. A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations. *New Journal of Physics*, 10(7):073013, July 2008. URL: <https://iopscience.iop.org/article/10.1088/1367-2630/10/7/073013>, doi:10.1088/1367-2630/10/7/073013. [p. 5]
- [Osb06] Tobias J. Osborne. Statics and dynamics of quantum XY and Heisenberg systems on graphs. *Physical Review B*, 74:094411, Sep 2006. URL: <https://link.aps.org/doi/10.1103/PhysRevB.74.094411>, doi:10.1103/PhysRevB.74.094411. [pp. 9, 16]
- [PKT22] Maurice Pouzet, Hamza Si Kaddour, and Bhalchandra Thatte. On the boolean dimension of a graph and other related parameters. *Discrete Mathematics & Theoretical Computer Science*, 23(Special issues):2, #5, 2022. doi:10.46298/dmtcs.7437. [p. 50]
- [PM17] Stephen Piddock and Ashley Montanaro. The complexity of antiferromagnetic interactions and 2D lattices. *Quantum Information & Computation*, 17(7&8):636–672, 2017. doi:10.26421/QIC17.7-8-6. [p. 4]
- [Pro07] Claudio Procesi. *Lie groups*. Universitext. Springer, New York, 2007. An approach through invariants and representations. doi:10.1007/978-0-387-28929-8. [pp. 6, 7, 13, 22, 24]
- [Pro21] Claudio Procesi. A note on the Formanek Weingarten function. *Note Mat.*, 41(1):69–109, 2021. doi:10.1285/i15900932v41n1p69. [pp. 13, 44]
- [PT21] Ojas Parekh and Kevin Thompson. Application of the Level-2 Quantum Lasserre Hierarchy in Quantum Approximation Algorithms. In *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, volume 198 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 102:1–102:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. URL: <https://drops.dagstuhl.de/opus/volltexte/2021/14171>, doi:10.4230/LIPIcs.ICALP.2021.102. [pp. 4, 5, 6, 9]
- [PT22] Ojas Parekh and Kevin Thompson. An Optimal Product-State Approximation for 2-Local Quantum Hamiltonians with Positive Terms, June 2022. arXiv:2206.08342 [quant-ph]. URL: <http://arxiv.org/abs/2206.08342>, doi:10.48550/arXiv.2206.08342. [pp. 4, 5, 6]
- [Rag08] Prasad Raghavendra. Optimal algorithms and inapproximability results for every CSP? In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 245–254, 2008. doi:10.1145/1374376.1374414. [p. 5]
- [Rag09] Prasad Raghavendra. *Approximating NP-hard problems efficient algorithms and their limits*. University of Washington, 2009. PhD thesis. [p. 5]

- [Sta99] Richard P. Stanley. *Enumerative combinatorics. Vol. 2*, volume 62 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, Cambridge, 1999. With a foreword by Gian-Carlo Rota and appendix 1 by Sergey Fomin. [doi:10.1017/CB09780511609589](https://doi.org/10.1017/CB09780511609589). [p. 25]
- [TRZ⁺] Jun Takahashi, Chaithanya Rayudu, Cunlu Zhou, Robbie King, Kevin Thompson, and Ojas Parekh. An $SU(2)$ -symmetric semidefinite programming hierarchy for Quantum Max Cut. *preprint*. [pp. 3, 7, 8, 85, 86, 87, 88]
- [Xiu12] Xingqiang Xiu. *Non-commutative Gröbner bases and applications*. PhD thesis, 2012. [pp. 40, 41]

Accepted in *Quantum* 2024-03-25, click title to verify. Published under CC-BY 4.0.

$$\text{Swap}_{12} \text{Swap}_{13}^2 = \text{Swap}_{12}$$

$$\text{Swap}_{12} \text{ Swap}_{13} \text{ Swap}_{23} = \text{Swap}_{13}$$

$$\begin{aligned} \text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{35} = & \frac{-1}{2} - \frac{\text{Swap}_{12}}{2} + \frac{\text{Swap}_{12} \text{Swap}_{13}}{2} + \frac{\text{Swap}_{12} \text{Swap}_{15}}{2} + \frac{\text{Swap}_{12} \text{Swap}_{35}}{2} - \frac{\text{Swap}_{13} \text{Swap}_{15}}{2} + \\ & \frac{\text{Swap}_{13} \text{Swap}_{25}}{2} + \frac{\text{Swap}_{15}}{2} - \frac{\text{Swap}_{15} \text{Swap}_{23}}{2} + \frac{\text{Swap}_{23}}{2} - \frac{\text{Swap}_{23} \text{Swap}_{25}}{2} + \frac{\text{Swap}_{35}}{2} \end{aligned}$$

$$\text{Swap}_{12} \text{ Swap}_{14} \text{ Swap}_{12} = \text{Swap}_{24}$$

$$\text{Swap}_{12} \text{Swap}_{14}^2 = \text{Swap}_{12}$$

$$\text{Swap}_{12} \text{ Swap}_{14} \text{ Swap}_{24} = \text{Swap}_{14}$$

$$\text{Swap}_{12} \text{Swap}_{14} \text{Swap}_{35} = \text{Swap}_{12} \text{Swap}_{14} \text{Swap}_{35}$$

$$\text{Swap}_{12} \text{ Swap}_{15} \text{ Swap}_{12} = \text{Swap}_{25}$$

$$\text{Swap}_{12} \text{Swap}_{15}^2 = \text{Swap}_{12}$$

Accepted in *Quantum* 2024-03-25, click title to verify. Published under CC-BY 4.0. 69

Accepted in *Quantum* 2024-03-25, click title to verify. Published under CC-BY 4.0. 70

Accepted in *Quantum* 2024-03-25, click title to verify. Published under CC-BY 4.0. 71

[illegible]

B.1 Linear space spanned by the products of at most three swap matrices

Lemma B.1. *A basis $\mathcal{B}_3$ for the linear span of all products of at most three swap matrices in the case $n = 5$ is $\mathcal{B}_2$ together with*

$$\begin{aligned} & \text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{45}, \quad \text{Swap}_{12} \text{Swap}_{14} \text{Swap}_{35}, \quad \text{Swap}_{12} \text{Swap}_{15} \text{Swap}_{34}, \\ & \text{Swap}_{13} \text{Swap}_{14} \text{Swap}_{25}, \quad \text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}, \quad \text{Swap}_{14} \text{Swap}_{15} \text{Swap}_{23}, \end{aligned}$$

Alternately, $\text{Swap}_{14} \text{Swap}_{15} \text{Swap}_{23}$ could be replaced by $\text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{35}$ in $\mathcal{B}_3$.

Proof. The add-on statement follows from the equality

$$\begin{aligned} 2 \text{Swap}_{14} \text{Swap}_{15} \text{Swap}_{23} &= \text{Swap}_{12} - \text{Swap}_{14} - \text{Swap}_{23} + \text{Swap}_{24} - \text{Swap}_{25} + \text{Swap}_{35} \\ &\quad - \text{Swap}_{12} \text{Swap}_{34} - \text{Swap}_{12} \text{Swap}_{35} - \text{Swap}_{12} \text{Swap}_{45} \\ &\quad + \text{Swap}_{13} \text{Swap}_{14} - \text{Swap}_{13} \text{Swap}_{15} - \text{Swap}_{13} \text{Swap}_{24} \\ &\quad + \text{Swap}_{13} \text{Swap}_{25} + \text{Swap}_{14} \text{Swap}_{15} + \text{Swap}_{14} \text{Swap}_{23} \\ &\quad + \text{Swap}_{14} \text{Swap}_{25} + \text{Swap}_{15} \text{Swap}_{23} - \text{Swap}_{15} \text{Swap}_{24} \\ &\quad + \text{Swap}_{23} \text{Swap}_{45} - \text{Swap}_{24} \text{Swap}_{35} + \text{Swap}_{25} \text{Swap}_{34} \\ &\quad - \text{Swap}_{34} \text{Swap}_{35} + 2 \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{35} \\ &\quad - 2 \text{Swap}_{13} \text{Swap}_{14} \text{Swap}_{25} + 2 \text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}. \end{aligned}$$

For the spanning property it suffices to consider products of three swap matrices of the form $b \text{Swap}_{pq}$ for some quadratic $b \in \mathcal{B}_2$, leaving us with $25 \cdot 10 - 6 = 244$ cases to treat. The identities needed are given in Appendix A.

Finally, to establish linear independence, assume there is a linear combination of elements of $\mathcal{B}_3$ that vanishes. Now consider the six cubic terms expanded in terms of the Paulis. Then $\sigma_X^1 \sigma_Z^2 \sigma_Y^3 \sigma_X^4 \sigma_X^5$ only appears in $\text{Swap}_{12} \text{Swap}_{13} \text{Swap}_{45}$, whence the coefficient next to it has to be 0. Next, $\sigma_X^1 \sigma_Y^2 \sigma_X^3 \sigma_Z^4 \sigma_X^5$ only appears in $\text{Swap}_{12} \text{Swap}_{14} \text{Swap}_{35}$, so the latter also cannot appear in a linear dependence relation. Similarly, $\sigma_X^1 \sigma_Y^2 \sigma_X^3 \sigma_X^4 \sigma_Z^5$ eliminates $\text{Swap}_{12} \text{Swap}_{15} \text{Swap}_{34}$. Next, $\sigma_X^1 \sigma_Y^2 \sigma_Y^3 \sigma_Z^4 \sigma_Y^5$ appears in $\text{Swap}_{13} \text{Swap}_{14} \text{Swap}_{25}$, but not in $\text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}$ or $\text{Swap}_{14} \text{Swap}_{15} \text{Swap}_{23}$, thus also eliminating $\text{Swap}_{13} \text{Swap}_{14} \text{Swap}_{25}$. Next, $\sigma_X^1 \sigma_Y^2 \sigma_Y^3 \sigma_Y^4 \sigma_Z^5$ appears in $\text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}$ but not in $\text{Swap}_{14} \text{Swap}_{15} \text{Swap}_{23}$, so $\text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}$ also cannot appear in a linear dependence relation. Since $\text{Swap}_{13} \text{Swap}_{15} \text{Swap}_{24}$ contains degree five terms in the Paulis which cannot appear in an element of $\mathcal{B}_2$, we conclude that $\mathcal{B}_3$ is linearly independent. $\square$

Proposition B.2. *A basis $\mathcal{B}_3$ for the linear space spanned by monomials of degree at most three in the swap algebra M_n^{swap} consists of $\mathcal{B}_2$ and two types of cubics,*

(Type 6) $\text{Swap}_{ij} \text{Swap}_{k\ell} \text{Swap}_{pq}, \quad i < j, k < \ell, p < q, (i, j) < (k, \ell) < (p, q)^8;$

(Types 5) $\begin{aligned} & \text{Swap}_{ij} \text{Swap}_{ik} \text{Swap}_{pq}, \quad \text{Swap}_{ij} \text{Swap}_{ip} \text{Swap}_{kq}, \quad \text{Swap}_{ij} \text{Swap}_{iq} \text{Swap}_{kp}, \\ & \text{Swap}_{ik} \text{Swap}_{ip} \text{Swap}_{jq}, \quad \text{Swap}_{ik} \text{Swap}_{iq} \text{Swap}_{jp}, \quad \text{Swap}_{ip} \text{Swap}_{iq} \text{Swap}_{jk}, \\ & \quad \quad \quad i < j < k < p < q. \end{aligned}$

Proof. The spanning property follows from Lemma B.1, so it suffices to establish linear independence of $\mathcal{B}_3$.

We shall mimic the proof of Proposition 4.12. Assume there is a linear dependence among elements of $\mathcal{B}_3$. Consider the cubic terms in the swap matrices in this dependence.

⁸Pairs are compared w.r.t. the lex ordering.

Firstly, let us look at the (Type 6) terms. The expansion of $\text{Swap}_{ij} \text{Swap}_{kl} \text{Swap}_{pq}$ in terms of the Paulis will yield a term $\sigma_X^i \sigma_X^j \sigma_Y^k \sigma_Y^\ell \sigma_Z^p \sigma_Z^q$ that only occurs in one of the (Type 6) elements. Thus the linear dependence cannot contain any (Type 6) elements.

Next, consider (Type 5) elements. Since given $i < j < k < p < q$, not each of the six corresponding (Type 5) cubics has unique degree 5 terms in terms of the Paulis (cf. proof of Lemma B.1), some care is needed. As in Lemma B.1, for each $i < j < k < p < q$, uniqueness of degree five terms eliminates the first four in each list of associated (Type 5) cubics, namely $\text{Swap}_{ij} \text{Swap}_{ik} \text{Swap}_{pq}$, $\text{Swap}_{ij} \text{Swap}_{ip} \text{Swap}_{kq}$, $\text{Swap}_{ij} \text{Swap}_{iq} \text{Swap}_{kp}$, $\text{Swap}_{ik} \text{Swap}_{ip} \text{Swap}_{jq}$. Once these are eliminated, we can as in Lemma B.1 also eliminate the remaining two (Type 5)s for each $i < j < k < p < q$. We are thus left with only terms from $\mathcal{B}_2$ which are linearly independent by Proposition 4.12. $\square$

Remark B.3. The cardinality of $\mathcal{B}_3$ is

$$\begin{aligned} \#\mathcal{B}_3 &= \#\mathcal{B}_2 + \frac{1}{3!} \binom{n}{2} \binom{n-2}{2} \binom{n-4}{2} + 6 \binom{n}{5} \\ &= \frac{1}{240} (5n^6 - 63n^5 + 335n^4 - 845n^3 + 1100n^2 - 532n + 240). \end{aligned}$$

$\square$

B.2 Linear space spanned by the products of at most four swap matrices

While we shall not bore the reader with the full details, the situation does change a bit in degree four products of the swap matrices. Namely, products of four swap matrices on disjoint indices cease to be linearly independent over smaller products, as shown by the following identity:

$$\begin{aligned} \text{Swap}_{18} \text{Swap}_{27} \text{Swap}_{36} \text{Swap}_{45} &= \\ &\text{Swap}_{18} \text{Swap}_{27} \text{Swap}_{34} \text{Swap}_{56} + \text{Swap}_{18} \text{Swap}_{25} \text{Swap}_{36} \text{Swap}_{47} - \text{Swap}_{18} \text{Swap}_{25} \text{Swap}_{34} \text{Swap}_{67} \\ &- \text{Swap}_{18} \text{Swap}_{23} \text{Swap}_{47} \text{Swap}_{56} + \text{Swap}_{18} \text{Swap}_{23} \text{Swap}_{45} \text{Swap}_{67} + \text{Swap}_{16} \text{Swap}_{27} \text{Swap}_{38} \text{Swap}_{45} \\ &- \text{Swap}_{16} \text{Swap}_{27} \text{Swap}_{34} \text{Swap}_{58} - \text{Swap}_{16} \text{Swap}_{25} \text{Swap}_{37} \text{Swap}_{48} + \text{Swap}_{16} \text{Swap}_{25} \text{Swap}_{34} \text{Swap}_{78} \\ &- \text{Swap}_{16} \text{Swap}_{24} \text{Swap}_{38} \text{Swap}_{57} + \text{Swap}_{16} \text{Swap}_{24} \text{Swap}_{37} \text{Swap}_{58} + \text{Swap}_{16} \text{Swap}_{23} \text{Swap}_{48} \text{Swap}_{57} \\ &- \text{Swap}_{16} \text{Swap}_{23} \text{Swap}_{45} \text{Swap}_{78} - \text{Swap}_{15} \text{Swap}_{26} \text{Swap}_{38} \text{Swap}_{47} + \text{Swap}_{15} \text{Swap}_{26} \text{Swap}_{37} \text{Swap}_{48} \\ &+ \text{Swap}_{15} \text{Swap}_{24} \text{Swap}_{38} \text{Swap}_{67} - \text{Swap}_{15} \text{Swap}_{24} \text{Swap}_{37} \text{Swap}_{68} - \text{Swap}_{15} \text{Swap}_{23} \text{Swap}_{48} \text{Swap}_{67} \\ &+ \text{Swap}_{15} \text{Swap}_{23} \text{Swap}_{47} \text{Swap}_{68} - \text{Swap}_{14} \text{Swap}_{27} \text{Swap}_{38} \text{Swap}_{56} + \text{Swap}_{14} \text{Swap}_{27} \text{Swap}_{36} \text{Swap}_{58} \\ &+ \text{Swap}_{14} \text{Swap}_{26} \text{Swap}_{38} \text{Swap}_{57} - \text{Swap}_{14} \text{Swap}_{26} \text{Swap}_{37} \text{Swap}_{58} + \text{Swap}_{14} \text{Swap}_{25} \text{Swap}_{37} \text{Swap}_{68} \\ &- \text{Swap}_{14} \text{Swap}_{25} \text{Swap}_{36} \text{Swap}_{78} - \text{Swap}_{14} \text{Swap}_{23} \text{Swap}_{57} \text{Swap}_{68} + \text{Swap}_{14} \text{Swap}_{23} \text{Swap}_{56} \text{Swap}_{78} \\ &- \text{Swap}_{13} \text{Swap}_{26} \text{Swap}_{48} \text{Swap}_{57} + \text{Swap}_{13} \text{Swap}_{26} \text{Swap}_{47} \text{Swap}_{58} + \text{Swap}_{13} \text{Swap}_{25} \text{Swap}_{48} \text{Swap}_{67} \\ &- \text{Swap}_{13} \text{Swap}_{25} \text{Swap}_{47} \text{Swap}_{68} - \text{Swap}_{13} \text{Swap}_{24} \text{Swap}_{58} \text{Swap}_{67} + \text{Swap}_{13} \text{Swap}_{24} \text{Swap}_{57} \text{Swap}_{68} \\ &+ \text{Swap}_{12} \text{Swap}_{38} \text{Swap}_{47} \text{Swap}_{56} - \text{Swap}_{12} \text{Swap}_{38} \text{Swap}_{45} \text{Swap}_{67} - \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{47} \text{Swap}_{58} \\ &+ \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{45} \text{Swap}_{78} + \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{58} \text{Swap}_{67} - \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{56} \text{Swap}_{78} \\ &- \frac{1}{2} \text{Swap}_{38} \text{Swap}_{47} \text{Swap}_{56} + \frac{1}{2} \text{Swap}_{38} \text{Swap}_{45} \text{Swap}_{67} + \frac{1}{2} \text{Swap}_{36} \text{Swap}_{47} \text{Swap}_{58} - \frac{1}{2} \text{Swap}_{36} \text{Swap}_{45} \text{Swap}_{78} \\ &- \frac{1}{2} \text{Swap}_{34} \text{Swap}_{58} \text{Swap}_{67} + \frac{1}{2} \text{Swap}_{34} \text{Swap}_{56} \text{Swap}_{78} + \frac{1}{2} \text{Swap}_{27} \text{Swap}_{38} \text{Swap}_{56} - \frac{1}{2} \text{Swap}_{27} \text{Swap}_{38} \text{Swap}_{45} \\ &- \frac{1}{2} \text{Swap}_{27} \text{Swap}_{36} \text{Swap}_{58} + \frac{1}{2} \text{Swap}_{27} \text{Swap}_{36} \text{Swap}_{45} + \frac{1}{2} \text{Swap}_{27} \text{Swap}_{34} \text{Swap}_{58} - \frac{1}{2} \text{Swap}_{27} \text{Swap}_{34} \text{Swap}_{56} \\ &+ \frac{1}{2} \text{Swap}_{26} \text{Swap}_{48} \text{Swap}_{57} - \frac{1}{2} \text{Swap}_{26} \text{Swap}_{47} \text{Swap}_{58} - \frac{1}{2} \text{Swap}_{26} \text{Swap}_{38} \text{Swap}_{57} + \frac{1}{2} \text{Swap}_{26} \text{Swap}_{38} \text{Swap}_{47} \\ &+ \frac{1}{2} \text{Swap}_{26} \text{Swap}_{37} \text{Swap}_{58} - \frac{1}{2} \text{Swap}_{26} \text{Swap}_{37} \text{Swap}_{48} - \frac{1}{2} \text{Swap}_{25} \text{Swap}_{48} \text{Swap}_{67} + \frac{1}{2} \text{Swap}_{25} \text{Swap}_{47} \text{Swap}_{68} \\ &- \frac{1}{2} \text{Swap}_{25} \text{Swap}_{37} \text{Swap}_{68} + \frac{1}{2} \text{Swap}_{25} \text{Swap}_{37} \text{Swap}_{48} + \frac{1}{2} \text{Swap}_{25} \text{Swap}_{36} \text{Swap}_{78} - \frac{1}{2} \text{Swap}_{25} \text{Swap}_{36} \text{Swap}_{47} \end{aligned}$$

$-\frac{1}{2}$	Swap ₂₅	Swap ₃₄	Swap ₇₈	$+\frac{1}{2}$	Swap ₂₅	Swap ₃₄	Swap ₆₇	$+\frac{1}{2}$	Swap ₂₄	Swap ₅₈	Swap ₆₇	$-\frac{1}{2}$	Swap ₂₄	Swap ₅₇	Swap ₆₈
$-\frac{1}{2}$	Swap ₂₄	Swap ₃₈	Swap ₆₇	$+\frac{1}{2}$	Swap ₂₄	Swap ₃₈	Swap ₅₇	$+\frac{1}{2}$	Swap ₂₄	Swap ₃₇	Swap ₆₈	$-\frac{1}{2}$	Swap ₂₄	Swap ₃₇	Swap ₅₈
$+\frac{1}{2}$	Swap ₂₃	Swap ₅₇	Swap ₆₈	$-\frac{1}{2}$	Swap ₂₃	Swap ₅₆	Swap ₇₈	$+\frac{1}{2}$	Swap ₂₃	Swap ₄₈	Swap ₆₇	$-\frac{1}{2}$	Swap ₂₃	Swap ₄₈	Swap ₅₇
$-\frac{1}{2}$	Swap ₂₃	Swap ₄₇	Swap ₆₈	$+\frac{1}{2}$	Swap ₂₃	Swap ₄₇	Swap ₅₆	$+\frac{1}{2}$	Swap ₂₃	Swap ₄₅	Swap ₇₈	$-\frac{1}{2}$	Swap ₂₃	Swap ₄₅	Swap ₆₇
$+\frac{1}{2}$	Swap ₁₈	Swap ₄₇	Swap ₅₆	$-\frac{1}{2}$	Swap ₁₈	Swap ₄₅	Swap ₆₇	$-\frac{1}{2}$	Swap ₁₈	Swap ₃₆	Swap ₄₇	$+\frac{1}{2}$	Swap ₁₈	Swap ₃₆	Swap ₄₅
$+\frac{1}{2}$	Swap ₁₈	Swap ₃₄	Swap ₆₇	$-\frac{1}{2}$	Swap ₁₈	Swap ₃₄	Swap ₅₆	$-\frac{1}{2}$	Swap ₁₈	Swap ₂₇	Swap ₅₆	$+\frac{1}{2}$	Swap ₁₈	Swap ₂₇	Swap ₄₇
$+\frac{1}{2}$	Swap ₁₈	Swap ₂₇	Swap ₃₆	$-\frac{1}{2}$	Swap ₁₈	Swap ₂₇	Swap ₃₄	$+\frac{1}{2}$	Swap ₁₈	Swap ₂₅	Swap ₆₇	$-\frac{1}{2}$	Swap ₁₈	Swap ₂₅	Swap ₄₇
$-\frac{1}{2}$	Swap ₁₈	Swap ₂₅	Swap ₃₆	$+\frac{1}{2}$	Swap ₁₈	Swap ₂₅	Swap ₃₄	$-\frac{1}{2}$	Swap ₁₈	Swap ₂₃	Swap ₆₇	$+\frac{1}{2}$	Swap ₁₈	Swap ₂₃	Swap ₅₆
$+\frac{1}{2}$	Swap ₁₈	Swap ₂₃	Swap ₄₇	$-\frac{1}{2}$	Swap ₁₈	Swap ₂₃	Swap ₄₅	$-\frac{1}{2}$	Swap ₁₆	Swap ₄₈	Swap ₅₇	$+\frac{1}{2}$	Swap ₁₆	Swap ₄₅	Swap ₇₈
$+\frac{1}{2}$	Swap ₁₆	Swap ₃₈	Swap ₅₇	$-\frac{1}{2}$	Swap ₁₆	Swap ₃₈	Swap ₄₅	$-\frac{1}{2}$	Swap ₁₆	Swap ₃₇	Swap ₅₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₃₇	Swap ₄₈
$-\frac{1}{2}$	Swap ₁₆	Swap ₃₄	Swap ₇₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₃₄	Swap ₅₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₇	Swap ₅₈	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₇	Swap ₄₅
$-\frac{1}{2}$	Swap ₁₆	Swap ₂₇	Swap ₃₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₇	Swap ₃₄	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₅	Swap ₇₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₅	Swap ₄₈
$+\frac{1}{2}$	Swap ₁₆	Swap ₂₅	Swap ₃₇	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₅	Swap ₃₄	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₄	Swap ₅₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₄	Swap ₅₇
$+\frac{1}{2}$	Swap ₁₆	Swap ₂₄	Swap ₃₈	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₄	Swap ₃₇	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₃	Swap ₇₈	$-\frac{1}{2}$	Swap ₁₆	Swap ₂₃	Swap ₅₇
$-\frac{1}{2}$	Swap ₁₆	Swap ₂₃	Swap ₄₈	$+\frac{1}{2}$	Swap ₁₆	Swap ₂₃	Swap ₄₅	$+\frac{1}{2}$	Swap ₁₅	Swap ₄₈	Swap ₆₇	$-\frac{1}{2}$	Swap ₁₅	Swap ₄₇	Swap ₆₈
$-\frac{1}{2}$	Swap ₁₅	Swap ₃₈	Swap ₆₇	$+\frac{1}{2}$	Swap ₁₅	Swap ₃₈	Swap ₄₇	$+\frac{1}{2}$	Swap ₁₅	Swap ₃₇	Swap ₆₈	$-\frac{1}{2}$	Swap ₁₅	Swap ₃₇	Swap ₄₈
$-\frac{1}{2}$	Swap ₁₅	Swap ₂₆	Swap ₄₈	$+\frac{1}{2}$	Swap ₁₅	Swap ₂₆	Swap ₄₇	$+\frac{1}{2}$	Swap ₁₅	Swap ₂₆	Swap ₃₈	$-\frac{1}{2}$	Swap ₁₅	Swap ₂₆	Swap ₃₇
$+\frac{1}{2}$	Swap ₁₅	Swap ₂₄	Swap ₆₈	$-\frac{1}{2}$	Swap ₁₅	Swap ₂₄	Swap ₆₇	$-\frac{1}{2}$	Swap ₁₅	Swap ₂₄	Swap ₃₈	$+\frac{1}{2}$	Swap ₁₅	Swap ₂₄	Swap ₃₇
$-\frac{1}{2}$	Swap ₁₅	Swap ₂₃	Swap ₆₈	$+\frac{1}{2}$	Swap ₁₅	Swap ₂₃	Swap ₆₇	$+\frac{1}{2}$	Swap _{15</}						

$$\begin{aligned}
& -\frac{1}{2} \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{78} + \frac{1}{2} \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{58} + \frac{1}{2} \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{47} - \frac{1}{2} \text{Swap}_{12} \text{Swap}_{36} \text{Swap}_{45} \\
& + \frac{1}{2} \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{78} - \frac{1}{2} \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{67} - \frac{1}{2} \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{58} + \frac{1}{2} \text{Swap}_{12} \text{Swap}_{34} \text{Swap}_{56}.
\end{aligned}$$

To form a basis $\mathcal{B}_4$ for the linear space spanned by products of at most four swap matrices, one takes $\mathcal{B}_3$ and adds

(Types 7) For each 7-tuple $i < j < k < \ell < p < q < r$ a select 36 products of four swap matrices involving these seven indices;

(Type 8) a certain selection of 91 out of the 105 distinct products of four swap matrices on distinct indices $i < j < k < \ell < p < q < r < s$.

Thus

$$\begin{aligned}
\#\mathcal{B}_4 &= \#\mathcal{B}_3 + 36 \binom{n}{7} + \frac{91}{105} \frac{1}{4!} \binom{n}{2} \binom{n-2}{2} \binom{n-4}{2} \binom{n-6}{2} \\
&= \frac{1}{40320} (91n^8 - 2260n^7 + 24094n^6 - 138544n^5 + 460579n^4 \\
&\quad - 869260n^3 + 865956n^2 - 340656n + 40320).
\end{aligned}$$

C Gröbner basis for the Swap algebra $\mathcal{A}_4^{\text{swap}}$

Example C.1 ($n = 4$). The GB for $\mathcal{I}_4^{\text{swap}}$ has 34 elements, namely

$$\begin{aligned}
& -1 + s_{12}^2, 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{12}s_{23}, 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{12}s_{24}, \\
& 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{13}s_{12}, -1 + s_{13}^2, -s_{12}s_{13} + s_{13}s_{23}, 1 - s_{13} - s_{14} - s_{34} + s_{13}s_{14} + s_{13}s_{34}, \\
& 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{14}s_{12}, 1 - s_{13} - s_{14} - s_{34} + s_{13}s_{14} + s_{14}s_{13}, -1 + s_{14}^2, \\
& -s_{12}s_{14} + s_{14}s_{24}, -s_{13}s_{14} + s_{14}s_{34}, -s_{12}s_{13} + s_{23}s_{12}, \\
& 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{23}s_{13}, -s_{14}s_{23} + s_{23}s_{14}, -1 + s_{23}^2, \\
& 1 - s_{23} - s_{24} - s_{34} + s_{23}s_{24} + s_{23}s_{34}, -s_{12}s_{14} + s_{24}s_{12}, -s_{13}s_{24} + s_{24}s_{13}, \\
& 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{24}s_{14}, 1 - s_{23} - s_{24} - s_{34} + s_{23}s_{24} + s_{24}s_{23}, -1 + s_{24}^2, \\
& -s_{23}s_{24} + s_{24}s_{34}, -s_{12}s_{34} + s_{34}s_{12}, -s_{13}s_{14} + s_{34}s_{13}, \\
& 1 - s_{13} - s_{14} - s_{34} + s_{13}s_{14} + s_{34}s_{14}, -s_{23}s_{24} + s_{34}s_{23}, 1 - s_{23} - s_{24} - s_{34} + s_{23}s_{24} + s_{34}s_{24}, -1 + s_{34}^2, \\
& -1/2 + s_{12}/2 + s_{14}/2 + s_{23}/2 + s_{34}/2 - s_{12}s_{13}/2 - s_{12}s_{14}/2 - s_{12}s_{34}/2 - s_{13}s_{14}/2 + s_{13}s_{24}/2 - s_{14}s_{23}/2 - s_{23}s_{24}/2 + s_{12}s_{13}s_{14}, \\
& 1/2 - s_{12}/2 - s_{14}/2 + s_{23}/2 - s_{34}/2 - s_{12}s_{13}/2 + s_{12}s_{14}/2 + s_{12}s_{34}/2 + s_{13}s_{14}/2 - s_{13}s_{24}/2 - s_{14}s_{23}/2 - s_{23}s_{24}/2 + s_{12}s_{13}s_{24}, \\
& 1/2 - s_{12}/2 + s_{14}/2 - s_{23}/2 - s_{34}/2 + s_{12}s_{13}/2 - s_{12}s_{14}/2 + s_{12}s_{34}/2 - s_{13}s_{14}/2 - s_{13}s_{24}/2 - s_{14}s_{23}/2 + s_{23}s_{24}/2 + s_{12}s_{14}s_{23}, \\
& -1/2 + s_{12}/2 + s_{14}/2 + s_{23}/2 + s_{34}/2 - s_{12}s_{13}/2 - s_{12}s_{14}/2 - s_{12}s_{34}/2 - s_{13}s_{14}/2 + s_{13}s_{24}/2 - s_{14}s_{23}/2 - s_{23}s_{24}/2 + s_{13}s_{14}s_{23}, \\
& -1/2 + s_{12}/2 + s_{14}/2 + s_{23}/2 + s_{34}/2 - s_{12}s_{13}/2 - s_{12}s_{14}/2 - s_{12}s_{34}/2 - s_{13}s_{14}/2 + s_{13}s_{24}/2 - s_{14}s_{23}/2 - s_{23}s_{24}/2 + s_{14}s_{23}s_{24}
\end{aligned}$$

Notice that unlike in the $n = 3$ case of Proposition 5.8 also higher degree polynomials, namely cubics, appear.

Example C.2. The following table gathers some data about GBs for $\mathcal{A}_n^{\text{swap}}$ for small values of n .

n	size(GB)	highest degree polynomial in GB	number of highest degree polynomials in GB
5	110	3	35
6	305	4	10
7	620	4	102
8	1665	5	37

C.1 Gröbner bases for irreps $[4 - k, k]$

Under the irrep $[3, 1]$ the image of h_{K_4} is $8 I$. Under the irrep $[2, 2]$ the image is $12 I$. With this we can easily compute $\text{GB}_{3,1}$ and $\text{GB}_{2,2}$ with the help of a computer algebra system. The former is given in Eq. (C.1), and the latter in Eq. (C.2).

$$\begin{aligned}
& -2 + s_{12} + s_{13} + s_{14} + s_{23} + s_{24} + s_{34}, -1 + s_{12}^2, \\
& 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{12}s_{23}, 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{12}s_{24}, \\
& 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{13}s_{12}, -1 + s_{13}^2, -s_{12}s_{13} + s_{13}s_{23}, \\
& 1 - s_{13} - s_{24} + s_{13}s_{24}, 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{14}s_{12}, \\
& -1 + s_{12} + s_{23} + s_{24} + s_{13}s_{14} + s_{14}s_{13}, -1 + s_{14}^2, 1 - s_{14} - s_{23} + s_{14}s_{23}, \\
& -s_{12}s_{14} + s_{14}s_{24}, -s_{12}s_{13} + s_{23}s_{12}, 1 - s_{12} - s_{13} - s_{23} + s_{12}s_{13} + s_{23}s_{13}, \\
& 1 - s_{14} - s_{23} + s_{23}s_{14}, -1 + s_{23}^2, s_{13} - s_{24} - s_{12}s_{13} + s_{12}s_{14} - s_{13}s_{14} + \\
& s_{23}s_{24}, -s_{12}s_{14} + s_{24}s_{12}, 1 - s_{13} - s_{24} + s_{24}s_{13}, \\
& 1 - s_{12} - s_{14} - s_{24} + s_{12}s_{14} + s_{24}s_{14}, -1 + s_{12} + s_{14} + s_{24} + s_{12}s_{13} - \\
& s_{12}s_{14} + s_{13}s_{14} + s_{24}s_{23}, -1 + s_{24}^2, s_{13} - s_{12}s_{13} - s_{13}s_{14} + s_{12}s_{13}s_{14}
\end{aligned} \tag{C.1}$$

$$s_{12} + s_{13} + s_{14}, s_{12} + s_{13} + s_{23}, -s_{13} + s_{24}, -s_{12} + s_{34}, -1 + s_{12}^2, 1 + s_{12}s_{13} + s_{13}s_{12}, -1 + s_{13}^2 \tag{C.2}$$

D Example 4.8²

We present the second relaxation of Example 4.8. Let $n = 3$ and $d = 2$. Then

$$V_2(3) = (1, s_{12}, s_{13}, s_{23}, s_{12}^2, s_{12}s_{13}, s_{12}s_{23}, s_{13}s_{12}, s_{13}^2, s_{13}s_{23}, s_{23}s_{12}, s_{23}s_{13}, s_{23}^2)^*.$$

Thus $\mathcal{M}_2^3$ is the following 13×13 matrix

$$\begin{bmatrix}
1 & s_{12} & s_{13} & s_{23} & s_{12}^2 & s_{12}s_{13} & s_{12}s_{23} & s_{13}s_{12} & s_{13}^2 & s_{13}s_{23} & s_{23}s_{12} & s_{23}s_{13} & s_{23}^2 \\
s_{12} & s_{12}^2 & s_{12}s_{13} & s_{12}s_{23} & s_{12}^3 & s_{12}^2s_{13} & s_{12}^2s_{23} & s_{12}s_{13}s_{12} & s_{12}s_{13}^2 & s_{12}s_{13}s_{23} & s_{12}s_{23}s_{12} & s_{12}s_{23}s_{13} & s_{12}s_{23}^2 \\
s_{13} & s_{13}s_{12} & s_{13}^2 & s_{13}s_{23} & s_{13}s_{12}^2 & s_{13}s_{12}s_{13} & s_{13}s_{12}s_{23} & s_{13}^2s_{12} & s_{13}^2 & s_{13}^2s_{23} & s_{13}s_{23}s_{12} & s_{13}s_{23}s_{13} & s_{13}s_{23}^2 \\
s_{23} & s_{23}s_{12} & s_{23}s_{13} & s_{23}^2 & s_{23}s_{12}^2 & s_{23}s_{12}s_{13} & s_{23}s_{12}s_{23} & s_{23}s_{13}s_{12} & s_{23}s_{13}^2 & s_{23}s_{13}s_{23} & s_{23}^2s_{12} & s_{23}^2s_{13} & s_{23}^3 \\
s_{12}^2 & s_{12}^3 & s_{12}^2s_{13} & s_{12}^2s_{23} & s_{12}^4 & s_{12}^3s_{13} & s_{12}^3s_{23} & s_{12}^2s_{13}s_{12} & s_{12}^2s_{13}^2 & s_{12}^2s_{13}s_{23} & s_{12}^2s_{23}s_{12} & s_{12}^2s_{23}s_{13} & s_{12}^2s_{23}^2 \\
s_{13}s_{12} & s_{13}s_{12}^2 & s_{13}s_{12}s_{13} & s_{13}s_{12}s_{23} & s_{13}s_{12}^3 & s_{13}s_{12}^2s_{13} & s_{13}s_{12}^2s_{23} & s_{13}s_{12}s_{13}s_{12} & s_{13}s_{12}s_{13}^2 & s_{13}s_{12}s_{13}s_{23} & s_{13}s_{12}s_{23}s_{12} & s_{13}s_{12}s_{23}s_{13} & s_{13}s_{12}s_{23}^2 \\
s_{23}s_{12} & s_{23}s_{12}^2 & s_{23}s_{12}s_{13} & s_{23}s_{12}s_{23} & s_{23}s_{12}^3 & s_{23}s_{12}^2s_{13} & s_{23}s_{12}^2s_{23} & s_{23}s_{13}s_{12}s_{12} & s_{23}s_{13}s_{12}^2 & s_{23}s_{13}s_{12}s_{23} & s_{23}s_{13}s_{23}s_{12} & s_{23}s_{13}s_{23}s_{13} & s_{23}s_{13}s_{23}^2 \\
s_{12}s_{13} & s_{12}s_{13}s_{12} & s_{12}s_{13}^2 & s_{12}s_{13}s_{23} & s_{12}s_{13}s_{12}^2 & s_{12}s_{13}s_{12}s_{13} & s_{12}s_{13}s_{12}s_{23} & s_{12}s_{13}^2s_{12} & s_{12}s_{13}^2 & s_{12}s_{13}^2s_{23} & s_{12}s_{13}s_{23}s_{12} & s_{12}s_{13}s_{23}s_{13} & s_{12}s_{13}s_{23}^2 \\
s_{23}s_{13} & s_{23}s_{13}s_{12} & s_{23}s_{13}^2 & s_{23}s_{13}s_{23} & s_{23}s_{13}s_{12}^2 & s_{23}s_{13}s_{12}s_{13} & s_{23}s_{13}s_{12}s_{23} & s_{23}s_{13}^2s_{12} & s_{23}s_{13}^2 & s_{23}s_{13}^2s_{23} & s_{23}s_{13}s_{23}s_{12} & s_{23}s_{13}s_{23}s_{13} & s_{23}s_{13}s_{23}^2 \\
s_{12}s_{23} & s_{12}s_{23}s_{12} & s_{12}s_{23}s_{13} & s_{12}s_{23}^2 & s_{12}s_{23}s_{12}^2 & s_{12}s_{23}s_{12}s_{13} & s_{12}s_{23}s_{12}s_{23} & s_{12}s_{23}s_{13}s_{12} & s_{12}s_{23}s_{13}^2 & s_{12}s_{23}s_{13}s_{23} & s_{12}s_{23}^2s_{12} & s_{12}s_{23}^2s_{13} & s_{12}s_{23}^2 \\
s_{13}s_{23} & s_{13}s_{23}s_{12} & s_{13}s_{23}s_{13} & s_{13}s_{23}^2 & s_{13}s_{23}s_{12}^2 & s_{13}s_{23}s_{12}s_{13} & s_{13}s_{23}s_{12}s_{23} & s_{13}s_{23}s_{13}s_{12} & s_{13}s_{23}s_{13}^2 & s_{13}s_{23}s_{13}s_{23} & s_{13}s_{23}^2s_{12} & s_{13}s_{23}^2s_{13} & s_{13}s_{23}^2 \\
s_{23}^2 & s_{23}^2s_{12} & s_{23}^2s_{13} & s_{23}^3 & s_{23}^2s_{12}^2 & s_{23}^2s_{12}s_{13} & s_{23}^2s_{12}s_{23} & s_{23}^2s_{13}s_{12} & s_{23}^2s_{13}^2 & s_{23}^2s_{13}s_{23} & s_{23}^2s_{12}s_{12} & s_{23}^2s_{12}s_{13} & s_{23}^2s_{12}s_{23}
\end{bmatrix}$$

leading to the second relaxation which would be a 13×13 SDP. However, the Veronese $V_2(3)$ has redundancies. For instance, $s_{12}^2 = 1$, etc. Eliminating redundancies (more precisely, picking a basis for the image of $\mathbb{R}\langle s \rangle_2$ in $\mathbb{R}\langle s \rangle / \mathcal{I}^{S_3}$) leads to the reduced Veronese, which we shall by abuse of notation still call $V_2(3)$:

$$V_2(3) = (1, s_{12}, s_{13}, s_{23}, s_{12}s_{13}, s_{12}s_{23})^*.$$

This yields a significantly smaller moment matrix pattern,

$$\mathcal{M}_2^3 = \begin{bmatrix}
1 & s_{12} & s_{13} & s_{23} & s_{12}s_{13} & s_{12}s_{23} \\
s_{12} & s_{12}^2 & s_{12}s_{13} & s_{12}s_{23} & s_{12}^2s_{13} & s_{12}^2s_{23} \\
s_{13} & s_{13}s_{12} & s_{13}^2 & s_{13}s_{23} & s_{13}s_{12}s_{13} & s_{13}s_{12}s_{23} \\
s_{23} & s_{23}s_{12} & s_{23}s_{13} & s_{23}^2 & s_{23}s_{12}s_{13} & s_{23}s_{12}s_{23} \\
s_{13}s_{12} & s_{13}s_{12}^2 & s_{13}s_{12}s_{13} & s_{13}s_{12}s_{23} & s_{13}s_{12}^2s_{13} & s_{13}s_{12}^2s_{23} \\
s_{23}s_{12} & s_{23}s_{12}^2 & s_{23}s_{12}s_{13} & s_{23}s_{12}s_{23} & s_{23}s_{12}^2s_{13} & s_{23}s_{12}^2s_{23}
\end{bmatrix}$$

Applying the replacement rules of Example 4.8 simplifies this matrix further,

$$\mathcal{M}_2^3 = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{23} & s_{12}s_{13} & s_{12}s_{23} \\ s_{12} & 1 & s_{12}s_{13} & s_{12}s_{23} & s_{13} & s_{23} \\ s_{13} & s_{12}s_{23} & 1 & s_{12}s_{13} & s_{23} & s_{12} \\ s_{23} & s_{12}s_{13} & s_{12}s_{23} & 1 & s_{12} & s_{13} \\ s_{12}s_{23} & s_{13} & s_{23} & s_{12} & 1 & s_{12}s_{13} \\ s_{12}s_{13} & s_{23} & s_{12} & s_{13} & s_{12}s_{23} & 1 \end{bmatrix}$$

We thus obtain a smaller second level hierarchy SDP:

$$\begin{aligned} \alpha_2(h) = \max & \ell_{12} + \ell_{13} + \ell_{23} \\ \text{s.t.} & \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} & \ell_{12,13} & \ell_{12}\ell_{23} \\ \ell_{12} & 1 & \ell_{12,13} & \ell_{12,23} & \ell_{13} & \ell_{23} \\ \ell_{13} & \ell_{12,23} & 1 & \ell_{12,13} & \ell_{23} & \ell_{12} \\ \ell_{23} & \ell_{12,13} & \ell_{12,23} & 1 & \ell_{12} & \ell_{13} \\ \ell_{12,23} & \ell_{13} & \ell_{23} & \ell_{12} & 1 & \ell_{12}\ell_{13} \\ \ell_{12,13} & \ell_{23} & \ell_{12} & \ell_{13} & \ell_{12,23} & 1 \end{bmatrix} \succeq 0. \end{aligned} \quad (\text{D.1})$$

Since the matrix in Eq. (D.1) must be symmetric, we obtain $\ell_{12,23} = \ell_{12,13}$, reducing the number of unknowns by one,

$$\begin{aligned} \alpha_2(h) = \max & \ell_{12} + \ell_{13} + \ell_{23} \\ \text{s.t.} & \begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{23} & \ell_{12,13} & \ell_{12}\ell_{23} \\ \ell_{12} & 1 & \ell_{12,13} & \ell_{12,13} & \ell_{13} & \ell_{23} \\ \ell_{13} & \ell_{12,13} & 1 & \ell_{12,13} & \ell_{23} & \ell_{12} \\ \ell_{23} & \ell_{12,13} & \ell_{12,13} & 1 & \ell_{12} & \ell_{13} \\ \ell_{12,13} & \ell_{13} & \ell_{23} & \ell_{12} & 1 & \ell_{12}\ell_{13} \\ \ell_{12,13} & \ell_{23} & \ell_{12} & \ell_{13} & \ell_{12,13} & 1 \end{bmatrix} \succeq 0 \\ & = 3. \end{aligned} \quad (\text{D.2})$$

Finally, since $\dim \mathbb{C}[S_3] = 6$ is equal to the size of the reduced Veronese, the second relaxation $\alpha_2(h)$ is automatically exact.

Example D.1. Now consider $n = 4$, and $\mathcal{I} = \mathcal{I}^{S_4}$. In this case the replacement rules are⁹

$$\begin{aligned} s_{12}^2 &\rightarrow 1, s_{13}s_{12} \rightarrow s_{12}s_{23}, s_{13}^2 \rightarrow 1, s_{13}s_{23} \rightarrow s_{12}s_{13}, s_{14}s_{12} \rightarrow s_{12}s_{24}, s_{14}s_{13} \rightarrow s_{13}s_{34}, \\ s_{14}^2 &\rightarrow 1, s_{14}s_{24} \rightarrow s_{12}s_{14}, s_{14}s_{34} \rightarrow s_{13}s_{14}, s_{23}s_{12} \rightarrow s_{12}s_{13}, s_{23}s_{13} \rightarrow s_{12}s_{23}, \\ s_{23}s_{14} &\rightarrow s_{14}s_{23}, s_{23}^2 \rightarrow 1, s_{24}s_{12} \rightarrow s_{12}s_{14}, s_{24}s_{13} \rightarrow s_{13}s_{24}, s_{24}s_{14} \rightarrow s_{12}s_{24}, \\ s_{24}s_{23} &\rightarrow s_{23}s_{34}, s_{24}^2 \rightarrow 1, s_{24}s_{34} \rightarrow s_{23}s_{24}, s_{34}s_{12} \rightarrow s_{12}s_{34}, s_{34}s_{13} \rightarrow s_{13}s_{14}, \\ s_{34}s_{14} &\rightarrow s_{13}s_{34}, s_{34}s_{23} \rightarrow s_{23}s_{24}, s_{34}s_{24} \rightarrow s_{23}s_{34}, s_{34}^2 \rightarrow 1. \end{aligned} \quad (\text{D.3})$$

⁹In order to obtain a Gröbner basis (cf. Section 5) for the grlex order w.r.t. $s_{12} < s_{13} < s_{14} < s_{23} < s_{24} < s_{34}$, one needs to add three additional replacement rules to the ones given in Eq. (D.3), namely $s_{13}s_{14}s_{23} \rightarrow s_{12}s_{13}s_{14}$, $s_{14}s_{23}s_{24} \rightarrow s_{12}s_{13}s_{14}$, $s_{14}s_{23}s_{34} \rightarrow s_{12}s_{14}s_{23}$.

Then $V_1(4) = (1, s_{12}, s_{13}, s_{14}, s_{23}, s_{24}, s_{34})^*$, and

$$\mathcal{M}_1^4 = \begin{bmatrix} 1 & s_{12} & s_{13} & s_{14} & s_{23} & s_{24} \\ s_{12} & 1 & s_{12}s_{13} & s_{12}s_{14} & s_{12}s_{23} & s_{12}s_{24} \\ s_{13} & s_{12}s_{23} & 1 & s_{13}s_{14} & s_{12}s_{13} & s_{13}s_{24} \\ s_{14} & s_{12}s_{24} & s_{13}s_{34} & 1 & s_{14}s_{23} & s_{12}s_{14} \\ s_{23} & s_{12}s_{13} & s_{12}s_{23} & s_{14}s_{23} & 1 & s_{23}s_{24} \\ s_{24} & s_{12}s_{14} & s_{13}s_{24} & s_{12}s_{24} & s_{23}s_{34} & 1 \end{bmatrix}$$

leading to the SDP constraint

$$\begin{bmatrix} 1 & \ell_{12} & \ell_{13} & \ell_{14} & \ell_{23} & \ell_{24} \\ \ell_{12} & 1 & \ell_{12,13} & \ell_{12,14} & \ell_{12,23} & \ell_{12,24} \\ \ell_{13} & \ell_{12,23} & 1 & \ell_{13,14} & \ell_{12,13} & \ell_{13,24} \\ \ell_{14} & \ell_{12,24} & \ell_{13,34} & 1 & \ell_{14,23} & \ell_{12,14} \\ \ell_{23} & \ell_{12,13} & \ell_{12,23} & \ell_{14,23} & 1 & \ell_{23,24} \\ \ell_{24} & \ell_{12,14} & \ell_{13,24} & \ell_{12,24} & \ell_{23,34} & 1 \end{bmatrix} \succeq 0.$$

As before, symmetry of this matrix yields a few additional linear constraints, namely

$$\ell_{12,13} = \ell_{12,23}, \ell_{12,14} = \ell_{12,24}, \ell_{13,14} = \ell_{13,34}.$$

The full $V_2(4)$ has 43 entries, and its reduced form is

$$V_2(4) = (1, s_{12}, s_{13}, s_{14}, s_{23}, s_{24}, s_{34}, s_{12}s_{13}, s_{12}s_{14}, s_{12}s_{23}, s_{12}s_{24}, s_{12}s_{34}, s_{13}s_{14}, s_{13}s_{24}, s_{13}s_{34}, s_{14}s_{23}, s_{23}s_{24}, s_{23}s_{34})^*,$$

leading to the 18×18 moment matrix pattern

$$\begin{bmatrix} 1 & s_{12} & s_{13} & s_{14} & s_{23} & s_{24} & s_{34} & s_{12}s_{13} & s_{12}s_{14} & s_{12}s_{23} & s_{12}s_{24} & s_{12}s_{34} & s_{13}s_{14} & s_{13}s_{24} & s_{13}s_{34} & s_{14}s_{23} & s_{23}s_{24} & s_{23}s_{34} \\ s_{12} & 1 & s_{12}s_{13} & s_{12}s_{14} & s_{12}s_{23} & s_{12}s_{24} & s_{12}s_{34} & s_{13} & s_{14} & s_{23} & s_{24} & s_{34} & s_{12}s_{13}s_{14} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{12}s_{14}s_{23} & s_{12}s_{14}s_{24} & s_{12}s_{14}s_{34} \\ s_{13} & s_{12}s_{23} & 1 & s_{13}s_{14} & s_{12}s_{13} & s_{13}s_{24} & s_{13}s_{34} & s_{23} & s_{12}s_{14}s_{23} & s_{12} & s_{12}s_{23}s_{24} & s_{12}s_{23}s_{34} & s_{14} & s_{24} & s_{34} & s_{12}s_{13}s_{14} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} \\ s_{14} & s_{12}s_{24} & s_{13}s_{34} & 1 & s_{14}s_{23} & s_{12}s_{14} & s_{13}s_{14} & s_{12}s_{13}s_{24} & s_{24} & s_{12}s_{23}s_{34} & s_{12} & s_{12}s_{23}s_{24} & s_{34} & s_{12}s_{13}s_{34} & s_{13} & s_{23} & s_{12}s_{13}s_{14} & s_{12}s_{14}s_{23} \\ s_{23} & s_{12}s_{13} & s_{12}s_{23} & s_{14}s_{23} & 1 & s_{23}s_{24} & s_{23}s_{34} & s_{12} & s_{12}s_{13}s_{14} & s_{13} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{12}s_{14}s_{23} & s_{12}s_{14}s_{24} & s_{12}s_{14}s_{34} & s_{14} & s_{24} & s_{34} \\ s_{24} & s_{12}s_{14} & s_{13}s_{24} & s_{12}s_{24} & s_{23}s_{34} & 1 & s_{23}s_{24} & s_{12}s_{13}s_{34} & s_{12} & s_{12}s_{14}s_{23} & s_{14} & s_{12}s_{13}s_{14} & s_{12}s_{23}s_{24} & s_{13} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{34} & s_{23} \\ s_{34} & s_{12}s_{34} & s_{13}s_{14} & s_{13}s_{24} & s_{23}s_{34} & s_{23}s_{24} & 1 & s_{12}s_{13}s_{14} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{12}s_{23}s_{24} & s_{12}s_{23}s_{34} & s_{12} & s_{13} & s_{12}s_{14}s_{23} & s_{14} & s_{12}s_{13}s_{24} & s_{23} & s_{24} \\ s_{12}s_{23} & s_{13} & s_{23} & s_{12}s_{14}s_{23} & s_{12} & s_{12}s_{23}s_{24} & s_{12}s_{23}s_{34} & 1 & s_{13}s_{14} & s_{12}s_{13} & s_{13}s_{24} & s_{13}s_{34} & s_{14}s_{23} & s_{23}s_{24} & s_{23}s_{34} & s_{12}s_{14} & s_{12}s_{24} & s_{12}s_{34} \\ s_{12}s_{24} & s_{14} & s_{12}s_{13}s_{24} & s_{24} & s_{12}s_{23}s_{34} & s_{12} & s_{12}s_{23}s_{24} & s_{13}s_{34} & 1 & s_{14}s_{23} & s_{12}s_{14} & s_{13}s_{14} & s_{23}s_{24} & s_{12}s_{13} & s_{13}s_{24} & s_{23}s_{34} & s_{12}s_{34} & s_{12}s_{23} \\ s_{12}s_{13} & s_{23} & s_{12} & s_{12}s_{13}s_{14} & s_{13} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{12}s_{23} & s_{14}s_{23} & 1 & s_{23}s_{24} & s_{23}s_{34} & s_{12}s_{14} & s_{12}s_{24} & s_{12}s_{34} & s_{13}s_{14} & s_{13}s_{24} & s_{13}s_{34} \\ s_{12}s_{14} & s_{24} & s_{12}s_{13}s_{34} & s_{12} & s_{12}s_{14}s_{23} & s_{14} & s_{12}s_{13}s_{14} & s_{13}s_{24} & s_{12}s_{24} & s_{23}s_{34} & 1 & s_{23}s_{24} & s_{12}s_{34} & s_{13}s_{34} & s_{12}s_{13} & s_{14}s_{23} & s_{13}s_{14} & s_{14}s_{23} \\ s_{12}s_{34} & s_{34} & s_{12}s_{13}s_{14} & s_{12}s_{13}s_{24} & s_{12}s_{23}s_{34} & s_{12} & s_{13}s_{14} & s_{13}s_{34} & s_{23}s_{24} & s_{23}s_{34} & s_{12}s_{24} & s_{12}s_{34} & s_{12}s_{23} & 1 & s_{12}s_{14} & s_{12}s_{24} & s_{12}s_{34} & s_{12}s_{23} \\ s_{13}s_{14} & s_{12}s_{23}s_{34} & s_{14} & s_{34} & s_{12}s_{13}s_{24} & s_{12}s_{13}s_{34} & s_{13} & s_{14}s_{23} & s_{23}s_{34} & s_{12}s_{24} & s_{12}s_{34} & s_{12}s_{23} & s_{12}s_{14} & s_{13}s_{14} & s_{23}s_{24} & s_{12}s_{13} & s_{13}s_{24} & s_{13}s_{34} \\ s_{13}s_{24} & s_{12}s_{14}s_{23} & s_{24} & s_{12}s_{23}s_{24} & s_{12}s_{13}s_{34} & s_{13} & s_{12}s_{13}s_{24} & s_{23}s_{34} & s_{12}s_{23} & s_{12}s_{14} & s_{13}s_{14} & s_{14}s_{23} & s_{12}s_{24} & 1 & s_{23}s_{24} & s_{12}s_{34} & s_{13}s_{34} & s_{12}s_{13} \\ s_{13}s_{14} & s_{12}s_{23}s_{24} & s_{34} & s_{13} & s_{12}s_{13}s_{14} & s_{12}s_{14}s_{23} & s_{14} & s_{23}s_{24} & s_{13}s_{24} & s_{12}s_{34} & s_{12}s_{23} & s_{12}s_{24} & s_{13}s_{34} & s_{23}s_{34} & 1 & s_{12}s_{13} & s_{14}s_{23} & s_{12}s_{14} \\ s_{14}s_{23} & s_{12}s_{13}s_{24} & s_{12}s_{23}s_{34} & s_{23} & s_{14} & s_{12}s_{13}s_{14} & s_{12}s_{14}s_{23} & s_{12}s_{24} & s_{23}s_{24} & s_{13}s_{34} & s_{12}s_{13} & s_{13}s_{24} & s_{23}s_{34} & s_{12}s_{34} & s_{12}s_{23} & 1 & s_{12}s_{14} & s_{13}s_{14} \\ s_{23}s_{34} & s_{12}s_{13}s_{34} & s_{12}s_{14}s_{23} & s_{12}s_{23}s_{34} & s_{24} & s_{34} & s_{23} & s_{12}s_{14} & s_{12}s_{34} & s_{13}s_{24} & s_{13}s_{34} & s_{12}s_{13} & s_{12}s_{23} & s_{13}s_{14} & s_{14}s_{23} & s_{12}s_{24} & 1 & s_{23}s_{24} \\ s_{23}s_{24} & s_{12}s_{13}s_{14} & s_{12}s_{23}s_{24} & s_{12}s_{13}s_{24} & s_{34} & s_{23} & s_{24} & s_{12}s_{34} & s_{12}s_{13} & s_{13}s_{14} & s_{14}s_{23} & s_{12}s_{14} & s_{13}s_{24} & s_{12}s_{23} & s_{12}s_{24} & s_{13}s_{34} & s_{23}s_{34} & 1 \end{bmatrix}$$

We leave the construction of the corresponding SDP constraint to the interested reader.

E Comparison with [TRZ⁺]

An independent and simultaneously released work ([TRZ⁺]) gives results that share some commonalities with those presented in this paper. These include construction of a new semidefinite programming hierarchy based on the swap matrices and exact solutions to Quantum Max Cut on specific graphs. Here we briefly compare and contrast these two papers.

First we outline key differences in notation between the two papers. Before defining a new semidefinite programming hierarchy, both papers introduce some additional formalism

in order to focus on the algebraic structure of the swap matrices. However, the authors of [TRZ⁺] state their results primarily in terms of *operator programs*, while we work with **-algebras* and *positive linear functionals* (*operator algebraic states*) defined on them. Moving back and forth between these two concepts is straightforward. In the language of operator programs, one considers maximizing (or minimizing) an expression of the form

$$\langle \psi | \theta(\{a_i\}) | \psi \rangle \quad (\text{E.1})$$

where the $\{a_i\}$ are non-commuting and self-adjoint operator variables, the ψ are arbitrary vectors satisfying $|\psi\rangle\langle\psi| = 1$, and the $\{a_i\}$ satisfy constraints

$$\eta_j(\{a_i\}) = 0 \quad \forall j \quad (\text{E.2})$$

for some set of constraint polynomials $\{\eta_j\}$. In the language of *-algebras, one first defines a *-algebra $\mathcal{A}$ to be the algebra generated by self adjoint nc variables $\{a_i\}$ which satisfy relations

$$\eta_j(\{a_i\}) = 0 \quad \forall j \quad (\text{E.3})$$

then computes the maximum over positive linear functionals $L : \mathcal{A} \rightarrow \mathbb{C}$ with $L(1) = 1$ of the expression

$$L(\theta(\{a_i\})). \quad (\text{E.4})$$

That these two formulations are equivalent follows from the GNS construction.

Both this paper and [TRZ⁺] give a set of constraints $\{\eta_j\}$ which characterize the swap matrices. Formally, the papers prove that maximizing an expression of the form given in Equation (E.1) (equivalently Equation (E.4)) subject to these constraints is equivalent to computing the max eigenvalue of the matrix $\theta(\text{SWAP})$ obtained by replacing the $\{a_i\}$ variables in $\theta(\{a_i\})$ with explicit swap matrices. In [TRZ⁺] the operator program obtained when the $\{a_i\}$ variables (instead denoted $\{p_{ij}\}$) are subject to these constraints is denoted $\mathcal{Perm}(V, w)$. In this paper the *-algebra obtained when the $\{a_i\}$ variables (instead denoted $\{s_{ij}\}$) are subject to these constraints is called the *Symbolic Swap Algebra*.

Both papers also introduce hierarchies of semidefinite programs which give a converging series of upper bounds on Equation (E.1) (equivalently Equation (E.4)). At each finite level both hierarchies construct a linear “pseudoexpectation”, denoted here by $\tilde{\mathbb{E}}$, which assigns values (“pseudomoments”) to all polynomials of degree $\leq 2d$ in the variables $\{a_i\}$. In both papers, these pseudoexpectations satisfy

$$\tilde{\mathbb{E}}(1) = 1 \quad \text{and} \quad \tilde{\mathbb{E}}(q^*q) \geq 0 \quad (\text{E.5})$$

for any nc polynomial q of degree at most d . However, there is an important difference between how strictly the pseudoexpectations constructed in the two papers enforce the constraints $\{\eta_j\}$. The pseudoexpectation for the d -th level hierarchy constructed in this paper – which we call the d -th swap relaxation to Quantum Max Cut– satisfies $\tilde{\mathbb{E}}[p] = 0$ for any degree $\leq 2d$ polynomial in the two sided ideal generated by $\{\eta_j\}$, so,

$$\tilde{\mathbb{E}}[p] = 0 \text{ if } \deg(p) \leq 2d \text{ and } \exists \text{ monomials } \{\beta_{ij}\}, \{\gamma_{ij}\} \text{ s.t. } p = \sum_{i,j} \beta_{ij} \eta_j \gamma_{ij}. \quad (\text{E.6})$$

Contrastingly, the pseudoexpectation for the d -th level hierarchy constructed in [TRZ⁺] only enforces the constraints in the *truncated ideal* generated by $\{\eta_j\}$, meaning

$$\tilde{\mathbb{E}}[p] = 0 \text{ if } \deg(p) \leq 2d \text{ and } \exists \text{ monomials } \{\beta_{ij}\}, \{\gamma_{ij}\} \text{ s.t. } p = \sum_{i,j} \beta_{ij} \eta_j \gamma_{ij}$$

$$\text{with } \deg(\beta_{ij}\eta_j\gamma_{ij}) \leq 2d \text{ for all } i, j. \quad (\text{E.7})$$

In the case of swaps the polynomials p defined in Equation (E.6) above could also be defined to be those p of degree $\leq 2d$ which annihilate all of the swap matrices, namely the set of polynomials p in the variables s_{ij} such that

$$p(\{\text{Swap}_{ij}\}) = 0 \quad (\text{E.8})$$

where Swap_{ij} is the matrix obtained by replacing each s_{ij} variable with the corresponding Swap matrix. This follows from Proposition 3.5 in our paper or [TRZ⁺, Theorem 3.8]. The ideal corresponding to the swaps can be defined by different sets of constraint polynomials. Equation (E.8) implies the value obtained by our d th relaxation depends only on the ideal and does not depend on the choice of defining constraints $\{\eta_j\}$. On the other hand, this property may not be shared by d -th relaxed value of [TRZ⁺].

Further discussion of this difference is given in Remark 4.3 in this paper, where the bound obtained at level d of our hierarchy is denoted $\nu_d(h)$ and the bound obtained by the hierarchy in [TRZ⁺] is denoted ν_d . While both hierarchies converge after finitely many steps, this difference means that at each level the hierarchy presented in this paper gives an upper bound that may be tighter than the one presented in [TRZ⁺].

We found numerically (up to SDP accuracy in Mathematica, 10^{-7}) for graphs with up to 8 vertices and uniform edge weights the value of our second swap relaxation equals the numerically exact max/min eigenvalue of the QMC Hamiltonian. In contrast, [TRZ⁺] do an impressive numerical study on a sampling of these same graphs and find the performance of their first relaxation in swaps is not numerically exact (to the same tolerance) on numerous graphs with 6, 7 and 8 vertices, for details see [TRZ⁺, Figure 4 of Section 5].

As mentioned in Section 1.2, computing all constraints of the form given in Equation (E.6) at higher levels d requires considerable technical work. This is the content of the later part of Section 4 of our paper (which gives an explicit linear algebraic basis for the degree d monomials in the Symbolic Swap Algebra when $d \leq 4$) and of Section 5 (which applies Gröbner Basis to swap polynomials). These techniques for simplifying swap polynomials are not present in [TRZ⁺]. One consequence of our swap algebra theory, see Theorem 4.10, is that the level $\lceil \frac{n}{2} \rceil$ swap relaxation in our hierarchy solves the quantum max cut problem exactly. The corresponding result for the hierarchy present in [TRZ⁺] is stated as an open question after Proposition 3.13 in [TRZ⁺], which gives the bound $\binom{n}{2}$.

The present paper and [TRZ⁺] both theoretically analyze specific instances where the highest eigenvalue of Quantum Max Cut can be computed exactly, but with diverging aims. We provide a method to compute (in exact arithmetic) the maximum eigenvalue of the QMC Hamiltonian on a certain family of graphs with uniform edge-weights, extending previously known results on exact solutions to Quantum Max Cut [LM62]. This result builds on the characterization of the QMC Hamiltonian in terms of irreducible representations of the symmetric group via Schur-Weyl duality, and is largely independent of our construction of the swap relaxations.

[TRZ⁺], on the other hand, is concerned with understanding instances where the SDP relaxations exactly solve the Quantum Max Cut problem as this can improve the analysis of approximation algorithms. Specifically, [TRZ⁺] investigates whether either the first swap relaxation or the second quantum Lasserre relaxation over Paulis are exact or not on a number of graphs. Interestingly, there are cases where the SDP hierarchies considered by [TRZ⁺] are proven to be inexact (see Section 4 of [TRZ⁺]) but for which exact arithmetic solutions are known. An example of this is the QMC Hamiltonian with uniform edge-weights on a clique with an odd number of vertices, which have exact solutions via representation

theory (see, for example, Section 2.5 of this paper). One special class of graphs solved by our methods are complete k -partite graphs for any constant k , special cases of which include the clique and the crown graph. These latter graphs are shown to be exactly computable by the SDPs in [TRZ⁺]. In other cases, such as a uniformly weighted double-star graph and a star graph with positive weights, the SDP relaxations are shown to be numerically exact but our representation theoretic method is unable to provide an exact solution.